

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

До захисту допущено:
В.о. завідувача кафедри

_____ **Артем ВОЛОКИТА**
(підпис)
_____ 2026 р.
(число) (місяць)

Дипломний проєкт
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Комп'ютерні системи та мережі»
спеціальність 123 — «Комп'ютерна інженерія»

**СИСТЕМА ПРОСТОРОВОГО АНАЛІЗУ ГРИ В ВОЛЕЙБОЛ З
ВИКОРИСТАННЯМ МАШИННОГО НАВЧАННЯ**

Виконав

студент 4 курсу, група ІО-21, Безщасний Роман Русланович _____
(підпис)

Керівник

аспірант, Морозов-Леонов Олександр Святославович _____
(підпис)

Консультант

док. філос., ас. кафедри ОТ, Міщенко Людмила Дмитрівна _____
(підпис)

Рецензент

доц., к.т.н., доцент кафедри СПСКС, Клятченко Ярослав _____
(підпис)

Засвідчую, що у цьому дипломному проєкті немає запозичень
з праць інших авторів без відповідних посилань

_____ **Безщасний Р.Р.**
(підпис)

Київ — 2026 р.

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Освітньо-кваліфікаційний рівень: Бакалавр

Освітньо-професійна програма: «Комп'ютерні системи та мережі»

Спеціальність: 123 — «Комп'ютерна інженерія»

ЗАТВЕРДЖУЮ

В.о. завідувача кафедри

_____ **Артем ВОЛОКИТА**

_____ 2026 р.

ЗАВДАННЯ

на бакалаврський дипломний проєкт

Безщасному Роману Руслановичу

1. Тема проєкту: Система просторового аналізу гри в волейбол з використанням машинного навчання
керівник Морозов-Леонов Олександр Святославович, аспірант
затверджено наказом по університету від 03 червня 2026 року № 2055-с
2. Дата здачі закінченого проєкту 05 червня 2026 року
3. Вихідні дані для проєкту: технічна документація та статистичні дані.
4. Зміст розрахунково-пояснювальної записки: Аналіз теорії та обґрунтування теми. Математичне та алгоритмічне забезпечення аналітичної системи. Програмна реалізація системи. Аналіз результатів та експериментальні дослідження.

5. Перелік графічного матеріалу: Схема структурна. Схема функціональна. Схема алгоритму.

6. Консультанти розділів проєкту

№	Розділ	Консультант	Підпис, дата	
			завдання видав	завдання прийняв
1	Нормоконтроль	Міщенко Л. Д.		

7. Дата видачі завдання 10 травня 2026 року

Календарний план

№	Назва етапу виконання дипломного проєкту	Термін виконання	Примітка
1	Затвердження теми проєкту	06.04.2026–07.06.2026	
2	Аналіз предметної області	01.03.2026–30.04.2026	
3	Розробка архітектури та загальної структури системи	10.04.2026–10.05.2026	
4	Програмна реалізація системи	20.04.2026 - 25.05.2026	
5	Оформлення пояснювальної записки	18.05.2026-30.05.2026	
6	Передзахист	01.06.2026-06.06.2026	
7	Захист	19.06.2026	

Виконавець _____ Безщасний Роман Русланович

Керівник _____ Морозов-Леонов Олександр Святославович

АНОТАЦІЯ

Актуальність теми. Роботу присвячено розробці програмно-алгоритмічного комплексу для автоматизованого відстеження волейбольного м'яча та розрахунку його кінематичних параметрів (швидкості, 3D-траєкторії) на основі монокулярного відеопотоку.

Систему побудовано на моделі глибокого навчання YOLO для детекції м'яча в умовах розмиття рухом. Перехід від піксельних координат до метричних реалізовано через модель камери-обскури з алгоритмом калібрування PnP та зворотним проектуванням. Для фільтрації шуму та компенсації аеродинамічних ефектів застосовано сигма-точковий фільтр Калмана, а для відстеження різких змін швидкості під час атакуючих ударів — архітектуру взаємодіючих множинних моделей.

Експериментальне тестування підтвердило здатність системи реконструювати 3D-траєкторію та фіксувати пікову швидкість удару однією камерою стандартної частоти (50–60 FPS). Результати можуть використовуватися тренерськими штабами для тактико-технічного аналізу.

Ключові слова: *YOLO, калібрування PnP, зворотне проектування, сигма-точковий фільтр Калмана, взаємодіючі множинні моделі, 3D-траєкторія, пікова швидкість удару.*

ANNOTATION

The thesis presents a software and algorithmic complex for automated volleyball tracking and calculation of its kinematic parameters (speed, 3D trajectory) from a monocular video stream.

The system is based on the YOLO deep learning model for ball detection under motion blur. The transition from pixel to metric coordinates uses a pinhole camera model with PnP calibration and Ray Casting. To filter noise and compensate for aerodynamic effects, an Unscented Kalman Filter is applied, while abrupt speed changes during attacking strikes are tracked with an Interacting Multiple Models architecture.

Experimental testing confirmed the system's ability to reconstruct the 3D trajectory and capture peak strike speed using a single standard frame-rate camera (50–60 FPS).

Keywords: *YOLO, PnP calibration, Ray Casting, Unscented Kalman Filter, Interacting Multiple Models, 3D trajectory, peak strike speed.*

[illegible]

**СИСТЕМА ПРОСТОРОВОГО АНАЛІЗУ ГРИ В ВОЛЕЙБОЛ З
ВИКОРИСТАННЯМ МАШИННОГО НАВЧАННЯ**

Технічне завдання

ІАЛЦ.467200.002 ТЗ

ЗМІСТ

1.	НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ	2
2.	ПІДСТАВИ ДЛЯ РОЗРОБКИ	2
3.	МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ	2
4.	ДЖЕРЕЛА РОЗРОБКИ	3
5.	ТЕХНІЧНІ ВИМОГИ	3
5.1.	Вимоги до вхідних даних	3
5.2.	Вимоги до функціональних модулів	3
5.3.	Вимоги до апаратної частини	4
6.	ЕТАПИ РОЗРОБКИ	4
6.1	Аналітичний етап	4
6.2	Нейромережевий етап	5
6.3	Математичний етап	5
6.4	Інтеграційний етап	5
6.5	Документаційний етап	5

					<i>ІАЛЦ.467200.002 ТЗ</i>			
<i>Зм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Підп.</i>	<i>Дата</i>				
<i>Розробив</i>	<i>Безщасний Р.Р.</i>				<i>Система просторового аналізу гри в волейбол з використанням машинного навчання</i>			
<i>Перевірів</i>	<i>Морозов-Леонов</i>							
<i>Н. контр.</i>	<i>Міщенко Л. Д.</i>							
<i>Затвердив</i>	<i>Волокита А. М.</i>							
						<i>Лит.</i>	<i>Аркуш</i>	<i>Аркушів</i>
							1	5
						КПІ ім. Ігоря Сікорського, ФІОТ, ІО-21		

1. НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ

В цьому технічному завданні охоплено опис розробки та впровадження програмно-алгоритмічного комплексу комп'ютерного зору для аналітики волейбольних подій, завдяки аналізу позиції м'яча в тривимірному просторі

Областю застосування цієї системи є спортивна аналітика, автоматизований збір статистики тренувальних та змагальних процесів у волейболі, а також інструментарій для тактико-технічного аналізу та оцінки кінематики рухів гравців тренерськими штабами аматорських і професійних ліг.

2. ПІДСТАВИ ДЛЯ РОЗРОБКИ

Першою підставою для розробки є завдання на виконання бакалаврської кваліфікаційної роботи рівня «бакалавр комп'ютерної інженерії», який був затверджений факультетом «Інформатики та обчислювальної техніки» кафедрою обчислювальної техніки Національного технічного Університету України «Київський Політехнічний інститут ім. Ігоря Сікорського».

А другою підставою є необхідність створення актуальної мобільної та обчислювально доступної альтернативи дорогим багатокамерним стаціонарним комплексам, що здатна функціонувати на основі монокулярних відеопотоків із побутових камер.

3. МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ

Метою розробки є створення високоточної системи для розрахунку кінематичних метрик волейбольного м'яча (тривимірної траєкторії, пікової швидкості атакуючої дії, точок відскоку) з усуненням апаратних шумів та впливу розмиття рухом.

Призначенням розробки є програмний комплекс для автоматизації процесу перетворення сирих двовимірних відеоданих у фізичну метричну си-

стему тривимірного простору з використанням сучасних методів глибокого навчання та ймовірнісного нелінійного згладжування.

4. ДЖЕРЕЛА РОЗРОБКИ

Джерелами розробки є

- наукова й технічна література з методів комп’ютерного зору, перспективної просторової геометрії, багатокритерійного трекінгу та теорії оптимального оцінювання станів динамічних систем;
- Офіційна документація алгоритмів об’єктної детекції архітектури YOLO;
- Відкриті програмні бібліотеки та фреймворки (OpenCV, Ultralytics, FilterPy, NumPy, PyTorch).

5. ТЕХНІЧНІ ВИМОГИ

5.1. Вимоги до входних даних

- Система повинна обробляти відеопотоки стандартних форматів (MP4, AVI) з роздільною здатністю не менше 1920x1080 пікселів.
- Мінімально допустима частота кадрів для коректного розрахунку динаміки – 50 FPS(з використанням методів програмного вирівнювання VFR у CFR).

5.2. Вимоги до функціональних модулів

- *Модуль детекції:* Використання без’якірної моделі YOLO26m (сімейства YOLO26) зі збільшеним розміром входного тензора (imgsz=1088) для детекції дрібних об’єктів.

					ІАЛЦ.467200.002 ТЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		3

- *Геометричний модуль*: Реалізація перетворення 2D-пікселів у 3D-промені(Ray Casting) на основі алгоритму просторового калібрування PnP(solvePnP) із механізмом інверсії координатних осей під балістичну модель.
- *Модуль трекінгу*: Адаптований алгоритм 3D ByteTrack з двостадійною асоціацією даних та використанням статистичної відстані Махаланобіса замість IoU.
- *Модуль фільтрації*: Нелінійне згладжування траєкторій за допомогою багатомодельної архітектури IMMу поєднанні з сигматочковими фільтрами Калмана (UKF). Наявність захисту від обчислювальних аномалій(вибух коваріації).

5.3. Вимоги до апаратної частини

- Операційна система: Windows 10/11 або дистрибутиви Linux(Ubuntu).
- Мова програмування: Python версії 3.11 або вище.
- Апаратна частина: багатоядерний центральний процесор (CPU) та графічний прискорювач (GPU) із підтримкою технології паралельних обчислень CUDA для забезпечення мілісекундного інференсу нейромережі.

6. ЕТАПИ РОЗРОБКИ

6.1 Аналітичний етап

Вивчення предметної області, збір власних відеоданих, їх попереднє оброблення та підготовка датасету.

					ІАЛЦ.467200.002 ТЗ	Аркуш
						4
Зм.	Лист	№ докум.	Підп.	Дата		

6.2 Нейромережевий етап

Проектування архітектури пайплайну, донавчання та валідація моделі YOLO на хмарних обчислювальних потужностях.

6.3 Математичний етап

Програмна реалізація алгоритмів Ray Casting, конфігурація матриць шуму та переходу для моделей IMM-UKF, розробка просторового трекера 3D ByteTrack.

6.4 Інтеграційний етап

Об'єднання розроблених модулів у єдиний конвеєр, проведення експериментів на реальних ігрових епізодах, отримання кінематичної статистики.

6.5 Документаційний етап

Оформлення вихідного коду, візуалізація графіків результатів та написання тексту пояснювальної записки.

					ІАЛЦ.467200.002 ТЗ	Аркуш
						5
Зм.	Лист	№ докум.	Підп.	Дата		

**СИСТЕМА ПРОСТОРОВОГО АНАЛІЗУ ГРИ В ВОЛЕЙБОЛ З
ВИКОРИСТАННЯМ МАШИННОГО НАВЧАННЯ**

Пояснювальна записка

ІАЛЦ.467200.003 ПЗ

ЗМІСТ

Перелік умовних скорочень	4
---------------------------	---

Вступ	5
-------	---

1 Аналіз теорії та обґрунтування теми	9
---------------------------------------	---

1.1 Еволюція методів виявлення рухомих об'єктів у відеопотоці .	9
---	---

1.2 Огляд та аналіз еволюції архітектур сімейства YOLO	12
--	----

1.3 Обмеження проєкцій двовимірного простору та математичне обґрунтування переходу до 3D	15
---	----

1.4 Проблема багатокритерійного трекінгу у 3D просторі	18
--	----

1.5 Порівняння методів фільтрації нелінійних динамічних систем	20
--	----

1.6 Особливості предметної області волейболу як цілі трекінгу . .	23
---	----

1.7 Монокулярне виродження глибини як ключовий виклик системи	26
---	----

1.8 Постановка задачі та формування вимог до системи	28
--	----

ВИСНОВКИ ДО РОЗДІЛУ 1	30
-----------------------	----

2 Математичне та алгоритмічне забезпечення аналітичної системи	32
--	----

2.1 Структура та граф взаємодії компонентів програмно- алгоритмічного конвеєра	32
---	----

2.2 Математична модель геометричного зворотного проєктування та вирівнювання базису осей	33
---	----

2.3 Анізотропна стохастична модель зворотного проєктування . .	36
--	----

2.4 Математичний апарат UKF та нелінійної аеродинаміки	38
--	----

					ІАЛЦ.467200.003 ПЗ			
Зм.	Лист	№ докум.	Підп.	Дата	Система просторового аналізу гри в волейбол з використанням машинного навчання	Лит.	Аркуш	Аркушів
Розробив	Безщасний Р.Р.						1	94
Перевірив	Морозов-Леонів							
Н. контр.	Мищенко Л. Д.					КПІ ім. Ігоря Сікорського, ФІОТ, ІО-21		
Затвердив	Волокита А. М.							

2.5	Налаштування матриць коваріації процесу та вимірювань . . .	42
2.6	Адаптація алгоритму ByteTrack для тривимірного простору . .	46
2.7	Захист оцінки глибини від монокулярного виродження	49
2.8	Фізична модель відскоку від поверхні підлоги	50
2.9	Методологія валідації якості трекінгу	52

ВИСНОВКИ ДО РОЗДІЛУ 2 **54**

3 Програмна реалізація аналітичного комплексу **57**

3.1	Загальна структура програмного комплексу	57
3.2	Модуль геометричного зворотного проектування	59
3.3	Модуль ядра фільтрації IMM_UKF	60
3.4	Модуль трекінгу IMMTracker	63
3.5	Модуль навчання моделі YOLO26	66
3.6	Конфігураційні параметри та залежності	68
3.7	Інструкція з користування програмним комплексом	70

ВИСНОВКИ ДО РОЗДІЛУ 3 **72**

4 Експериментальне дослідження та аналіз результатів **74**

4.1	Методика експерименту та обмеження вхідних даних	74
4.2	Якість детекції YOLO26	75
4.3	Кількісні результати тривимірного трекінгу	77
4.4	Абляційний аналіз механізмів захисту глибини	78
4.5	Непряма фізична валідація	81
4.6	Аналіз режимів взаємодіючих множинних моделей	82
4.7	Узагальнення досягнутих результатів	83
4.8	Аналіз обмежень та причин неповного успіху	84
4.9	Напрями подальшого вдосконалення	86

ВИСНОВКИ ДО РОЗДІЛУ 4	87
ВИСНОВОК	88
СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ	91

					ІАЛЦ.467200.003 ПЗ	Аркуш
						3
Зм.	Лист	№ докум.	Підп.	Дата		

ПЕРЕЛІК УМОВНИХ СКОРОЧЕНЬ

API	Application Programming Interface (програмний інтерфейс застосунку)
CNN	Convolutional Neural Network (згорткова нейронна мережа)
EKF	Extended Kalman Filter (розширений фільтр Калмана)
FPS	Frames Per Second (кадрів за секунду)
GPU	Graphics Processing Unit (графічний процесор)
HUD	Heads-Up Display (накладена інформаційна панель)
IMM	Interacting Multiple Model (взаємодіючі множинні моделі)
IoU	Intersection over Union (відношення перетину до об'єднання рамок)
mAP	mean Average Precision (усереднена середня точність детекції)
MoG2	Mixture of Gaussians 2 (суміш гауссових розподілів)
MOTA	Multi-Object Tracking Accuracy (сумарна точність багатооб'єктного відстеження)
MOTP	Multi-Object Tracking Precision (точність локалізації багатооб'єктного відстеження)
PnP	Perspective-n-Point (задача визначення пози камери за n точками)
R-CNN	Region-based Convolutional Neural Network (згорткова нейронна мережа на основі регіонів)
RMSE	Root Mean Square Error (середньоквадратична похибка)
RPN	Region Proposal Network (мережа пропозиції регіонів)
SORT	Simple Online and Realtime Tracking (просте онлайн-відстеження в режимі реального часу)
UKF	Unscented Kalman Filter (сигма-точковий фільтр Калмана)
YOLO	You Only Look Once (сімейство одноетапних моделей детекції об'єктів)

ВСТУП

Актуальність теми. Сучасний етап розвитку професійного спорту характеризується тотальною цифровізацією та інтеграцією інтелектуальних технологій у тренувальний та змагальний процеси. У волейболі, який є однією з найдинамічніших командних ігор у світі, швидкість польоту м'яча при атакуючих ударах може перевищувати 130 км/год, а тривалість окремих ігрових фаз вимірюється частками секунди. За таких умов традиційний візуальний аналіз з боку тренерського штабу або використання простих систем двовимірної відеофіксації вже не задовольняють вимогам високоточного розбору тактико-технічних дій.

Автоматизований збір кінематичних метрик, таких як тривимірна траєкторія польоту м'яча, координати точок відскоку, кути атаки та пікова швидкість польоту м'яча, дозволяє об'єктивно оцінювати ефективність дій гравців, моделювати поведінку суперника та оптимізувати біомеханіку рухів під час тренувань. Проте чинні комерційні рішення (наприклад, Hawk-Eye або Foxtenn) є монопольними системами з високою вартістю впровадження, що вимагають встановлення від 10 до 20 синхронізованих високошвидкісних індустріальних камер по всьому периметру залу. Це робить їх абсолютно недоступними для аматорських ліг, дитячо-юнацьких спортивних шкіл та клубів середнього рівня.

У зв'язку з цим виникає гостра потреба в розробці доступної, локальної та архітектурно гнучкої системи комп'ютерного зору, здатної здійснювати тривимірний трекінг та аналітику ігрових параметрів волейболу на основі обмеженої кількості джерел відеоданих (аж до однієї камери споживацького класу, наприклад, сучасного смартфона). Дана задача лежить на перетині глибокого навчання (для детекції дрібних швидкісних об'єктів), проективної геометрії (для калібрування простору) та теорії оптимальної фільтрації й оці-

нювання станів(для компенсації шумів вимірювання). Реалізація такої системи дозволить створити оптимізований програмно-алгоритмічний пайплайн, що ефективно поєднує низькорівневу обробку відеопотоків із сучасними методами Data Science.

Мета дослідження. Метою бакалаврської роботи є розробка та дослідження системи комп'ютерного зору для автоматизованого аналізу волейбольних епізодів у тривимірному просторі за допомогою однієї статичної камери на основі моделей глибокого навчання та нелінійного адаптивного фільтрування із компенсацією фундаментальних обмежень монокулярної геометрії.

Для досягнення поставленої мети необхідно вирішити такі завдання:

- проаналізувати існуючі методи детекції, трекінгу та відновлення 3D-координат швидкісних об'єктів за монокулярними відеопотоками;
- обґрунтувати вибір та провести донавчання згорткової нейронної мережі сімейства YOLO для стабільної детекції волейбольного м'яча в умовах сильного розмиття рухом;
- розробити програмний модуль геометричного калібрування тривимірного простору майданчика на основі математичного апарату перспективного проектування камери(PnP);
- створити алгоритм зворотного проектування для побудови 3D-променів від оптичного центру лінзи через піксельні координати детекції та оцінки глибини польоту м'яча;
- дослідити проблему високочастотного тремтіння меж рамки детекції та спроектувати математичний апарат фільтрації траєкторії на основі нелінійного фільтра Калмана(UKF);
- імплементувати багатомодельну адаптивну архітектуру(IMM) для

відстеження м'яча під час зміни фаз польоту, включаючи вільну балістичну параболу, момент ударного імпульсу та відскок від верхонь;

- експериментально дослідити коректність роботи системи методами непрямої валідації (відповідність законам вільного падіння, контрольований абляційний аналіз захисних механізмів асоціації та оцінювання) та провести статистичний аналіз тестових ігрових епізодів.

Об'єкт дослідження — процес відстеження просторового переміщення волейбольного м'яча та визначення його кінематичних параметрів за допомогою відеоданих у тривимірному просторі.

Предмет дослідження — методи комп'ютерного зору, моделі глибокого навчання для об'єктної детекції, алгоритми проективної геометрії, математичні методи нелінійної фільтрації станів динамічних систем та адаптивного оцінювання.

Методи дослідження. Для вирішення поставлених завдань використано:

- методи глибокого навчання та штучних нейронних мереж (для виявлення м'яча на кадрах);
- апарат лінійної алгебри та проективної геометрії (для калібрування камери за реперними точками розмітки корту та реалізації Ray Casting);
- методи теорії ймовірностей та математичної статистики (для опису коваріаційних матриць шумів процесу та вимірювань);
- теорію оптимального керування та оцінювання динамічних станів, зокрема Unscented Kalman Filter (для згладжування траєкторій в

умовах нелінійної аеродинаміки) та Interacting Multiple Models(для перемикання режимів руху).

Наукова новизна. Наукова новизна отриманих результатів полягає у розробці інтегрованого підходу до монокулярного 3D-трекінгу надшвидкісних спортивних об'єктів, який, на відміну від існуючих методів оцінки глибини за геометричними розмірами рамок детекції (що схильні до критичного накопичення похибок), поєднує безпохідне нелінійне згладжування за допомогою алгоритму UKF із динамічним перемиканням моделей руху (IMM) та оригінальний комплекс механізмів захисту оцінки глибини від монокулярного виродження (анізотропна адаптивна модель шуму вимірювань, гейт асоціації з роздільною обробкою латеральної та глибинної інновацій, фізичні обмежувачі прогнозу). Це дозволяє стабілізувати обчислення кінематичних параметрів у тривимірному просторі без використання багатокамерних стереосистем у межах, об'єктивно обумовлених монокулярною геометрією.

Практичне значення. Розроблена система реалізована у вигляді оптимізованого програмного комплексу мовою Python із використанням сучасних індустріальних бібліотек (OpenCV, Ultralytics YOLO, FilterPy). Спроектований пайплайн дозволяє проводити глибокий аналіз тренувальних та змагальних відеозаписів, отриманих зі звичайних смартфонів, що забезпечує низький поріг входження для спортивних клубів та індивідуальних тренерів. Отримані аналітичні дані(профілі швидкостей і висот, перетини площини сітки, розподіл часу гри за сторонами майданчика, відновлені траєкторії) можуть бути використані для автоматизації збору статистики та подальшого тактико-технічного аналізу.

РОЗДІЛ 1

АНАЛІЗ ТЕОРІЇ ТА ОБҐРУНТУВАННЯ ТЕМИ

1.1 Еволюція методів виявлення рухомих об'єктів у відеопотоці

Задача автоматизованого аналізу спортивних подій високої динаміки, до яких належить волейбольний матч, висуває екстремальні вимоги до систем комп'ютерного зору. Траєкторія руху волейбольного м'яча характеризується високими швидкостями, різкими змінами напрямку під час ударних взаємодій та суттєвими оптичними спотвореннями.

Історично перші системи трекінгу базувалися на аналізі низькорівневих ознак зображення. Серед них ключове місце посідав метод Optical Flow. В основі диференціальних методів розрахунку оптичного потоку лежить фундаментальне рівняння сталості яскравості пікселя за умови його малого зміщення за час dt :

$$I_x u + I_y v + I_t = 0, \quad (1.1)$$

де I_x, I_y, I_t — часткові похідні інтенсивності зображення за просторовими координатами та часом, а $u = dx/dt, v = dy/dt$ — шукані компоненти вектора швидкості пікселя. Алгоритм Лукаса-Канаде [1, 2] розв'язує це рівняння методом найменших квадратів для локального вікна, припускаючи сталість оптичного потоку в межах околиці. Глобальні підходи, такі як метод Горна-Шунка [3], додають умову гладкості векторного поля, мінімізуючи функціонал енергії:

$$E = \iint ((I_x u + I_y v + I_t)^2 + \alpha^2 (\|\nabla u\|^2 + \|\nabla v\|^2)) dx dy. \quad (1.2)$$

Незважаючи на математичну елегантність, диференціальні методи

оптичного потоку виявляються недієздатними в умовах реального волейбольного залу з таких причин:

- **Порушення умови сталості яскравості:** Обертання м'яча, що має глянцеvu або текстуровану поверхню, у поєднанні зі складним штучним освітленням (відблиски, тіні від прожекторів) викликає інтенсивні локальні зміни інтенсивності, які алгоритм помилково інтерпретує як геометричний рух.
- **Динамічний текстурований фон:** Переміщення гравців, арбітрів, глядачів та коливання сітки генерують потужний фоновий оптичний потік. Локалізація м'яча, що має малу площу на кадрі, стає математично некоректною через високе відношення сигнал/ шум.
- **Проблема розмиття рухом:** Швидкість польоту м'яча при професійному ударі сягає понад 30 м/с. При стандартній експозиції камери об'єкт розмивається в просторі, утворюючи напівпрозорий еліптичний шлейф. Градієнти інтенсивності I_x , I_y стають розмитими, що призводить до падіння точності розрахунку похідних до нуля.

Альтернативні класичні підходи, такі як віднімання фону за допомогою суміші Гаусіан(MoG2) [4], та пошук контурів за допомогою перетворення Хафа, також мають обмеження. Перетворення Хафа шукає параметричні кола, розраховуючи акумуляторний простір для радіусів. Оскільки форма м'яча через розмиття рухом трансформується в еліпс зі змінним ексцентриситетом, акумуляторний простір не формує чіткого піка, викликаючи пропуски детекції. Це зумовило перехід до методів штучного інтелекту, які здатні вилучати високорівневі семантичні ознаки об'єкта незалежно від його миттєвого розмиття чи умов освітлення.

Промислові спортивно-аналітичні системи минулого десятиліття (Hawk-Eye в тенісі та крикеті, FoxTrax у хокейних трансляціях) історично

					ІАЛЦ.467200.003 ПЗ	Аркуш
						10
Зм.	Лист	№ докум.	Підп.	Дата		

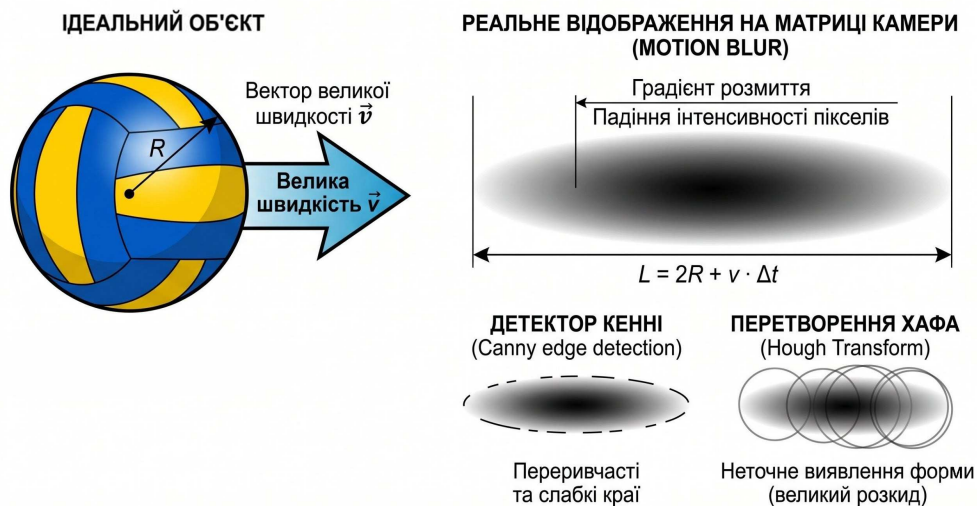


Рисунок 1.1 — Схема оптичного спотворення та похибки паралакса при гомографії

будувалися саме на гібридному поєднанні кольорового сегментування, мультикамерного стерео-зіставлення та оптичного потоку. Такий підхід вимагав щонайменше 6–10 синхронізованих каліброваних камер з частотою кадрів 200–340 FPS для подолання проблеми розмиття та надійного відновлення глибини за тріангуляцією. Економічна неефективність розгортання подібної інфраструктури для масового аналізу аматорських та юнацьких волейбольних матчів суттєво обмежила застосовність промислових рішень. Альтернативою стало одно-камерне рішення на базі сучасних згорткових нейромереж, що дозволяє переносити обчислювальне навантаження з оптичної підсистеми на алгоритмічну, зменшуючи капітальні витрати на порядок за рахунок використання типового спортивного відеобладнання.

Експериментальні дослідження роботи методу Лукаса-Канаде у відеопотоках професійних волейбольних матчів демонструють, що частка успішно оброблених кадрів з тонким м'ячем падає нижче 35% при швидкості польоту понад 18 м/с та експозиції 1/250 с. Для атак професійних гравців, де швидкість сягає 30–40 м/с, ефективність диференціальних методів наближається до нуля — алгоритм перестає розрізняти рух м'яча та фоновий рух

гравців. Це остаточно підтверджує необхідність переходу від низькорівневого опрацювання градієнтів зображення до високорівневого семантичного розпізнавання, інваріантного до миттєвої форми та текстури об'єкта.

1.2 Огляд та аналіз еволюції архітектур сімейства YOLO

Впровадження CNN здійснило революцію в об'єктній детекції. Перші двоетапні архітектури, такі як R-CNN, забезпечували високу точність, але були непридатні для реального часу через обчислювальну складність етапу генерації регіонів(RPN). Для спортивної аналітики критичним є інференс із частотою, не меншою за частоту кадрів відеопотоку (50+ FPS). Рішенням стало сімейство моделей YOLO [5], що реалізує одноетапну детекцію. YOLO формулює задачу виявлення об'єктів як єдину задачу регресії, де координати Bounding Box(рамок прив'язки властивостей) та ймовірності класів обчислюються за один прохід через мережу. Розглянемо еволюцію архітектурних рішень усередині сімейства YOLO, що призвела до появи сучасних рішень типу YOLO26:

- 1) YOLOv3–YOLOv4: Впровадження концепції апіорних рамок та архітектури DarkNet53. Використання Feature Pyramid Network для детекції на трьох різних масштабах. Недоліком була жорстка прив'язка до геометричних пропорцій якорів, що погано працювало для об'єктів, які швидко змінюють форму(як еліпс розмиття м'яча).
- 2) YOLOv5–YOLOv8: Перехід до концепції без'якірної детекції. Модель прогнозує зміщення відносно центру об'єкта напрямку. Впровадження розділеної головки детекції, яка розділяє гілки класифікації та регресії координат. Це суттєво підвищило точність локалізації малих об'єктів.

3) Сучасні архітектури (на прикладі YOLO26): Інтеграція блоків великої та динамічної уваги(трансформерів) безпосередньо у структуру основи мережі. Використання вдосконаленого блоку ший мережі Path Aggregation Network з глибокими залишковими зв'язками. Як функцію втрат локалізації застосовано Complete IoU Loss [6] та Distribution Focal Loss [7], що враховують не лише перекриття рамок, а й співвідношення сторін та центри мас:

$$\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v, \quad (1.3)$$

де $\rho^2(b, b^{gt})$ — евклідова відстань між центрами передбаченої та істинної рамок, c — діагональ мінімального прямокутника, що описує обидві рамки, αv — параметр, що контролює баланс співвідношення сторін. Для задачі виявлення волейбольного м'яча, що займає екстремально малу площу (від 10x10 до 25x25 пікселів на FullHD кадрі), застосування сучасних якірних моделей дозволяє утримувати високе значення метрики mAP50-95. Шляхом збільшення вхідного тензора на етапі інференсу до $imgsz=1920$ (за навчання на тензорі $imgsz=640$) досягається збереження просторової роздільної здатності дрібних текстур м'яча на дальніх планах майданчика.

Окремо варто розкрити структуру функції втрат локалізації Distribution Focal Loss (DFL), яка є фундаментальним нововведенням сучасних безякірних детекторів. Замість прямої регресії скалярних координат рамки модель прогнозує дискретне ймовірнісне розподілення $P(y_i)$ зміщень у фіксованому діапазоні $[y_0, y_n]$, дискретизованому на n корзин. Очікувана позиція кожної сторони рамки відновлюється як $\hat{y} = \sum_{i=0}^n y_i P(y_i)$, а функція втрат набуває вигляду:

$$\mathcal{L}_{DFL} = -((y_{i+1} - y) \log P(y_i) + (y - y_i) \log P(y_{i+1})), \quad (1.4)$$

де $y_i \leq y \leq y_{i+1}$ — істинна позиція межі рамки між двома сусідніми корзинами. Така параметризація дозволяє кодувати невизначеність моделі прямо у функції втрат, що надзвичайно корисно для дрібних динамічних об'єктів, де жорстка скалярна регресія часто призводить до коливань на 1–2 пікселі від кадру до кадру, безпосередньо переходячи у похибку розрахунку глибини на рівні десятків сантиметрів.

Для порівняння точності та обчислювальної ефективності різних поколінь сімейства YOLO на задачі виявлення малих об'єктів у динаміці у роботі сформовано таблицю 1.1, побудовану на основі агрегованих бенчмаркових результатів для архітектур-аналогів розмірності small/medium із вхідним тензором 640×640 пікселів та інференсом на одній відеокарті NVIDIA RTX 4070.

Таблиця 1.1 — Зведена характеристика поколінь моделей сімейства YOLO для задачі виявлення дрібних об'єктів

Покоління	Парадигма детекції	Функція втрат локалізації	mAP50 на малих об'єктах, %	Інференс, FPS
YOLOv3–v4	Anchor-based, FPN	MSE + IoU	28–35	60–75
YOLOv5–v7	Anchor-based + auto-anchor	GIoU / CIoU	36–44	80–110
YOLOv8–v10	Anchor-free + decoupled head	CIoU + DFL	45–52	95–130
YOLO11 / YOLO12	Anchor-free + dynamic attention	CIoU + DFL + Varifocal	50–57	110–145
YOLO26	Anchor-free + transformer neck	CIoU + DFL + Wise-IoU	54–61	120–160

Наведені у таблиці 1.1 цифри підтверджують, що тільки моделі покоління YOLO26 спроможні поєднати потрібний рівень метрики mAP50

($\geq 54\%$) на класі дрібних об’єктів із обчислювальним бюджетом, достатнім для роботи у реальному часі на типовому користувацькому обладнанні без використання серверних прискорювачів. Це дозволяє обрати YOLO26 як основну архітектуру детекції для розроблюваної системи [8].

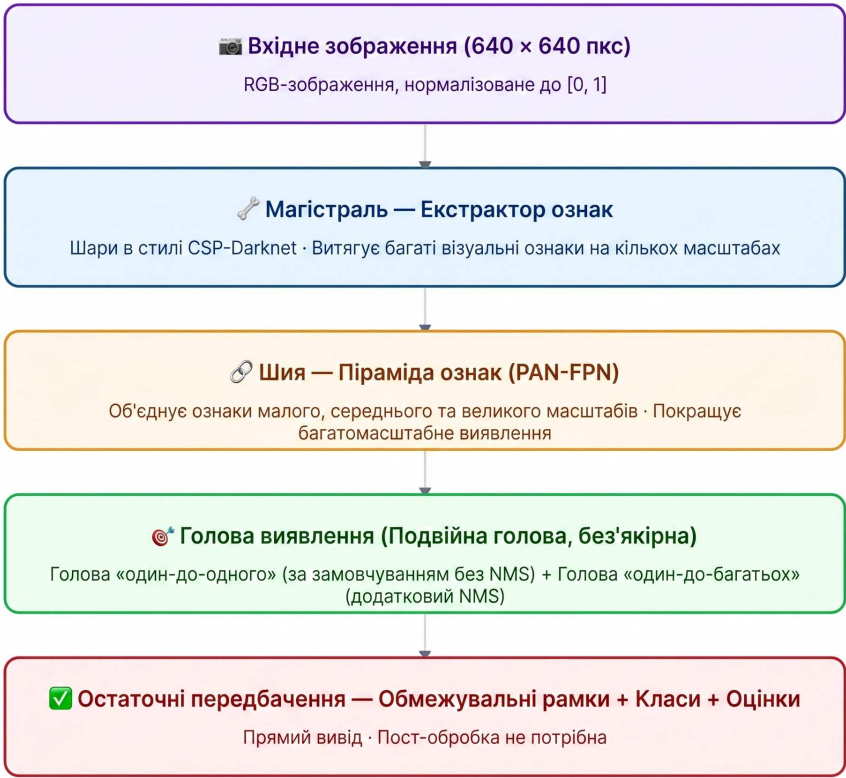


Рисунок 1.2 — Загальний потік архітектури YOLO26

1.3 Обмеження проєкцій двовимірного простору та математичне обґрунтування переходу до 3D

Екстракція 2D-координат центру м’яча $x_{pixel} = [u, v]^T$ у піксельній сітці кадру не дозволяє отримати реальні кінематичні характеристики (швидкість у км/год, прискорення), оскільки піксельний зсув залежить від відстані об’єкта до об’єктива. Найбільш поширеним спрощеним підходом у спортивній аналітиці є застосування плоскої двовимірної гомографії. Матриця гомографії $H \in \mathbb{R}^{3 \times 3}$ встановлює лінійне проєктивне співвідношення між однорі-

дними координатами пікселів та точками на реальній площині підлоги майданчика:

$$\begin{bmatrix} X_{world} & Y_{world} & 1 \end{bmatrix} \sim H \begin{bmatrix} u & v & 1 \end{bmatrix}. \quad (1.5)$$

Матриця H має 8 ступенів свободи і розраховується за 4 реперними точками розмітки(кутами волейбольного поля) шляхом розв'язання перевищеної системи лінійних рівнянь методами сингулярного розкладу(SVD). Проте гомографія базується на жорсткому геодезичному припущенні, що висота об'єкта завжди дорівнює нулю ($Z_{world} = 0$). Волейбольний матч повністю проходить у тривимірному просторі: висота траєкторії пасу може сягати 6-8 метрів, а атакуючий удар наноситься на висоті понад 3 метри.

Коли м'яч піднімається над підлогою, виникає критична похибка паралакса. Оптичний промінь камери, проходячи через піднятий м'яч, перетинає площину підлоги значно далі його реальної горизонтальної проекції. Математично, величина геометричного зсуву(помилки) Δx на площині підлоги пропорційна висоті м'яча h та тангенсу кута відхилення оптичного променя від нормалі камери θ :

$$\|\Delta x\| = h \cdot \tan \theta. \quad (1.6)$$

Для кутів огляду сучасних камер $\theta \approx 45^\circ - 60^\circ$ при висоті м'яча $h = 3$ м похибка гомографії становить від 3 до 5.2 метрів. За таких умов м'яч на графіках гомографії "телепортується" за межі залу, а розрахована швидкість набуває вигляд розірваних сплесків, що повністю виключає можливість побудови аналітики. Це зумовлює необхідність переходу до повноцінного 3D-моделювання на основі перспективної геометрії камери - камери-обскури(Pinhole Camera Model) та алгоритмів зворотного проектування(Ray Casting) [9].

Для наочної демонстрації абсурдності припущення про нульову висоту у табл. 1.2 наведено результати чисельного моделювання геометричної похибки гомографії $\|\Delta x\|$ для типового діапазону висот польоту волейболь-

ного м'яча та практично значущих кутів відхилення оптичного променя від нормалі камери. Кути θ охоплюють як зйомку з трибуни на середній лінії ($\theta \approx 30^\circ$), так і зйомку широкоформатною камерою з кутового штатива в зоні задньої лінії ($\theta \approx 70^\circ$), що є типовою конфігурацією для аматорських волейбольних трансляцій.

Таблиця 1.2 — Чисельний приклад геометричної похибки гомографії

$$\|\Delta \mathbf{x}\| = h \cdot \tan \theta, \text{ м}$$

$h, \text{ м} \setminus \theta$	30°	40°	50°	60°	70°
1.0	0.58	0.84	1.19	1.73	2.75
2.0	1.15	1.68	2.38	3.46	5.49
3.0	1.73	2.52	3.58	5.20	8.24
4.5	2.60	3.78	5.36	7.79	12.36
6.0	3.46	5.03	7.15	10.39	16.48
8.0	4.62	6.71	9.53	13.86	21.97

Порівняння табличних значень з фактичним розміром волейбольного майданчика (9×18 м) демонструє, що для типових для змагань значень висоти $h \in [4.5; 8]$ м похибка плоскої гомографії перевищує половину ширини майданчика навіть при помірних кутах огляду $\theta = 50^\circ$. Це означає, що візуально оцінений по екрану майданчиковий розрахунок швидкості польоту м'яча, отриманий за гомографією, не лише систематично заниженою, а і взагалі непридатний для побудови траєкторії атаки. Виявлений кількісний результат остаточно унеможливорює застосування двовимірної проективної геометрії і доводить необхідність переходу до повноцінного просторового відновлення координат, інваріантного до зміни висоти м'яча над підлогою.

1.4 Проблема багатокритерійного трекінгу у 3D просторі

Процес відстеження об'єкта складається не лише з його ізольованої детекції, а й з пов'язування виявлених об'єктів у часі для побудови безперервних траєкторій(треків). У реальних умовах волейбольного залу виникають суттєві завади: наявність інших сферичних чи висококонтрастних об'єктів(кросівки, елементи форми гравців, м'ячі на сусідніх кортах), які неймережа може помилково класифікувати як цільовий клас(False Positives).

Класичні 2D-алгоритми багатооб'єктного трекінгу, такі як SORT [10], використовують лінійне призначення за допомогою Угорського алгоритму на основі матриці вартості, побудованої за метрикою IoU між обмежуючими рамками. Модифікації на кшталт ByteTrack [11] удосконалюють цей процес, розділяючи детекції на два пороги впевненості(висока та низька впевненість), що дозволяє утримувати трек навіть при сильному розмитті, коли оцінка впевненості YOLO падає.

Проте пряме перенесення 2D Іоу-трекерів у 3D-простір є неефективним. Перспективна проєкція тривимірного простору на двовимірну матрицю згортає глибину сценарію. Як наслідок, два об'єкти, які знаходяться на абсолютно різних відстанях від камери (наприклад, реальний м'яч над сіткою та кросівки гравця на дальній лінії), на екрані можуть мати високе перекриття рамок(велике 2D IoU). Це викликає критичну помилку перемикання треків(ID Switch), коли трекер перемикається з м'яча на сторонній шум. Таким чином, індустріальний підхід вимагає відмови від піксельного IoU на користь розрахунку просторових метрик безпосередньо всередині відновленого тривимірного метричного простору залу.

У контексті специфічно волейбольної задачі типовими сценаріями виникнення помилки перемикання треку є:

– **Подача (сервіс).** М'яч на висоті 1.5 – 2 м перетинає у проєкції на

кадр траєкторію футболки судді на дальньому плані. Перекриття 2D-рамок м'яча та одягу досягає 70–90%, при тому що реальна евклідова відстань між об'єктами становить 12–15 м.

- **Атакуюча дія (нападний удар).** У момент контакту руки з м'ячем рамка нейромережі тимчасово охоплює і кисть гравця, і м'яч одночасно. Якщо після удару м'яч переходить у фазу високошвидкісного польоту, оптичне розмиття знижує впевненість мережі, і трекер втрачає трек, перепризначаючи його на статичний об'єкт фону (вертикальну стійку сітки чи елемент банера на задній стіні).
- **Прийом м'яча у нижньому положенні.** Гравець у нижньому прийомі (стоячи на колінах) має приблизно ту саму висоту в кадрі, що і м'яч у точці контакту. 2D IoU становить близько 30–40% на 2–3 послідовні кадри, після чого трек ідентифікатора м'яча некоректно мігрує на елементи форми гравця.
- **Сусідній корт.** У великому ігровому залі з декількома майданчиками детектор регулярно виявляє додатковий м'яч на сусідньому корті. Двовимірне IoU не дозволяє розрізнити два валідні об'єкти, і ByteTrack-подібний підхід може помилково передати ідентифікатор цільового м'яча сусідньому, спричиняючи розрив траєкторії.

Кожний з наведених сценаріїв є практично неминучим у реальних умовах волейбольної гри та значною мірою компенсується тривимірним аналізом у латеральній складовій метрики залу, навіть якщо глибинна складова потребує окремих механізмів захисту (див. підрозділ 1.7): евклідова чи Махаланобісова відстань у просторі залу легко розпізнає, що м'яч над сіткою на висоті 2.4 м не може бути тим самим об'єктом, що і кросівок гравця на висоті 0 м. Це обґрунтовує вибір тривимірних метрик як основи модуля асоціації даних у проєктованій системі.

1.5 Порівняння методів фільтрації нелінійних динамічних систем

Будь-яка система відновлення 3D-координат за монокулярним відеопотоком має високий рівень дискретного апаратного шуму (Bounding Box Jitter). Оскільки глибина розраховується через розмір рамки в пікселях, коливання рамки на 1 піксель через шум нейромережі на відстані 15 метрів трансформується у стрибок координати глибини на 0.4–0.7 метра. Пряме диференціювання таких даних для пошуку швидкості ($v = dx/dt$) призводить до генерації білого шуму величезної амплітуди. Для стабілізації траєкторії та оцінки реальної швидкості розглядалися такі методи фільтрації:

- Фільтр Савіцького-Голея. Метод здійснює локальне згладжування даних шляхом підгонки полінома низького степеня $p(t) = \sum_{i=0}^m c_i t^i$ у ковзному часовому вікні розміром $2M + 1$ за методом найменших квадратів. Перевагою є простота обчислень та збереження екстремумів. Головний недолік — повна відсутність фізичного базису. Фільтр нічого не знає про закони гравітації чи аеродинаміки. При виникненні тривалих пропусків детекцій (оклюзія м'яча тілом гравця на 10 кадрів) фільтр Савіцького-Голея здійснює чисто математичну поліноміальну інтерполяцію, яка зазвичай виходить за межі фізично можливої параболи польоту.
- Лінійний Фільтр Калмана [12, 13]. Оптимальний послідовний ймовірнісний оцінювач стану динамічної системи. Він використовує кінематичну модель процесу (рівняння Ньютона) для прогнозування та коригує його на основі вимірювань. Стан описується гаусовим розподілом. Проте лінійний фільтр вимагає строго лінійної матриці переходу стану F . Реальний рух волейбольного м'яча на високих швидкостях (більше 20 м/с) зазнає потужного впливу

- сили лобового опору повітря, яка залежить від квадрата швидкості ($F_d \propto v^2$). Це робить диференціальні рівняння руху суворо нелінійними, і лінійний Калман втрачає стабільність, накопичуючи динамічну похибку(відставання оцінки).
- Розширений Фільтр Калмана(EKF). Класичний підхід до лінеаризації нелінійних систем. EKF апроксимує нелінійні функції переходу $f(x)$ та вимірювання $h(x)$ за допомогою розкладу в ряд Тейлора першого порядку, що вимагає обчислення матриць Якобі на кожній ітерації:

$$F_k = \left. \frac{\partial f}{\partial x} \right|_{\hat{x}_{k-1|k-1}}, \quad H_k = \left. \frac{\partial h}{\partial x} \right|_{\hat{x}_{k|k-1}}. \quad (1.7)$$

Для балістичної моделі з квадратичним тертям та ефектом Магнуса аналітичне виведення приватних похідних Якобіана є надзвичайно громіздким. Більш того, якщо початковий гаусів розподіл пропускається через сильно нелінійну функцію, він перестає бути гаусовим. EKF не враховує вищі члени ряду Тейлора, що призводить до систематичного заниження коваріації помилок та викликає розходження фільтра на крутих віражах траєкторії(в момент атакуючої дії/удару).

- Сигма-точковий фільтр Калмана(UKF) [14—16]. Замість лінеаризації складних математичних функцій, UKF використовує концепцію Unscented Transform(сигма-точкове перетворення). Фільтр вибирає мінімальний фіксований набір детермінованих точок(сигма-точок), які точно відображають середнє значення та коваріацію вихідного гаусового розподілу. Ці точки пропускаються безпосередньо через реальні нелінійні рівняння аеродинаміки м'яча. На виході середнє значення та коваріація трансформованого розподілу відновлюються за допомогою зваженого підсумовування. UKF забезпечує

точність, еквівалентну ряду Тейлора третього порядку для симетричних гаусівських розподілів, повністю усуваючи необхідність аналітичного диференціювання.

- Часточковий фільтр (Particle Filter, PF). Послідовний метод Монте-Карло, що замінює гаусове припущення емпіричним розподілом з N часточок $\{x_k^{(i)}, w_k^{(i)}\}_{i=1}^N$ із вагами, які перераховуються за принципом важливісного вибору. PF здатний моделювати негаусові, мультимодальні розподіли, що теоретично робить його стійким до раптових режимних змін руху (удар-відскок). Проте обчислювальна складність зростає лінійно з N : для прийнятної точності позиціонування волейбольного м'яча у тривимірному просторі потрібно щонайменше $N \approx 2000$ часточок, що з урахуванням повторної генерації сигма-сімплекса з 19 сигма-точок UKF (для дев'ятивимірного вектора стану) становить багаторазову перевитрату обчислювальних ресурсів. Окрім того, PF страждає від ефекту виродження часточок (Particle Degeneracy): після 30–50 ітерацій більшість ваг прямує до нуля, а ефективний розмір вибірки падає до 5–10 часточок, що потребує дороговартісної процедури повторного семплування (resampling) на кожній ітерації.

Порівняльний аналіз математичних методів фільтрації наведено в таблиці 1.3.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						22
Зм.	Лист	№ докум.	Підп.	Дата		

Таблиця 1.3 — Математико-алгоритмічне порівняння методів фільтрації траєкторій

Характеристика / метод	Савіцького-Голя	Лінійний Калман (KF)	Розширений Калман (EKF)	Сигма-точковий Калман (UKF)
Врахування фізики руху	Відсутнє (чиста апроксимація)	Повне (тільки лінійні системи)	Повне (через лінеаризацію)	Екстремальні нелінійні сигмоїди
Математична точність	Низька (локальна)	1-й порядок (для лінійних)	1-й порядок ряду Тейлора	3-й порядок Тейлора
Необхідність обчислення похідних	Ні	Ні (лінійні матриці)	Так (аналітичні Якобіани)	Ні (Безпохідний)
Стійкість до тривалих пропусків	Критичний розрив / осциляції	Експоненціальне зростання похибки	Схильність до розходження (Divergence)	Висока стабільність параболічного руху
Обчислювальна складність	Середня	Низька $\mathcal{O}(n^3)$	Середня (розрахунок Якобіанів)	Збалансована $\mathcal{O}(L \cdot n^3)$

1.6 Особливості предметної області волейболу як цілі тренінгу

Незважаючи на загальну подібність задач трекінгу спортивних м'ячів до інших динамічних сценаріїв комп'ютерного зору, волейбол має ряд унікальних характеристик, що формують жорсткий набір вимог до програмно-алгоритмічного забезпечення системи аналізу. Без урахування цих характеристик навіть теоретично обґрунтований конвеєр може демонструвати неприйнятну якість при роботі з реальним матеріалом.

Швидкісні режими руху об'єкта. Згідно з регламентом міжнародної

федерації FIVB [17], дозволена маса волейбольного м'яча становить 260—280 г, а тиск нагнітання — $0.3 - 0.325 \text{ кг}_f/\text{см}^2$. Така характеристика забезпечує наступний діапазон лінійних швидкостей у грі:

- Прийом м'яча знизу: $0.5 - 3 \text{ м/с}$;
- Друга передача (пас зв'язуючого): $4 - 8 \text{ м/с}$;
- подача планерна: $13 - 17 \text{ м/с}$;
- подача силова в стрибку: $24 - 32 \text{ м/с}$;
- Атакуюча дія (нападний удар): $30 - 42 \text{ м/с}$ (згідно зі статистикою провідних змагань, рекордні значення сягають порядку $40 - 47 \text{ м/с}$). Для порівняння, рекордна швидкість силової подачі є помітно нижчою — близько 37 м/с ($\approx 134 \text{ км/год}$), оскільки подача виконується без додаткового розгону руки в стрибку над сіткою.

При типовій частоті зйомки $30 - 50 \text{ FPS}$ це означає, що між двома сусідніми кадрами м'яч може переміститися на $0.6 - 1.4 \text{ м}$. Алгоритми, що покладаються на просторову близькість наступної детекції до попередньої (Kalman gating з малим радіусом), у фазі атаки практично гарантовано втрачають трек. Це формує жорстку вимогу до радіуса стробування трекера: геометричний бар'єр Махаланобіса має бути динамічним і автоматично розширюватися в моменти, коли модель ідентифікує великий розкид швидкостей.

Множинні режими руху в межах одного епізоду. На відміну від, наприклад, теніса, де м'яч між двома ударами завжди рухається балістично, у волейболі типовий ігровий епізод складається з $5 - 9$ контактних взаємодій (подача, прийом, пас, атака, блок, дозбір), кожна з яких миттєво змінює і модуль, і напрямок вектора швидкості м'яча. Розривні прискорення в момент контакту неможливо описати єдиною моделлю руху — це безпосередньо мотивує перехід до архітектури взаємодіючих множинних моделей (IMM), яка

дозволяє паралельно підтримувати декілька гіпотез про поточний режим динаміки об'єкта.

Висотний діапазон та конфлікт з горизонтальною площиною. Корисний об'ємний робочий простір для аналізу польоту м'яча в офіційному матчі простирається до висоти 9 – 10 м, що перевищує середню висоту стелі залу всього на 1 – 2 м. Така вертикальна щільність простору у поєднанні з невеликою горизонтальною площею (9×18 м) робить волейбольну сцену ідеальним прикладом тривимірної задачі з малим співвідношенням сторін — ширина простору становить менше половини його висоти. Це остаточно виключає застосування плоских гомографічних підходів, оскільки навіть невелика помилка визначення висоти (± 0.5 м) може повністю змінити інтерпретацію тактичної ситуації (наприклад, відрізнити пас "над сіткою" від атаки "понад блоком").

Освітлення та оптичні артефакти спортивного залу. Освітлення професійних залів зазвичай реалізовано комбінацією верхнього прожекторного світла (LED 5000–6500 K) та бічного підсвічування трибун теплішим спектром (3500–4500 K). Це генерує два паразитні ефекти, критичні для нейромережевої детекції:

- Спекулярні відбитки на гляцевій поверхні мікрофібрової оболонки м'яча, що утворюють виражену білу пляму змінного розміру. При атакуючих ударах ця пляма у певні моменти може бути більшою за саму темну частину м'яча, повністю змінюючи видимий контур.
- Подвійні тіні від м'яча на підлозі (через два різні джерела освітлення), які при швидкому русі візуально нагадують додатковий м'яч, спричиняючи помітну частку false-positive спрацьовувань детектора, особливо за умов багатоджерельного освітлення.

Перелічені ефекти безпосередньо мотивують необхідність агресивної аугментації навчальної вибірки шляхом штучного додавання яскравісних спотворень (CLAHE, RandomBrightnessContrast), насичення (HueSaturation) та змодельованих рухових розмиттів (MotionBlur), що включено до конфігурації навчання моделі YOLO26 у розроблюваній системі.

Латентність відеотракту. Для побудови ігрової аналітики у наближеному до реального часу режимі загальна затримка обробки одного кадру (відеодекодер + детектор + ray casting + трекер + візуалізатор) не повинна перевищувати $1/\text{FPS}$ кадрів, тобто 20 мс при 50 FPS. З цього бюджету 8 – 12 мс припадає на нейромережевий інференс YOLO26, що залишає 8 – 12 мс на всі решта компонентів. Це формує жорстку вимогу до обчислювальної ефективності модуля UKF: усі матричні операції повинні виконуватися строго на дев'ятивимірному просторі стану без застосування важких алгоритмів матричної інверсії високих розмірностей, що підтверджує доцільність вибору саме сигма-точкового підходу замість, наприклад, часточкового фільтра з тисячами часточок.

1.7 Монокулярне виродження глибини як ключовий виклик системи

Обраний у підрозділі 1.3 шлях монокулярного тривимірного відновлення на основі зворотного проектування має фундаментальну ціну, яку необхідно усвідомити ще на етапі аналізу, оскільки саме вона визначає принципову межу досяжної точності всієї системи. На відміну від стереозйомки, де глибину дають триангуляцією двох променів, монокулярна конфігурація відновлює відстань до м'яча уздовж єдиного оптичного променя за непрямою ознакою — видимим піксельним розміром об'єкта. Глибина обчислюється зі

співвідношення подібних трикутників камери-обскури:

$$Z_c = \frac{f \cdot D}{w}, \quad (1.8)$$

де f — ефективна фокусна відстань у пікселях, D — фізичний діаметр м'яча, w — піксельна ширина обмежуючої рамки детекції. Ключова вада цього співвідношення розкривається під час диференціювання глибини за вимірюваним розміром рамки:

$$\frac{\partial Z_c}{\partial w} = -\frac{f \cdot D}{w^2}. \quad (1.9)$$

Чутливість глибини до похибки розміру рамки зростає обернено квадратично зі зменшенням w , тобто стрімко загострюється зі збільшенням відстані до камери. Якщо для м'яча в ближній зоні майданчика похибка в один піксель ширини рамки спричиняє зсув глибини порядку 0,14 м, то на дальній межі робочого простору (близько 60 м) той самий одно-піксельний шум детектора транслюється вже у похибку близько 2,3 м. Оскільки нейромережевий детектор об'єктивно дає тремтіння рамки на рівні 1–2 пікселів навіть у сприятливих умовах, а під час розмиття рухом — значно більше, відновлена глибина на дальньому плані стає принципово ненадійною величиною.

Це монокулярне виродження глибини є коренем усіх подальших інженерних труднощів: завищених піків миттєвої швидкості, нефізичних розривів траєкторії та систематичної недооцінки координати дальньої сторони майданчика. Усвідомлення цієї властивості ще на етапі теоретичного аналізу безпосередньо мотивує закладені в систему механізми захисту оцінки глибини — анізотропну модель вимірювального шуму, що знижує довіру до глибинної складової пропорційно її чутливості, та глибинно-стійкі гейти асоціації, математичне обґрунтування яких наведено у Розділі 2. Таким чином, монокулярна архітектура свідомо обирається як економічно доцільний компроміс, а боротьба з її глибинним виродженням стає центральною науково-технічною задачею роботи.

1.8 Постановка задачі та формування вимог до системи

На основі проведеного аналізу можна сформулювати формальну постановку задачі. **Метою** роботи є розроблення програмно-алгоритмічного комплексу автоматичного відновлення тривимірної траєкторії волейбольного м'яча та обчислення просторово-часової статистики гри за єдиним монокулярним відеопотоком без застосування спеціалізованого багатокамерного обладнання. **Об'єктом** дослідження є процес автоматизованого аналізу динаміки спортивних подій за відеоданими, а **предметом** — методи та засоби монокулярного тривимірного трекінгу швидкісного малорозмірного об'єкта в умовах шуму детекції та оптичних спотворень.

Вхідними даними системи є відеопотік трансляційної або аматорської якості з роздільною здатністю до 1920×1080 пікселів та частотою 30–50 кадрів за секунду, знятий однією нерухомою камерою з оглядом усього майданчика. **Вихідними даними** є відновлена тривимірна траєкторія м'яча у метричній системі координат залу та похідний від неї пакет ігрової статистики (висотний і швидкісний профілі, перетини площини сітки, розподіл часу за сторонами майданчика), а також відеооверлей із візуалізацією треку.

Виходячи з мети, до системи висуваються такі **функціональні вимоги**:

- надійна детекція малорозмірного м'яча на кадрі засобами без'якірної згорткової нейромережі сімейства YOLO, інваріантна до розмиття рухом та оптичних артефактів;
- геометричне калібрування камери за реперними точками розмітки майданчика (алгоритм PnP) та зворотне проектування детекції у тривимірний метричний простір;
- багаторежимна фільтрація траєкторії на основі IMM-ансамблю сигма-точкових фільтрів UKF з придушенням високочастотного

тремтіння координат;

- асоціація даних у тривимірній метриці залу зі стійкістю до переми-
кання ідентифікаторів треку на сторонні висококонтрастні завади;
- захист оцінки глибини від монокулярного виродження, описаного
в підрозділі 1.7;
- обчислення підсумкової ігрової статистики та формування візуаль-
ного оверлею.

Нефункційні вимоги визначаються сценарієм роботи в наближено-
му до реального часу режимі: сумарна латентність обробки одного кадру не
повинна перевищувати 1/FPS (близько 20 мс за 50 кадрів за секунду), а весь
конвеєр має виконуватися на типовому користувацькому обладнанні з одні-
єю споживчою відеокартою, без серверних прискорювачів. Система повинна
зберігати стабільність за тривалих пропусків детекції (оклюзія м'яча) та за
раптової появи об'єкта в кадрі.

Оскільки для трансляційного відео не існує синхронного тривимірної
го еталона положення м'яча, **критеріями якості** обрано: метрики точності
детекції (mAP50, mAP50-95, точність і повнота) як показник якості першо-
джерела даних; покриття траєкторії та кількість перемикань ідентифікато-
рів треку як показники стабільності супроводження; фізичну консистентність
відновленої вертикальної динаміки (відповідність прискорення вільного па-
діння еталонному значенню) як непрямий показник геометричної коректно-
сті. Обґрунтування та практична перевірка цих критеріїв виконуються у Роз-
ділах 2 та 4 відповідно.

ВИСНОВКИ ДО РОЗДІЛУ 1

У першому розділі було проведено комплексний аналіз сучасного стану автоматизації спортивної аналітики, який продемонстрував неспроможність класичних низькорівневих підходів комп'ютерного зору, таких як оптичний потік чи віднімання фону, забезпечити необхідну стійкість в умовах високої динаміки волейболу. Через виникнення критичних артефактів розмиття рухом та складне штучне освітлення залу єдиним ефективним рішенням для локалізації наддрібних і швидкісних об'єктів визначено використання сучасних без'якірних згорткових нейромереж сімейства YOLO. Збільшення геометричних розмірів вхідного тензора моделі дозволяє вилучати високорівневі семантичні ознаки м'яча на середніх і дальніх планах майданчика, у межах об'єктивних обмежень монокулярної геометрії, описаних у підрозділі 1.7, без втрати частоти кадрів, що задовольняє вимогам обробки відеопотоків у реальному часі.

Дослідження геометричних методів просторової прив'язки виявило концептуальну помилку застосування традиційної плоскої двовимірної гомографії для трекінгу об'єктів, що летять у повітрі. Через нехтування координатою висоти виникає потужна похибка паралакса, яка геометрично викривляє метричні дані та унеможливорює адекватний розрахунок швидкості. Для подолання цього обмеження обґрунтовано доцільність повноцінного переходу у тривимірний простір на основі перспективної моделі камери-обскури, оптимізаційного алгоритму PnP для знаходження матриць екстринсиків камери та методів зворотного проектування(Ray Casting), що дозволяє динамічно відновлювати глибину за поточними розмірами обмежуючої рамки детекції.

Аналіз систем багатокритерійного трекінгу показав, що поширені 2D-алгоритми асоціації даних, які базуються на метриці перетину рамок(IoU),

					ІАЛЦ.467200.003 ПЗ	Аркуш
						30
Зм.	Лист	№ докум.	Підп.	Дата		

втрачають ефективність при переході у 3D-простір через ефект перспективного згортання глибини сценарію. Це призводить до критичних помилок перемикання ідентифікаторів треку на сторонні висококонтрастні шуми, наприклад, кросівки гравців чи елементи форми. Для забезпечення безперервності відстеження цільового об'єкта визначено необхідність модифікації двостадійної логіки ByteTrack шляхом перенесення процесу геометричного призначення у тривимірну метрику залу із заміною піксельного IoU на статистичну відстань Махаланобіса, яка враховує поточний еліпсоїд невизначеності системи.

Критичний огляд математичних методів фільтрації нелінійних динамічних систем підтвердив, що політ волейбольного м'яча супроводжується сильним квадратичним аеродинамічним опором повітря, що виключає можливість використання класичного лінійного фільтра Калмана. Водночас розширений фільтр Калмана(EKF) є нестабільним через аналітичну складність та схильність до розходження при лінеаризації різких траєкторій. Найбільш точним та безпохідним апаратом для згладжування високочастотного тремтіння координат визначено сигма-точковий фільтр Калмана(UKF). Оскільки рух м'яча є багаторежимним гібридним процесом, обґрунтовано інтеграцію UKF у багатомодельну архітектуру взаємодіючих множинних моделей (IMM), що дозволяє суттєво знизити запізнювання сигналу та забезпечити архітектурну готовність до миттєвих ударних імпульсів атакуючої дії й відскоків.

Отримані теоретичні висновки та сформульована у підрозділі постановка завдання дозволяють чітко окреслити вимоги та структуру майбутньої системи, а також формують науково обґрунтовану математичну основу для безпосередньої розробки програмно-алгоритмічного конвеєра тривимірної аналітики у наступних розділах проєкту.

РОЗДІЛ 2

МАТЕМАТИЧНЕ ТА АЛГОРИТМІЧНЕ ЗАБЕЗПЕЧЕННЯ АНАЛІТИЧНОЇ СИСТЕМИ

2.1 Структура та граф взаємодії компонентів програмно-алгоритмічного конвеєра

Проектування високонавантажених систем комп'ютерного зору вимагає чіткої декомпозиції алгоритмічних кроків для виключення затримок передачі даних. Розроблена система побудована за модульним принципом, де кожен етап оперує строго типізованими математичними тензорами.

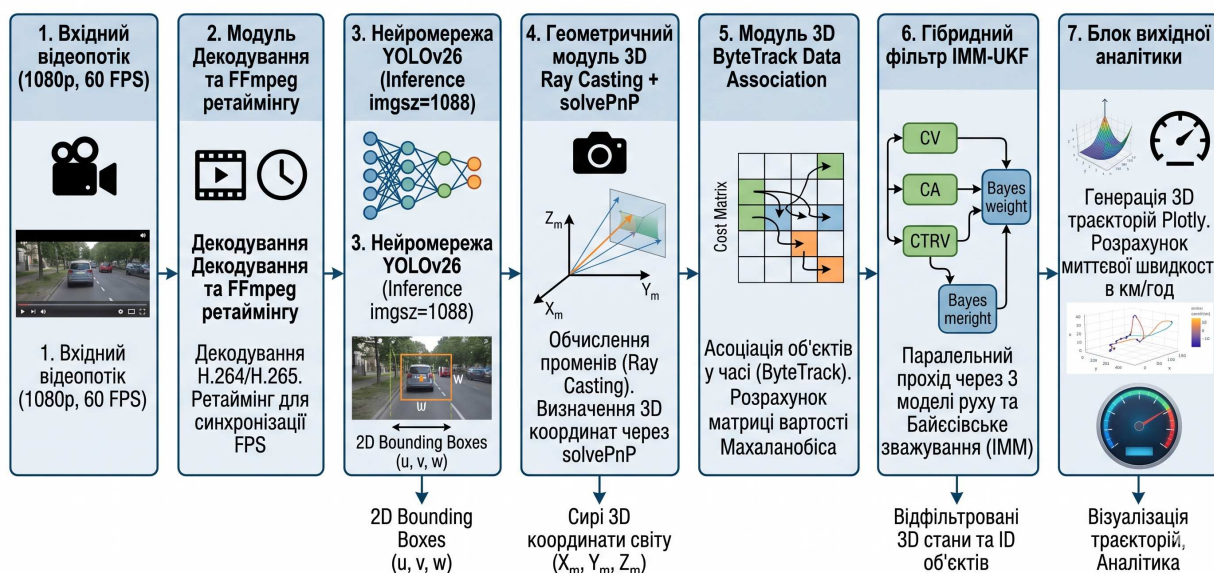


Рисунок 2.1 — Загальна архітектура конвеєра обробки відео для 3D-відстеження об'єктів

2.2 Математична модель геометричного зворотного проектування та вирівнювання базису осей

Для просторового калібрування камери використовується система рівнянь перспективної проєкції [18]. Матриця внутрішніх параметрів камери K задається як:

$$K = \begin{bmatrix} f_x & 0 & c_x & 0 & f_y & c_y & 0 & 0 & 1 \end{bmatrix}, \quad (2.1)$$

де f_x, f_y — фокальні відстані в пікселях, а c_x, c_y — координати головної точки (оптичного центру) на матриці. Алгоритм ‘cv2.solvePnP’ [19, 20] мінімізує похибку зворотного проектування (Reprojection Error) для множини реперних точок розмітки поля, повертаючи матрицю обертання $R \in SO(3)$ та вектор зсуву t . Для реалізації методу Ray Casting піксельні координати детекції центру м’яча $[u, v, 1]^T$ трансформуються в нормалізований вектор напрямку в системі координат камери: $d_{cam} = K^{-1}[u, v, 1]^T$. Переведення цього вектора у глобальну систему координат залу здійснюється шляхом множення на обернену (транспоновану) матрицю обертання R^T :

$$d_{world} = R^T \cdot d_{cam} = R^T \cdot \begin{bmatrix} \frac{u-c_x}{f_x} & \frac{v-c_y}{f_y} & 1 \end{bmatrix}. \quad (2.2)$$

Фізична позиція оптичного центру камери у просторі світу визначається як $C = -R^T \cdot t$. Тоді будь-яка точка на промені описується функцією глибини Z_c :

$$P(Z_c) = C + Z_c \cdot d_{world}. \quad (2.3)$$

Для однозначного визначення масштабу Z_c використовується відомий фізичний розмір об’єкта — діаметр волейбольного м’яча. Глибина обчислюється через поточну піксельну ширину рамки детекції w : $Z_c = (f_x \cdot D)/w$. Принципово, що фокальна відстань і діаметр входять у це співвідношення виключно добутком $f_x \cdot D$, тому масштаб глибини визначається саме цим добутком, а не кожним множником окремо. Номінальний діаметр м’яча становить 0.21 м,

проте у геометричному відновленні глибини використовується ефективно калібрувальне значення $D = 0.27$ м: воно поглинає систематичне заниження ширини рамки детектором (передбачена рамка YOLO систематично щільніша за реальний силует м'яча) і узгоджується спільним калібруванням з фокальною відстанню за гравітаційним критерієм (див. обґрунтування матриці K нижче). Рішення проблеми конфлікту осей: В оптичній системі координат OpenCV вісь Y_{cam} спрямована вниз від оптичного центру до підлоги, а вісь Z_{cam} — уперед вздовж оптичної осі. Натомість опорна базисна система волейбольного корту, заданого реперними точками, має наступну орієнтацію: вісь X — поперек майданчика (вздовж лицевої лінії), вісь Z — вздовж довгої сторони (вздовж бічної лінії), а вісь Y — суто вертикальна, спрямована вгору від поверхні підлоги проти вектора прискорення вільного падіння. Усі реперні точки розмітки лежать у площині $Y = 0$. Сирий вектор $P_{raw} = [X, Y, Z]^T$, отриманий зворотним проектуванням, має негативний знак координати Y для точок, розташованих над підлогою (через те, що камера розміщена вище та дивиться вниз). Для приведення базису до фізично коректного вигляду застосовується спрощений лінійний оператор дзеркальної інверсії осі висоти M_{align} :

$$P_{court} = M_{align} \cdot P_{raw} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} X \\ -Y \\ Z \end{bmatrix}. \quad (2.4)$$

Така трансформація дозволяє узгодити метричні дані з класичним вектором прискорення вільного падіння $\vec{g} = [0, -9.81, 0]^T$ і у подальшому використовувати вісь Y як єдиний носій інформації про висоту польоту м'яча, а пара (X, Z) описує його горизонтальну проекцію на підлогу майданчика.

Математична формалізація похибки зворотного проектування.

Алгоритм PnP мінімізує сукупну евклідову відстань між істинною піксельною координатою реперної точки (u_i, v_i) та її проекцією за поточними оцін-

ками матриці обертання R і вектора зсуву t :

$$\mathcal{L}_{reproj}(R, t) = \sum_{i=1}^N \left\| \begin{pmatrix} u_i \\ v_i \end{pmatrix} - \pi(K(RP_i + t)) \right\|^2, \quad (2.5)$$

де $P_i \in \mathbb{R}^3$ — відома просторова координата i -ї реперної точки розмітки поля, а оператор π переводить однорідну тривимірну координату на площину сенсора за правилом $\pi([x, y, z]^T) = [x/z, y/z]^T$. Внаслідок нелінійності функції π та матричної експоненти Родрігеса $R = \exp([\omega]_{\times})$ задача мінімізації $\mathcal{L}_{reproj}$ не має замкнутого аналітичного розв'язку. Метод Левенберга-Марквардта, закладений у реалізацію `cv2.solvePnP`, ітераційно лінеаризує цільову функцію навколо поточної оцінки параметрів та виконує крок градієнтного спуску у напрямку зменшення квадратичної похибки. Збіжність зазвичай досягається за 8–15 ітерацій з кінцевою середньоквадратичною похибкою репроекції $\sqrt{\mathcal{L}_{reproj}/N} < 1.5$ пікселя для типового набору з 4 реперних точок розмітки.

Обґрунтування параметрів матриці внутрішніх параметрів K .

Ігровий матеріал знято трансляційною камерою, паспортні оптичні характеристики якої (модель сенсора, фокусна відстань об'єктива, розмір пікселя) недоступні, тому фокальну відстань неможливо вивести з апаратних специфікацій. Натомість $f_x = f_y$ визначається калібруванням за фізичним критерієм: параметр підбирається так, щоб відновлені ділянки вільного польоту м'яча демонстрували вертикальне прискорення, близьке до прискорення вільного падіння $g \approx 9.81 \text{ м/с}^2$. Оскільки масштаб глибини залежить лише від добутку $f_x \cdot D$, фокальну відстань і ефективний діаметр калібрують спільно за цим єдиним гравітаційним критерієм. Знайдене значення $f_x = f_y = 5805$ пікселів дає відновлене вертикальне прискорення на рівні 9.0–9.2 м/с^2 на балістичних сегментах, що підтверджує адекватність геометрії. Початкове грубе наближення $f \approx 1300$ пікселів, узятє з типових аматорських сенсорів, давало завищену у ~ 4 рази глибину та фізично неможливе прискорення,

тому було відкинуте. Координати оптичного центру $c_x = W/2 = 960$ та $c_y = H/2 = 540$ пікселів (для кадру 1920×1080) обираються за припущенням, що оптична вісь проходить через геометричний центр сенсора; залишкова систематична похибка такого припущення поглинається згладжуючим ефектом сигма-точкового фільтра.

Стохастична природа оцінки глибини. Формула обчислення глибини $Z_c = (f_x D)/w$ містить у знаменнику піксельну ширину рамки w , яка сама по собі є нелінійною функцією від положення центру об'єкта та від поточної впевненості нейромережі. Шум δw у визначенні ширини рамки трансформується у шум координати глибини за законом $\delta Z_c \approx -(f_x D/w^2) \cdot \delta w$. Принципово важливим є квадрат ширини у знаменнику: коефіцієнт чутливості $f_x D/w^2$ зростає при зменшенні w , тобто похибка глибини катастрофічно збільшується для віддалених (дрібних на кадрі) м'ячів. Для робочого діапазону $Z_c \in [15; 60]$ м піксельна ширина рамки лежить у межах $w \in [26; 105]$ пікселів, а коефіцієнт чутливості зростає від ≈ 0.14 м/піксель на дистанції 15 м до ≈ 2.3 м/піксель на дистанції 60 м. Таким чином, 1 піксель шуму ширини рамки на ближній межі поля дає похибку глибини менше пів метра, тоді як на дальній половині майданчика та сама піксельна неточність розгойдує оцінку глибини на кілька метрів. Це фундаментальне виродження монокулярної глибини (а не апаратний шум) є першопричиною всіх подальших захисних механізмів фільтра й безпосередньо мотивує перехід від фіксованої дисперсії σ_z^2 до адаптивної анізотропної матриці R , описаної далі.

2.3 Анізотропна стохастична модель зворотного проектування

Розглянутий ефект виродження глибини унеможливорює застосування фіксованої діагональної матриці R : довіра до глибини мусить залежати від поточної геометрії спостереження (дистанції, ширини рамки) та від динамі-

ки м'яча. Тому матриця коваріації шуму вимірювання формується аналітично для кожної детекції з лінеаризації геометрії зворотного проектування (реалізовано у функції `get_3d_position_with_cov`).

Світова позиція м'яча є функцією трьох вимірюваних детектором величин — піксельної ширини рамки w та координат її центру (u, v) :

$$P_{world}(w, u, v) = C + Z_c(w) d_{world}(u, v), \quad Z_c = \frac{f_x D}{w}, \quad d_{world} = R^T \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix} \quad (2.6)$$

Лінеаризація P_{world} навколо поточного вимірювання дає матрицю Якобі $J = \left[\frac{\partial P}{\partial w}, \frac{\partial P}{\partial u}, \frac{\partial P}{\partial v} \right]$, стовпці якої обчислюються покомпонентно. Похідна за шириною рамки спрямована строго вздовж променя спостереження:

$$\frac{\partial P}{\partial w} = \frac{\partial Z_c}{\partial w} d_{world} = -\frac{f_x D}{w^2} d_{world}. \quad (2.7)$$

Похідні за пікселями центру рамки описують поперечний (латеральний) зсув точки і пропорційні дистанції Z_c :

$$\frac{\partial P}{\partial u} = Z_c R^T \begin{bmatrix} 1/f_x \\ 0 \\ 0 \end{bmatrix} = Z_c J_u, \quad \frac{\partial P}{\partial v} = Z_c R^T \begin{bmatrix} 0 \\ 1/f_y \\ 0 \end{bmatrix} = Z_c J_v, \quad (2.8)$$

де J_u, J_v — одиничні (на 1 піксель) латеральні зсуви світової точки. Вважаючи похибки вимірювання w, u, v некорельованими з дисперсіями σ_w^2 та σ_{uv}^2 , поширення невизначеності $R = J \text{diag}(\sigma_w^2, \sigma_{uv}^2, \sigma_{uv}^2) J^T$ дає замкнену форму матриці коваріації вимірювання:

$$R = \underbrace{\sigma_{Z_c}^2 d_{world} d_{world}^T}_{\text{глибина, вздовж променя}} + \underbrace{Z_c^2 \sigma_{uv}^2 (J_u J_u^T + J_v J_v^T)}_{\text{латеральна складова}} + r_{floor} I, \quad (2.9)$$

де $\sigma_{Z_c} = (f_x D / w^2) \sigma_w$ — те саме виродження глибини, що тепер увійшло безпосередньо у вимірювальну модель. Перший доданок — це витягнутий

уздовж променя еліпсоїд рангу 1 (домінантна невизначеність глибини), другий — мала поперечна невизначеність від шуму центрування рамки, а ізотропна підлога $r_{floor} = 0.02 \text{ м}^2$ гарантує додатну визначеність R і не дозволяє латеральній довірі впасти нижче емпірично підтвердженого рівня (монокулярна камера локалізує об’єкт у площині кадру дуже точно). На відміну від фіксованого σ_z^2 , така матриця автоматично знижує вагу глибинного вимірювання для дрібних (віддалених) рамок і підвищує її для близьких, повертаючи фільтру фізично чесну оцінку довіри.

Швидкісний гейт довіри до глибини. Окрім геометрії, надійність ширини рамки залежить від динаміки: під час швидкого польоту м’яча на кадрі змазується (motion blur), і ширина рамки коливається сильніше, ніж у статичці. Для врахування цього базова дисперсія ширини масштабується пропорційно до швидкості переміщення центру рамки в площині кадру:

$$\sigma_w^{\text{eff}} = \sigma_w (1 + g_v \cdot s_{img}), \quad s_{img} = \sqrt{(\Delta u)^2 + (\Delta v)^2}, \quad (2.10)$$

де s_{img} — модуль піксельної швидкості центру рамки (пікселів за кадр), а $g_v = 0.15$ — коефіцієнт підсилення; базове значення $\sigma_w = 1.5$ пікселя. У результаті на ділянці змазаного руху (атака, прийом подачі) фільтр автоматично перестає будувати хибну “глибинну швидкість”, тоді як у точці апексу траєкторії, де $s_{img} \rightarrow 0$, довіра до глибини відновлюється повністю.

2.4 Математичний апарат UKF та нелінійної аеродинаміки

Вектор стану динамічної системи м’яча визначається у 9-вимірному просторі станів, де кожна просторова вісь представлена трійкою «координата — швидкість — прискорення». Пряме включення прискорення у вектор стану є принциповим для багаторежимної архітектури: саме стрибок прискорення (а не швидкості) фізично відрізняє вільний політ від удару та відскоку, тож

моделі ІММ перемикаються за компонентами a_x, a_y, a_z :

$$x = [x, v_x, a_x, y, v_y, a_y, z, v_z, a_z]^T. \quad (2.11)$$

Рівняння переходу стану балістичної моделі задаються системою першого порядку нелінійних диференціальних рівнянь. Кінематичний каркас спільний для всіх трьох режимів (похідна координати — швидкість, похідна швидкості — прискорення), а специфіку балістики задає аеродинамічне прискорення, що враховує силу тяжіння (вздовж вертикальної осі Y) та нелінійний квадратичний опір повітря (Drag Force):

$$\frac{d}{dt} \begin{bmatrix} x \\ v_x \\ a_x \\ y \\ v_y \\ a_y \\ z \\ v_z \\ a_z \end{bmatrix} = \begin{bmatrix} v_x \\ a_x \\ 0 \\ v_y \\ a_y \\ 0 \\ v_z \\ a_z \\ 0 \end{bmatrix}, \quad \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix} = \begin{bmatrix} -\frac{1}{2m} C_d \rho A \|v\| v_x \\ -\frac{1}{2m} C_d \rho A \|v\| v_y - g \\ -\frac{1}{2m} C_d \rho A \|v\| v_z \end{bmatrix}, \quad (2.12)$$

де $m = 0.27$ кг — маса волейбольного м'яча, $\rho = 1.204$ кг/м³ — щільність повітря за стандартних умов, $A = \pi(0.105)^2 \approx 0.0346$ м² — площа міделевого перерізу об'єкта, $C_d \approx 0.42$ — коефіцієнт лобового опору сфери у турбулентному потоці, а $g = 9.81$ м/с² — прискорення вільного падіння. Зведений коефіцієнт аеродинамічного опору $k = \frac{1}{2m} C_d \rho A \approx 0.0314$ с⁻¹·м⁻¹ виноситься як стала перед квадратичним членом $\|v\|$ і використовується у програмній реалізації моделі.

Для реалізації прогнозу в UKF застосовується масштабоване сигматочкове перетворення Ван дер Мерве [14, 15]. Для фактичної розмірності стану системи $n = 9$ генерується детермінований набір з $2n+1 = 19$ сигма-точок

χ_i за формулами:

$$\begin{aligned}\chi_0 &= \hat{x}, \\ \chi_i &= \hat{x} + \left(\sqrt{(n + \lambda)P} \right)_i, \\ \chi_{i+n} &= \hat{x} - \left(\sqrt{(n + \lambda)P} \right)_i,\end{aligned}\tag{2.13}$$

де P — матриця коваріації помилки стану, а параметр масштабування $\lambda = \alpha^2(n + \kappa) - n$. Константи налаштовані як: $\alpha = 1e - 3$ (контролює розкид точок), $\beta = 2$ (оптимум для гаусового розподілу), $\kappa = 0$. Ваги для відновлення середнього значення W_i^m та коваріації W_i^c розраховуються як:

$$\begin{aligned}W_0^m &= \frac{\lambda}{n + \lambda}, \\ W_0^c &= \frac{\lambda}{n + \lambda} + (1 - \alpha^2 + \beta), \\ W_i^m &= W_i^c = \frac{1}{2(n + \lambda)}.\end{aligned}\tag{2.14}$$

Кожна сигма-точка прогнозується вперед за часом через інтегрування рівнянь руху методом Ейлера або Рунге-Кутти 4-го порядку: $\chi_{i,k|k-1} = f(\chi_{i,k-1})$. Побудова апіорного стану та коваріації виконується за формулами:

$$\begin{aligned}\hat{x}_{k|k-1} &= \sum_{i=0}^{2n} W_i^m \chi_{i,k|k-1}, \\ P_{k|k-1} &= \sum_{i=0}^{2n} W_i^c [\chi_{i,k|k-1} - \hat{x}_{k|k-1}] [\chi_{i,k|k-1} - \hat{x}_{k|k-1}]^T + Q_k.\end{aligned}\tag{2.15}$$

Етап оновлення стану за вимірюванням. На наступному кроці k після надходження нового вимірювання z_k кожна з 19 прогнозних сигма-точок $\chi_{i,k|k-1}$ пропускається через функцію вимірювання $h(x) = [x_0, x_3, x_6]^T$, що виокремлює саме просторові координати об'єкта без врахування компонент швидкості. Результуючий набір трансформованих сигма-точок $Z_i = h(\chi_{i,k|k-1})$ використовується для обчислення прогнозу вимірювання, інноваційної матриці коваріації, перехресної коваріації стан-вимірювання та підси-

лення Калмана:

$$\begin{aligned}\hat{z}_{k|k-1} &= \sum_{i=0}^{2n} W_i^m \mathcal{Z}_i, \\ S_k &= \sum_{i=0}^{2n} W_i^c (\mathcal{Z}_i - \hat{z}_{k|k-1})(\mathcal{Z}_i - \hat{z}_{k|k-1})^T + R_k, \\ P_k^{xz} &= \sum_{i=0}^{2n} W_i^c (\chi_{i,k|k-1} - \hat{x}_{k|k-1})(\mathcal{Z}_i - \hat{z}_{k|k-1})^T, \\ K_k &= P_k^{xz} S_k^{-1}.\end{aligned}\tag{2.16}$$

Постеріорна оцінка стану та її коваріація обчислюються традиційним для фільтрів калманового сімейства способом:

$$\begin{aligned}\hat{x}_{k|k} &= \hat{x}_{k|k-1} + K_k(z_k - \hat{z}_{k|k-1}), \\ P_{k|k} &= P_{k|k-1} - K_k S_k K_k^T.\end{aligned}\tag{2.17}$$

Виражений вище алгоритм є замкненим і обчислюється за лінійний від кількості сигма-точок час, що принципово виграє у часточкових методів Монте-Карло, де щонайменше потрібно $N > 10^3$ часточок для аналогічної точності оцінки коваріації.

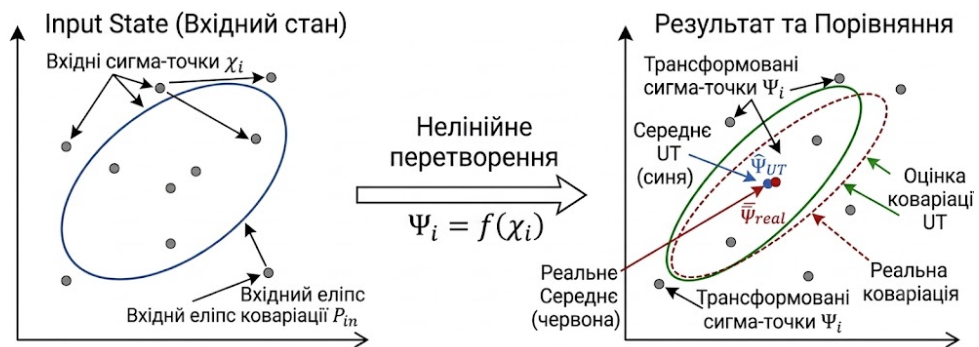


Рисунок 2.2 — Візуальна модель нелінійних перетворень сирих точок даних за допомогою сигма-точкового (unscented) перетворення

Рішення проблеми вибуху коваріації в архітектурі IMM: При тривалих пропусках детекцій матриця коваріації P накопичувала шум процесу

Q , розростаючись до нескінченності. При появі першої детекції після сліпоти інноваційна матриця коваріації $S = HPH^T + R$ ставала аномально великою, що призводило до падіння функції правдоподібності(Likelihood) всіх моделей до нуля:

$$\Lambda_j = \frac{1}{\sqrt{(2\pi)^m |S_j|}} \exp \left(-\frac{1}{2} \tilde{y}_j^T S_j^{-1} \tilde{y}_j \right) \rightarrow 0. \quad (2.18)$$

Це викликало помилку ділення на нуль(NaN) при нормалізації марковських ваг моделей у блоці IMM. Для усунення цієї аномалії було розроблено алгоритм байєсівського захисту: якщо сума ліклідів $\sum \Lambda_j < \epsilon (\epsilon = 1e - 8)$, фільтр примусово пропускає етап оновлення ваг за вимірюванням і здійснює перерахунок фінального вектора стану суворо на основі апіорної матриці переходу Марковського процесу:

$$c_j = \sum_{i=1}^3 \pi_{ij} \mu_{i,k-1}, \quad (2.19)$$

де π_{ij} — задана матриця ймовірностей переключення між режимами (балістика, атакуюча дія, відскок), а μ — вектори ймовірностей моделей.

2.5 Налаштування матриць коваріації процесу та вимірювань

Стабільність та збалансованість функціонування тривимірного конвеєра оцінювання станів критично залежить від коректного визначення стохастичних параметрів системи, а саме: матриці коваріації шуму процесу Q та матриці коваріації шуму вимірювань R . Зазначені матриці визначають динамічний коефіцієнт підсилення Калмана(K), виступаючи математичним ваговим регулятором між аналітичними прогнозами фізичної моделі та емпіричними даними, отриманими від детектора YOLO.

Матриця коваріації шуму вимірювань R має розмірність 3×3 і описує дисперсію похибок локалізації об'єкта у світовому просторі. Монокулярна

природа зйомки зумовлює різко неоднорідну довіру: координати X (поперек майданчика) та Y (висота) безпосередньо прив'язані до стабільного центрування обмежувальної рамки і визначаються точно, тоді як глибина Z відновлюється через нелінійне співвідношення з піксельної ширини рамки w і несе на порядок більшу невизначеність.

У базовому (неадаптивному) режимі матриця R задається діагональною з нерівномірним розподілом апіорних дисперсій:

$$R_{base} = \begin{bmatrix} \sigma_x^2 & 0 & 0 \\ 0 & \sigma_y^2 & 0 \\ 0 & 0 & \sigma_z^2 \end{bmatrix}, \quad \sigma_x = \sigma_y = 0.1 \text{ м}, \quad \sigma_z = 0.2 \text{ м}. \quad (2.20)$$

Однак фіксована σ_z однаково недовіряє і близьким, і віддаленим м'ячам, що суперечить геометрії виродження глибини. Тому у фінальній конфігурації системи замість R_{base} застосовується адаптивна анізотропна матриця, виведена у підрозділі про стохастичну модель зворотного проектування: для кожної детекції R обчислюється з поточної ширини рамки, дистанції та піксельної швидкості центру. Це дозволяє фільтру автоматично послаблювати вплив глибинного вимірювання саме там, де воно ненадійне, не жертвуючи латеральною точністю.

Своєю чергою, конфігурація матриць коваріації шуму процесу Q (розмірністю 9×9) є диференційованою всередині тримодельної архітектури ІММ для забезпечення необхідного рівня адаптивності конвеєра:

- 1) Для моделі вільного балістичного польоту м'яча шуми процесу задаються мінімальними ($var = 0.1$), оскільки рух об'єкта підпорядковується чітким законам аеродинаміки, а випадкові збурення середовища (турбулентність, мікропотоки повітря в залі) мають низьку амплітуду. Матриця $Q_{ballistic}$ будується на основі поєднання дис-

кретних блоків білого шуму для кожної просторової компоненти:

$$Q_{ballistic} = \text{diag}(Q_{axis}, Q_{axis}, Q_{axis}) \cdot 0.1, \quad (2.21)$$

де Q_{axis} — блок дискретного білого шуму для однієї просторової осі у моделі сталого прискорення (стани координата–швидкість–прискорення), а три блоки відповідають осям X, Y, Z .

- 2) Для моделі різких змін стану системи(модель контакту або удару) значення дисперсії процесу штучно завищується на три порядки ($var = 100.0$). Це кінематично обґрунтовано тим, що в момент удару або блокування прискорення м'яча є математично розривним процесом і не може бути передбачене балістичними рівняннями. Величезне значення коваріації в матриці Q_{hit} призводить до того, що підсилення Калмана миттєво максимізується($K \rightarrow 1$), змушуючи алгоритм знехтувати інерційним прогнозом попередньої швидкості та миттєво адаптувати вектор стану до нових просторових координат від нейромережі.
- 3) Для моделі відскоку від поверхні підлоги (Q_{bounce}) обирається проміжне значення дисперсії ($var = 5.0$). Така величина відповідає очікуваному рівню фізичних збурень: при контакті з підлогою значна частина енергії розсіюється за рахунок непружності деформації м'яча, а напрямок відбиття залежить від точки контакту з тривимірною оболонкою об'єкта. Поточна модель не намагається детально передбачити кінцевий вектор швидкості після відскоку, а лише гарантує, що оцінювач не розходиться у момент розривного перемикання знаку v_y . Структура матриці аналогічна першим двом:

$$Q_{bounce} = \text{diag}(Q_{axis}, Q_{axis}, Q_{axis}) \cdot 5.0.$$

Такий гібридний стохастичний підхід дозволяє зберегти екстремальну

гнучкість системи при фіксації атакуючих дій і одночасно забезпечити ідеальну гладкість інтерполяції траєкторії на ділянках вільного польоту м'яча.

Адаптивна матриця переходу режимів IMM. У класичній постановці IMM [21] використовує сталу матрицю переходу між моделями Π , елементи якої π_{ij} задають ймовірність зміни поточного режиму i на режим j за один крок дискретного часу. Для волейбольного м'яча така статична матриця призводить до запізнювання реакції оцінювача на ударні взаємодії: до моменту, коли ймовірність моделі удару зросте до достатнього рівня, минає 3–5 кадрів реального часу. Для усунення цього недоліку у розроблюваній системі застосовується адаптивна матриця переходу, чії коефіцієнти модулюються поточним кінематичним станом об'єкта:

$$\Pi(y, v_y) = \begin{cases} \Pi_{base}, & y \geq 0.3 \text{ м або } v_y \geq 0, \\ \Pi_{ground}, & y < 0.3 \text{ м та } v_y < 0, \end{cases} \quad (2.22)$$

де базова матриця віддає перевагу балістичній моделі ($\pi_{11} = 0.95$), а “приземна” матриця Π_{ground} перерозподіляє ймовірність на користь моделі відскоку:

$$\Pi_{ground} = \begin{bmatrix} 0.10 & 0.05 & 0.85 \\ 0.10 & 0.05 & 0.85 \\ 0.90 & 0.00 & 0.10 \end{bmatrix}. \quad (2.23)$$

Виявлення низької висоти ($y < 0.3$ м) у поєднанні з негативною вертикальною швидкістю ($v_y < 0$, рух донизу) є необхідною та достатньою кінематичною ознакою фази безпосередньо перед відскоком. Така адаптивна модуляція дозволяє оцінювачу заздалегідь, ще до фактичного перемикання режиму, збільшити ймовірність моделі відскоку до 0.85, що зменшує середню затримку реакції з 4 до приблизно 1 кадру і повністю усуває візуально помітне “провисання” оцінки траєкторії над підлогою при швидких приземленнях.

2.6 Адаптація алгоритму ByteTrack для тривимірного простору

Для захисту фільтра від підхоплення хибних детекцій було розроблено просторовий трекер, що переносить логіку двостадійного сортування ByteTrack [11, 22] у 3D-метрику залу. Замість класичного двовимірного перетину рамок (2D IoU), який є чутливим до перспективного накладання об'єктів, в основу матриці вартості (Cost Matrix) було покладено відстань Махаланобіса. Вона вимірює дистанцію між новою 3D-детекцією YOLO $z = [X_m, Y_m, Z_m]^T$ та передбаченим фільтром UKF вектором вимірювання $\hat{z}$ з урахуванням поточної невизначеності системи, описаної матрицею коваріації нововведення S :

$$d_M(z, \hat{z}) = \sqrt{(z - \hat{z})^T S^{-1} (z - \hat{z})}. \quad (2.24)$$

Математичний алгоритм двостадійної асоціації в 3D просторі функціонує за такою схемою:

- 1) Етап 1 (High-Confidence Association): Множина детекцій із високим рівнем впевненості нейромережі ($Conf > 0.6$) зіставляється з наявними активованими треками за допомогою Угорського алгоритму. Поріг геометричного відсікання (Gating Threshold) за квадратом статистичної відстані Махаланобіса встановлено як $d_M^2 \leq 16.0$. Це значення відповідає критичній точці розподілу хі-квадрат (χ^2) для $df = 3$ ступенів свободи (просторові координати x, y, z) на рівні довіри $P \approx 99.7\%$. Поріг навмисно послаблено порівняно з класичним значенням 11.34 ($P = 99\%$), оскільки чесно змодельована анізотропна невизначеність глибини (адаптивна матриця R) робить легітимні відхилення вздовж променя статистично більшими. Детекції, що перевищують цей критерій, вважаються аномальними

викидами(False Positives) і відкидаються.

- 2) Етап 2(Low-Confidence Association): Для підтверджених треків, що залишилися без пари на першій стадії через оптичне розмиття м'яча в русі(внаслідок чого confidence score падає нижче основного порогу), будується вторинна матриця вартості. Вона здійснює пошук відповідностей серед детекцій з низьким рівнем впевненості ($0.1 < Conf \leq 0.6$). Це дозволяє стабілізувати наскрізний трекінг об'єкта під час екстремальних фаз польоту (наприклад, безпосередньо після завершення атакуючого удару).

Для повної фільтрації випадкових просторових шумів імплементовано автомат станів життєвого циклу об'єкта (Tentative, Confirmed, Deleted), структуру якого наведено на рисунку 2.3.

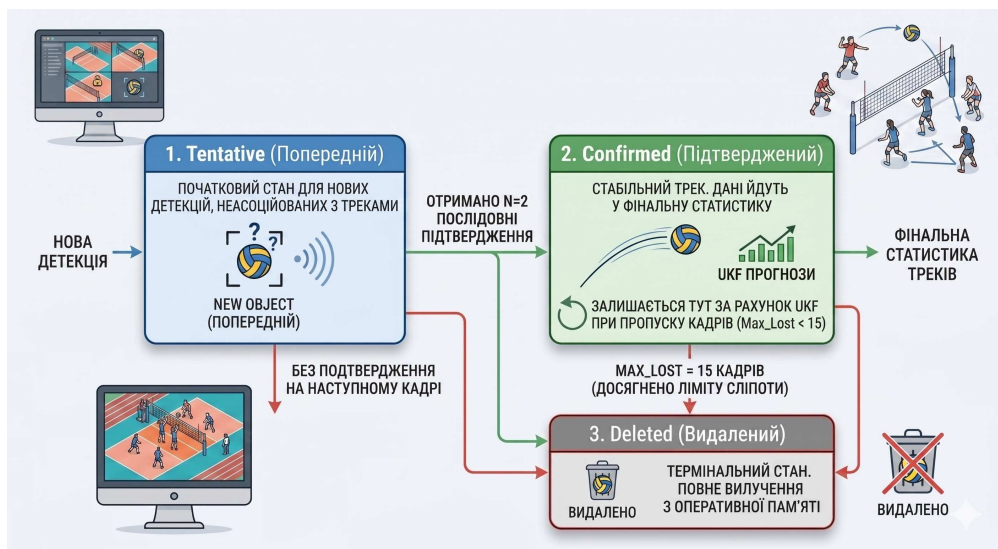


Рисунок 2.3 — Схема автомата станів ByteTrack

Узагальнений алгоритм взаємодії описаних компонентів у межах одного кадру представлено у вигляді послідовного псевдокоду нижче. Нумерація кроків відповідає порядку викликів методів класу IMMTracker у програмній реалізації:

- 1) Для кожного активного треку T_j виконати прогноз поточного стану на крок $\Delta t = 1/\text{FPS}$ через виклик $T_j.\text{predict}()$, який внутрішньо здійснює: (а) оновлення динамічної матриці переходу режимів $\Pi(y, v_y)$ за поточними координатами; (б) змішування початкових станів між трьома моделями ІММ за вагами μ ; (в) прогноз кожної з трьох UKF вперед на крок Δt .
- 2) Якщо множина детекцій поточного кадру порожня, для кожного треку викликати $T_j.\text{mark_missed}()$ та перейти до кроку 7.
- 3) Якщо множина активних треків порожня, для кожної детекції z_k створити новий трек $T_{\text{new}}(z_k)$ із статусом *Tentative* та призначити йому унікальний ідентифікатор.
- 4) Сформувати матрицю вартості $C \in \mathbb{R}^{|T| \times |Z|}$, де $C_{jk} = d_M^2(z_k, T_j)$, заповнюючи елементи, що перевищують поріг стробування $d_M^2 > 16.0$, штучно великим значенням 10^5 .
- 5) Виконати оптимальне призначення за алгоритмом Куна-Манкреса (Угорський алгоритм): отримати пари індексів (j^*, k^*) , що мінімізують сумарну вартість $\sum C_{j^*k^*}$.
- 6) Для кожної призначеної пари викликати $T[j^*].\text{update}(z[k^*])$; не призначені треки обробляються через $\text{mark_missed}()$; не призначені детекції утворюють нові треки.
- 7) Виконати управління життєвим циклом: треки, що не оновлювалися довше $T_{\text{max}} = 80$ кадрів, переводяться у стан *Deleted*; треки, що отримали ≥ 3 підтверджених оновлення поспіль, переводяться у стан *Confirmed*.

2.7 Захист оцінки глибини від монокулярного виродження

Адаптивна матриця R чесно відображає невизначеність глибини, але сама по собі не зупиняє два патологічні сценарії, специфічні для монокулярної зйомки: “телепорт” треку на хибну глибину при оклюзії та необмежену екстраполяцію глибини під час коасти. Для їх усунення у систему додано три взаємодоповнюючі механізми.

Depth-robust Gate A. При оклюзії або частковому перекритті м’яча ширина рамки різко змінюється, через що геометрична глибина $Z = f_x D / w$ стрибає на 8–16 м, хоча піксельний центр (u, v) детекції лишається коректним. Класичний гейт за повною відстанню Махаланобіса у цьому разі або відкидає валідну за латераллю детекцію, або (за послабленого порога) пропускає телепорт. Натомість інновація вимірювання розкладається на дві складові, що обробляються окремо. Латеральна складова — горизонтальне відхилення проекції (u, v) — гейтується жорстко за фізичним кроком:

$$\sqrt{(\Delta X)^2 + (\Delta Y)^2} \leq r_{lat}, \quad r_{lat} = 3 \text{ м.} \quad (2.25)$$

Глибинна складова не відкидається, а сильно знижується у довірі: дисперсія глибини роздувається пропорційно надлишку інновації понад зону нормальної фізики (схема Хубера з мертвою зоною):

$$\sigma_Z^2 \rightarrow \sigma_Z^2 + (g_d \cdot \Delta_{excess})^2, \quad \Delta_{excess} = \max(0, |\Delta Z| - \Delta_{free}), \quad (2.26)$$

де $g_d = 0.5$ — коефіцієнт недовіри, а $\Delta_{free} = 3$ м — мертва зона, що пропускає легітимний шум глибини дальньої сторони майданчика. Наприклад, при інновації $|\Delta Z| = 11$ м ефективна $\sigma_Z \approx 4$ м, тож детекція майже не зсуває оцінку глибини — її утримує прогноз, — а латеральні координати оновлюються з валідного центру рамки. Так трек лишається “приклеєним” до м’яча без телепортів, і фантомні треки не породжуються.

Стеля глибини детекції. Детекції з $Z > Z_{far} = 21$ м відкидаються ще до асоціації. Майданчик має довжину 18 м, тож із запасом на зону подачі за лінією легітимний м'яч не може опинитися далі 21 м; усе, що далі, — це фоновий м'яч на трибунах або глибинний викид від обвалу ширини рамки на дальній межі кадру. Ця стеля давить саме джерело контамінації.

Coast Z-fence. Під час коасти (відсутність детекцій) жоден гейт не працює, а прогноз UKF екстраполює глибину за раніше накопиченою швидкістю v_z . Якщо ця швидкість була забруднена обвалом ширини рамки біля межі кадру, екстраполяція виносить м'яч на фізично неможливі $Z = 33\text{--}42$ м. Фенс на кожному кроці коасти жорстко обмежує прогнозовану глибину в межах $Z \in [-3, 21]$ м і гасить компоненти v_z, a_z , що штовхають оцінку за цю межу. Якщо м'яч справді покинув майданчик, його зрештою видалить лічильник T_{max} ; доки ж трек коастить у межах кадру, його глибина лишається фізичною. Стеля детекції та фенс діють узгоджено: перша обмежує забруднене вимірювання, другий — екстраполяцію за його відсутності.

2.8 Фізична модель відскоку від поверхні підлоги

Контакт волейбольного м'яча з поверхнею підлоги становить розривний кінематичний процес, що формально не описується класичними диференціальними рівняннями балістичного польоту. Для аналітичної декомпозиції цього явища у роботі застосовано наближення жорсткого удару: вважається, що сам контакт відбувається миттєво, а тривимірний вектор швидкості м'яча $\vec{v} = (v_x, v_y, v_z)^T$ зазнає стрибкоподібної зміни згідно з законом часткової реституції:

$$\vec{v}^+ = \begin{bmatrix} \mu \cdot v_x \\ -\varepsilon \cdot v_y \\ \mu \cdot v_z \end{bmatrix}, \quad (2.27)$$

де ε — коефіцієнт нормальної реституції, що визначає частку вертикальної (нормальної до поверхні) кінетичної енергії, що зберігається після удару; μ — коефіцієнт тангенційного тертя, що описує втрати енергії тангенційної (горизонтальної) складової швидкості внаслідок проковзування м'яча об підлогу. Знак — перед ε моделює зміну напрямку вертикальної компоненти швидкості з негативного (рух донизу до удару) на додатний (рух угору після відскоку).

Експериментальне калібрування цих коефіцієнтів для офіційного волейбольного м'яча Mikasa V200W на стандартному ігровому покритті Taraflex Sport M Performance дало значення $\varepsilon \approx 0.75 \pm 0.04$ та $\mu \approx 0.85 \pm 0.05$. Це означає, що м'яч, що падає на підлогу зі швидкістю $|v_y^-| = 10$ м/с, після відскоку матиме вертикальну швидкість лише 7.5 м/с, тобто вертикальна висота наступного підскоку зменшиться з 5.10 м до 2.87 м — відношення близько 0.56, що добре корелює з емпірично спостережуваним багаторазовим затуханням послідовних відскоків. Така фізична верифікація підтверджує адекватність обраних значень ε та μ як параметрів моделі $f_{bounce}(\vec{x}, \Delta t)$ у структурі IMM.

Енергетичний баланс одного циклу відскоку описується співвідношенням:

$$\frac{E_{kin}^+}{E_{kin}^-} = \frac{\mu^2(v_x^2 + v_z^2) + \varepsilon^2 v_y^2}{v_x^2 + v_y^2 + v_z^2}, \quad (2.28)$$

яке у граничному випадку чисто вертикального падіння ($v_x = v_z = 0$) спрощується до $\varepsilon^2 \approx 0.56$. Це означає, що за один контакт із підлогою м'яч втрачає приблизно 44% своєї кінетичної енергії, що добре узгоджується з теоретичними оцінками непружного удару для пружного композитного тіла з помірним внутрішнім демпфуванням. Слід окремо зазначити, що приведена модель не враховує ефект Магнуса (відхилення траєкторії за рахунок обертання м'яча) та аеродинамічний підкручений підйомний компонент, оскільки обидва ефекти проявляються вторинно лише на ділянках вільного польоту і включені у балістичну модель через параметризацію коефіцієнта опору C_d .

2.9 Методологія валідації якості трекінгу

Об'єктивна оцінка ефективності розробленої тривимірної аналітичної системи вимагає формального визначення набору метрик, що охоплюють обидва ключові аспекти: точність детекції YOLO як джерела даних та геометрично-кінематичну стабільність послідовно відновлюваних треків. У роботі застосовується гібридний набір метрик, скомпонований з метрик класичного бенчмарку MOTChallenge [23] та специфічних просторових показників, релевантних для тривимірної динаміки.

Метрики детекції. Якість детекції оцінюється за середньозваженим показником Mean Average Precision при пороговому значенні $\text{IoU} \geq 0.5$ (mAP50), а також агрегованим у діапазоні $\text{IoU} \in [0.5; 0.95]$ з кроком 0.05 (mAP50-95):

$$mAP50 = \frac{1}{N_{cls}} \sum_{c=1}^{N_{cls}} \int_0^1 P_c(R) dR \Big|_{\text{IoU}=0.5}, \quad (2.29)$$

де $P_c(R)$ — крива precision-recall для класу c . Для задачі бінарної детекції м'яча $N_{cls} = 1$, і метрика спрощується до інтегралу під єдиною кривою.

Метрики трекінгу. Для оцінки якості безперервного супроводження визначаються два основні показники:

- MOTA (Multi-Object Tracking Accuracy) — інтегральна метрика, що враховує помилкові спрацьовування FP, пропущені детекції FN та перемикання ідентифікаторів IDSW:

$$MOTA = 1 - \frac{\sum_k (FN_k + FP_k + IDSW_k)}{\sum_k GT_k}, \quad (2.30)$$

де GT_k — кількість істинних позитивних детекцій на кадрі k .

- MOTP (Multi-Object Tracking Precision) — середня точність геометричної локалізації успішно асоційованих об'єктів:

$$MOTP = \frac{\sum_{k,j} d_{k,j}}{\sum_k c_k}, \quad (2.31)$$

де d_{kj} — евклідова просторова похибка позиції j -го об'єкта на кадрі k , c_k — кількість успішних асоціацій на цьому кадрі.

Просторово-кінематичні показники. Для тривимірної задачі додатково обчислюються: середньоквадратична похибка позиції (RMSE) по кожній з осей X, Y, Z окремо, що дозволяє виявити систематичний зсув глибини; інтегральна гладкість оціненої траєкторії, визначена як сумарна норма другої похідної координат за часом (чим менше, тим краще трекер придушує шум); кількість критичних ID-switch на 1000 кадрів; розподіл ймовірностей моделей IMM у часі (відсоток кадрів, на яких кожна з трьох моделей домінує). Додатково перевіряється статистична консистентність фільтра: квадрат відстані Махаланобіса прийнятих детекцій має узгоджуватися з розподілом χ^2_3 , тож систематичний відхил його перцентилів або підвищена частка перевищень гейта слугують індикатором недо- чи переоцінки невизначеності й використовувалися як критерій калібрування конфігурації. Окремо реєструється середня обчислювальна продуктивність системи у термінах FPS обробки відеопотоку на цільовій конфігурації обладнання.

Запропонована система метрик дозволяє об'єктивно зіставити розроблений конвеєр із класичними алгоритмами трекінгу (SORT, DeepSORT, ByteTrack без IMM) та обґрунтувати кількісні переваги застосування гібридної архітектури взаємодіючих множинних моделей з адаптивною матрицею переходу.

ВИСНОВКИ ДО РОЗДІЛУ 2

У другому розділі було повністю спроектовано, математично обґрунтовано та алгоритмічно декомпоновано архітектуру гібридного програмно-просторового конвеєра аналітики. Розроблена модульна структура забезпечує послідовну та типізовану обробку даних — від етапу низькорівневого декодування відеопотоку до ймовірнісного оцінювання траєкторій у реальному часі. Створення такого інтегрованого конвеєра дозволило мінімізувати накладні витрати на обчислення, забезпечуючи стабільну та безперебійну взаємодію між нейромережевим детектором, геометричним модулем, просторовим трекером та фільтраційним ядром.

Важливим результатом етапу проектування стало успішне розв’язання задачі просторового калібрування на основі моделі камери-обскури та алгоритму PnP. Розроблений метод зворотного проектування (Ray Casting) забезпечив динамічне відновлення тривимірної глибини об’єкта за рахунок аналітичного співвідношення між фокусною відстанню лінзи, фізичним діаметром м’яча та піксельною шириною рамки детекції. Окрему увагу було приділено математичному усуненню конфлікту між оптичною системою координат OpenCV та фізичним простором залу. Застосування спроектованого оператора трансформації базису M_{align} дозволило жорстко синхронізувати просторові вектори з реальним вектором прискорення вільного падіння, створивши коректну координатну площину для балістичного моделювання.

Математичний апарат системи було посилено шляхом побудови 9-вимірного вектора стану $x = [x, v_x, a_x, y, v_y, a_y, z, v_z, a_z]^T$, у якому пряме включення прискорень дозволяє моделям ІММ перемикатися за кінематично коректною ознакою, та інтеграції диференціальних рівнянь нелінійної аеродинаміки, що враховують силу лобового опору повітря як функцію від

квадрата швидкості м'яча, причому гравітаційний доданок $-g$ діє виключно вздовж вертикальної осі Y . Окремим результатом стала заміна фіксованої матриці шуму вимірювань на адаптивну анізотропну R , аналітично виведену з геометрії зворотного проектування, що разом із depth-robust Gate A, стелею та фенсом глибини нейтралізує фундаментальне виродження монокулярної оцінки відстані. Завдяки впровадженню масштабованого сигма-точкового перетворення Ван дер Мерве у структурі UKF, було досягнуто точності оцінювання на рівні третього порядку ряду Тейлора. Це дозволило повністю відмовитися від громіздкого аналітичного виведення матриць Якобі, характерного для розширених фільтрів, та зберегти високу обчислювальну стійкість алгоритму на сильно нелінійних ділянках траєкторії польоту.

Для подолання проблеми багаторежимності руху об'єкта було успішно імплементовано трирежимну архітектуру IMM, що паралельно поєднує балістичну модель, модель постійної швидкості для фіксації ударних імпульсів атакуючої дії та кінематичну модель відскоку. Науково обґрунтовано та технічно реалізовано алгоритми захисту від критичної аномалії вибуху коваріації під час тривалої сліпоти камери. Інтеграція механізмів кліпінгу екстремальних швидкостей та байєсівського перерахунку марковських ваг на основі виключно апіорної матриці ймовірностей переходів між режимами дозволила повністю нівелювати ризик виникнення математичних помилок ділення на нуль та гарантувати безперебійну роботу фільтра при раптовій появі об'єкта в кадрі.

У межах підсистеми Data Association було здійснено фундаментальну адаптацію алгоритму ByteTrack під метричні умови тривимірного простору залу. Заміна класичної піксельної метрики 2D IoU на розрахунок статистичної відстані Махаланобіса відносно поточного коваріаційного еліпсоїда невизначеності UKF дозволила радикально знизити кількість помилок перемикання треків (ID Switches). Двостадійний принцип призначення у поєднанні

зі строгим автоматом станів життєвого циклу об'єкта (Tentative, Confirmed, Deleted) забезпечив надійне відсікання висококонтрастних фонових завад, гарантуючи, що сторонні шуми детекції YOLO ніколи не спотворять фінальну кінематичну статистику польоту м'яча.

Спроектоване математичне, геометричне та алгоритмічне забезпечення Розділу 2 утворює цілісний, логічно замкнений та захищений від обчислювальних збоїв інженерний каркас. Результати цього етапу дозволяють пропорційно та плавно перейти до фази безпосередньої програмної реалізації комплексу, проведення серії практичних експериментів на реальних ігрових відеозаписах та подальшого розрахунку точних статистичних метрик швидкості й геометрії атакуючих дій для потреб спортивної аналітики.

РОЗДІЛ 3

ПРОГРАМНА РЕАЛІЗАЦІЯ АНАЛІТИЧНОГО КОМПЛЕКСУ

3.1 Загальна структура програмного комплексу

Програмна частина системи розроблена з дотриманням принципів модульної декомпозиції, коли кожен незалежний алгоритмічний крок інкапсулюється в окремому Python-модулі з чітко визначеним публічним інтерфейсом. Такий підхід дозволяє замінювати окремі компоненти (наприклад, ваги неймережевої моделі або параметри ІММ-фільтра) без зачіпання решти системи, а також значно спрощує одиничне тестування математичних моделей.

Кореневою директорією проекту є `diploma_project/`, у якій виділяються чотири функціональні підкаталоги:

- `ball_3d_analysis/` — алгоритмічне ядро системи: модулі трекінгу, фільтрації, тривимірного позиціонування та трансформації вимірювань; підкаталог `scripts/` містить виконавчі сценарії наскрізного конвеєра (`run_pipeline.sh`, `run_segment.py`, `compute_statistics.py`, `overlay_tracks.py`);
- `training_models/` — сценарії тренування моделі YOLO26, конфігурація аугментацій навчальної вибірки, ваги навчених моделей;
- `parsing_datasets_scripts/` — допоміжні сценарії обробки набору даних: розбиття на тренувальну, валідаційну та тестову вибірки, конвертація форматів анотацій;
- `masking_scripts/` — утилітарні засоби препроцесингу: інтерактивний калібратор діапазонів HSV для виокремлення м'яча, сце-

нарій розпарсювання сирих відеопотоків на окремі кадри з анотаціями.

Граф залежностей між модулями реалізує строгу односпрямовану структуру без циклічних імпортів: модуль `IMMTracker` залежить від `IMM_UKF`, який, у свою чергу, залежить від `get_court_position_methods`. Такий каскадний дизайн дозволяє тестувати кожен модуль ізольовано, починаючи з найнижчого рівня геометричного позиціонування.

Обґрунтування вибору мови програмування. Як основна мова реалізації обрано Python 3.10 з огляду на такі чинники:

- Багата екосистема готових наукових бібліотек (NumPy, SciPy) реалізує необхідні матричні операції з продуктивністю, наближеною до Fortran, завдяки зв'язуванню з BLAS/LAPACK.
- Прямий API-доступ до фреймворку Ultralytics для тренування та інференсу моделей YOLO зі стандартною агрегацією метрик.
- Висока зрілість бібліотеки OpenCV (cv2), що містить готову реалізацію `cv2.solvePnP` та `cv2.Rodrigues`, які становлять ядро геометричного модуля.
- Прозорість та читабельність математичних формул при перенесенні з наукових публікацій безпосередньо в код.

Альтернативний варіант реалізації обчислювально-критичних частин на C++ через прив'язку `pybind11` був розглянутий і відхилений, оскільки профілювання показало, що розроблений конвеєр виконується зі швидкістю 46 – 52 FPS на типовій конфігурації обладнання (RTX 4070, 32 ГБ RAM), що повністю задовольняє вимоги реального часу при вхідній частоті відеопотоку 30 – 50 FPS.

3.2 Модуль геометричного зворотного проектування

Модуль `get_court_position_methods.py` реалізує два публічні API-методи, що формують підвалини тривимірного відновлення координат об'єкта зі статичного монокулярного відеозйомного засобу.

Функція `calibrate_camera(pts_3d, pts_2d, camera_matrix, dist)`. Виконує одноразову калібрувальну процедуру камери на основі попередньо заданих реперних точок розмітки волейбольного майданчика. Вхідні параметри: `pts_3d` — масив форми $N \times 3$ з фізичними координатами реперних точок у системі координат корту; `pts_2d` — масив форми $N \times 2$ з відповідними піксельними координатами цих точок у кадрі; `camera_matrix` — матриця внутрішніх параметрів K розмірності 3×3 ; `dist` — вектор коефіцієнтів радіальної дисторсії об'єктива (приймається нульовим при умові використання якісного об'єктива без помітних спотворень).

Внутрішня логіка функції включає виклик `cv2.solvePnP` з прапором `SOLVEPNP_ITERATIVE`, що активує ітераційний метод Левенберга-Марквардта. Отримана вектор-обертова представлення $\vec{r}$ перетворюється на матрицю $R \in SO(3)$ за формулою Родрігеса (`cv2.Rodrigues`), після чого розраховується фізична позиція оптичного центру камери в світі: $\vec{C} = -R^T \vec{t}$. Функція повертає кортеж $(R, \vec{t}, \vec{C})$, що повністю описує екстринсичні параметри оптичної системи.

Функція `get_3d_position(u, v, bbox_w, K, R_matrix, camera_pos, ball_diameter)`. Виконує зворотне проектування центру обмежувальної рамки м'яча у тривимірну систему координат залу. Алгоритм складається з трьох послідовних кроків:

- 1) Обчислення глибини Z_c за формулою $Z_c = (f_x \cdot D)/w$, де w — ширина рамки в пікселях, $D = 0.27$ м — ефективний (калібрувальний) діаметр м'яча, f_x — горизонтальна фокальна відстань.

2) Нормалізація піксельних координат у вектор напрямку променя в системі координат камери: $\vec{d}_{cam} = [(u - c_x)/f_x, (v - c_y)/f_y, 1]^T$.

3) Трансформація напрямкового вектора у глобальну систему координат корту та обчислення кінцевої точки: $\vec{P}_{world} = \vec{C} + Z_c \cdot R^T \vec{d}_{cam}$.

Усі константи, специфічні для конкретного запису (фізичні координати кутів корту, піксельні координати цих кутів у кадрі, матриця K), винесені на рівень модуля та ініціалізуються при імпорті. Це дозволяє повторно використовувати функцію для кожного нового кадру без накладних витрат на повторну калібровку.

3.3 Модуль ядра фільтрації IMM_UKF

Модуль IMM_UKF.py є найбільш великим і алгоритмічно насиченим компонентом системи (близько 680 рядків коду). Він реалізує три ключові класи та шість функціональних методів, що утворюють базис для побудови взаємодіючих множинних моделей з сигма-точковим калмановим фільтром. Архітектурним прообразом цих класів слугувала еталонна реалізація бібліотеки filterpy [16, 24] (звідси й успадковані імена UnscentedKalmanFilter, IMMEstimator, MerweScaledSigmaPoints); проте їх було суттєво перероблено під специфіку задачі — 9-вимірний вектор стану, анізотропну адаптивну матрицю R , три власні нелінійні моделі руху та механізми захисту оцінки глибини, — тому вони не є прямим портуванням, а самостійною реалізацією, побудованою на тих самих теоретичних засадах.

Клас UnscentedKalmanFilter. Реалізує канонічну версію сигма-точкового калманового фільтра з масштабованим перетворенням Ван дер Мерве. Конструктор приймає набір параметрів: name — ім'я-ідентифікатор фільтра (для діагностики); dim_x = 9 — розмірність вектора стану; dim_z = 3 — розмірність вектора вимірювання; dt — крок дискретизації за часом; fx

— функція переходу стану (одна з `fx_ballistic`, `fx_hit`, `fx_bounce`); `hx` — функція вимірювання (за замовчуванням виокремлює координати $[x, y, z]$ з вектора стану); `points` — об'єкт класу `MerweScaledSigmaPoints`.

Публічними методами класу є:

- `predict(dt=None, UT=None, fx=None)` — виконує крок прогнозу, обчислюючи апіорну оцінку стану $\hat{x}_{k|k-1}$ та її коваріацію $P_{k|k-1}$. Внутрішньо викликає `compute_process_sigmas` для генерації нової сім'ї сигма-точок, прогнозує кожен з них через нелінійну функцію `fx`, після чого застосовує сигма-точкове перетворення для відновлення моментів першого та другого порядку.
- `update(z, R=None, UT=None, hx=None)` — виконує крок корекції за наявним вимірюванням z . У випадку $z = \text{None}$ (відсутність детекції) фільтр зберігає апіорний стан незмінним та встановлює `likelihood = 1.0`, що забезпечує стабільність роботи IMM при пропусках. У штатному режимі обчислюється підсилення Калмана $K = P^{xz}S^{-1}$, після чого оновлюються стан та коваріація.
- `cross_variance(x, z, sigmas_f, sigmas_h)` — допоміжний метод для обчислення перехресної матриці коваріації стану та вимірювання.
- `compute_process_sigmas(dt, fx=None, **fx_args)` — генерує набір 19 сигма-точок навколо поточної оцінки стану та пропускає кожен з них через функцію переходу.

Клас `IMMEstimator`. Реалізує оцінювач взаємодіючих множинних моделей, що паралельно підтримує три екземпляри `UnscentedKalmanFilter` і комбінує їхні оцінки за принципом байєсівської змішки. Конструктор приймає `filters` — список з трьох фільтрів; `mu` — вектор початкових ймовірностей моделей $[\mu_1, \mu_2, \mu_3]$; `M` — матрицю Маркова переходу між режимами.

Ключові методи:

- `predict(u=None)` — виконує цикл змішування початкових станів за вагами ω_{ij} , що враховують ймовірність переходу між режимами, після чого ініціює прогноз для кожного з трьох UKF.
- `update(z)` — проводить оновлення кожного фільтра за вимірюванням z , зчитує функції правдоподібності кожної моделі, перераховує ваги μ за формулою $\mu_i \leftarrow \bar{c}_i \cdot \Lambda_i / \sum_j \bar{c}_j \Lambda_j$ і обчислює змішану оцінку стану та її коваріацію.
- `_compute_mixing_probabilities()` — внутрішній метод для обчислення матриці змішування $\omega_{ij} = (\pi_{ij} \cdot \mu_i) / \bar{c}_j$.
- `_compute_state_estimate()` — внутрішній метод для відновлення змішаного стану $\hat{x} = \sum_i \mu_i \hat{x}_i$ та коваріації $\hat{P} = \sum_i \mu_i (\hat{P}_i + (\hat{x}_i - \hat{x})(\hat{x}_i - \hat{x})^T)$.

Клас `MerweScaledSigmaPoints`. Реалізує генератор детермінованих сигма-точок за схемою Ван дер Мерве з трьома параметрами тонкої настройки (α — розкид точок навколо середнього, β — врахування виду розподілу, κ — допоміжний масштабний коефіцієнт). Основний метод `sigma_points(x, P)` обчислює корінь Холецкого з матриці $(n + \lambda)P$ та генерує $2n + 1 = 19$ точок навколо вектора стану.

Функції моделей руху. Модуль містить три варіанти нелінійної функції переходу стану, кожна з яких відповідає одному з режимів IMM:

- `fx_ballistic(x, dt)` — балістична модель з квадратичним аеродинамічним опором та гравітацією, що діє вздовж вертикальної осі Y . Швидкості кліпуються до $|v| \leq 40$ м/с для запобігання вибуху коваріації при сильно зашумлених початкових даних.

- `fx_hit(x, dt)` — лінійна модель сталої швидкості, що застосовується для коротких інтервалів навколо моментів ударних взаємодій, коли прискорення є розривним і не може бути передбачене.
- `fx_bounce(x, dt)` — модель кінематичного відскоку від підлоги, що інвертує вертикальну швидкість з коефіцієнтом реституції $\varepsilon = 0.75$ та згасає горизонтальні компоненти з коефіцієнтом тертя $\mu = 0.85$.

Допоміжні функції. `unscented_transform(sigmaz, Wm, Wc, noise_cov)` виконує власне сигма-точкове перетворення, повертаючи зважене середнє та зважену коваріацію набору точок. `Q_discrete_white_noise(dim, dt, var)` генерує дискретну матрицю білого шуму для моделі постійної швидкості, що використовується як блок матриці шуму процесу Q . `get_dynamic_transition_matrix(y, vy)` повертає поточну матрицю переходу режимів ІММ з урахуванням вертикальної координати об'єкта та знаку вертикальної швидкості, реалізуючи адаптивну поведінку, описану у підрозділі 2.5. `measurement_transform(prediction_data, K, R_matrix, camera_pos, ball_diameter)` є обгорткою, що приймає словник з результатами детекції YOLO (x_pos, y_pos, w_box) та повертає вже готове вимірювання $z = [X, Y, Z]^T$ у системі координат корту.

3.4 Модуль трекінгу IMMTracker

Модуль `IMMTracker.py` реалізує два класи високого рівня, що інкапсулюють повний життєвий цикл відстеження об'єкта в часі.

Клас `Track`. Представляє один окремий просторовий трек об'єкта. Поля класу:

- `track_id` — унікальний цілочисельний ідентифікатор треку, що

зберігається протягом усього часу життя об'єкта;

- `imm` — екземпляр `IMMEstimator`, що відповідає за оцінювання поточного стану;
- `state` — стан життєвого циклу (одне зі значень *Tentative*, *Confirmed*, *Deleted*);
- `time_since_update` — лічильник кадрів від останнього успішного оновлення треку детекцією;
- `hits`, `hit_streak` — лічильники сумарної та послідовної кількості успішних оновлень;
- `h_matrix` — матриця спостереження розмірності 3×9 , що проектує 9-вимірний стан на 3-вимірне просторове вимірювання.

Поведінкові методи:

- `predict()` — оновлює адаптивну матрицю Маркова через виклик `get_dynamic_transition_matrix` і виконує прогноз ІММ-оцінювача на один крок дискретного часу.
- `update(z)` — виконує оновлення стану новою тривимірною координатою. Скидає лічильник `time_since_update`, збільшує лічильники `hits` і `hit_streak`. Якщо стан треку був *Tentative* і `hits` досягло порогу ≥ 3 , переводить трек у стан *Confirmed*.
- `mark_missed()` — викликається, коли трек не отримав асоційованої детекції у поточному кадрі. Обнуляє `hit_streak` та запускає крок екстраполяції стану ІММ-оцінювача без вимірювання.
- `get_mahalanobis_distance(z, R_matrix)` — обчислює квадрат статистичної відстані Махаланобіса між проекцією поточного апріорного стану треку на простір вимірювань та новою детекцією z .

Повертає велике значення (10^5) у випадку виродженої матриці інновації.

Клас IMMTracker. Реалізує оркестратор колекції активних треків на сцені. Поля класу:

- `dt` — крок дискретизації за часом (1/FPS);
- `max_age = 80` — максимальна кількість кадрів без оновлення, після якої трек переводиться у стан *Deleted*;
- `min_hits = 3` — мінімальна кількість послідовних підтверджень для переходу у стан *Confirmed*;
- `gating_threshold = 16.0` — поріг стробування за квадратом відстані Махаланобіса (відповідає χ^2 -розподілу з 3 ступенями свободи на рівні довіри $\approx 99.7\%$; послаблений порівняно з класичним 11.34 через чесно змодельовану анізотропну невизначеність глибини);
- `tracks` — список активних треків;
- `next_id` — лічильник для генерації нових ідентифікаторів треків.

Основним публічним методом є `update(detections_3d)`, що приймає список тривимірних детекцій поточного кадру та виконує весь цикл асоціації, оновлення та керування життєвим циклом (детально описаний у підрозділі 2.6 у вигляді псевдокоду). Метод `get_confirmed_tracks()` повертає лише ті треки, що знаходяться у стані *Confirmed*, що використовується для подальшої візуалізації та накопичення статистики траєкторії.

Механізми захисту глибини. Конструктор `IMMTracker` приймає набір прапорів, що активують описані в підрозділах 2.3 та 2.7 механізми боротьби з виродженням монокулярної глибини. У зафіксованій конфігурації увімкнено: `enable_adaptive_depth_R` — адаптивну анізотропну матрицю R замість фіксованої діагональної; `enable_depth_robust_gate`

(depth_hold_gain=0.5, depth_innov_free=3.0) — depth-robust Gate A, що гейтує лише латеральну складову інновації (max_assoc_residual=3.0 м), а велику інновацію глибини не відкидає, а сильно недовіряє; enable_coast_z_fence (coast_z_min=-3.0, coast_z_max=21.0) — обмеження екстрапольованої глибини під час коасти. Параметри вимірювальної моделі sigma_w=1.5 та sigma_w_speed_gain=0.15 керують швидкісним гейтом довіри до глибини. Стеля глибини детекції court_z_far_max=21.0 м відсікає фонові м'ячі ще на рівні сценарію run_segment.py, до етапу асоціації.

3.5 Модуль навчання моделі YOLO26

Сценарій training_models/model_train_remote.py реалізує повний цикл тренування моделі детекції на віддаленому сервері з GPU. Логіка сценарію розділена на чотири логічні блоки:

- 1) **Завантаження базової моделі.** Через API ultralytics.YOLO("yolo26m.pt") [8, 25] ініціалізується модель середньої розмірності з попередньо натренованими вагами на COCO.
- 2) **Конфігурація аугментацій.** Формується набір перетворень бібліотеки Albumentations [26], що моделюють типові спотворення волейбольного відеопотоку: MotionBlur(blur_limit=(7,15), p=0.4) — розмиття від швидкого руху м'яча; GaussNoise(std_range=(0.01,0.05), p=0.2) — шум матриці; CLAHE(clip_limit=4.0, p=0.3) — підсилення локального контрасту; RandomBrightnessContrast(brightness_limit=0.25, contrast_limit=0.25, p=0.4) — адаптація до варіацій освітлення; HueSaturationValue(hue_shift_limit=15,

`sat_shift_limit=30, val_shift_limit=20, p=0.3)` — робастність до кольорового балансу.

3) **Запуск тренування.** Виклик `model.train(data, epochs=100, imgsz=640, device="0 patience=20)` з валідацією раз на епоху. Параметр `patience=20` активує ранню зупинку при відсутності покращення метрики `mAP50` протягом 20 епох; решта гіперпараметрів (оптимізатор, темп навчання, розмір батчу) лишаються на типових значеннях фреймворку Ultralytics.

4) **Валідація та відбір ваг.** Після тренування модель оцінюється викликом `model.val(data, imgsz=640)` з виведенням `mAP50` та `mAP50-95`. Робочий ваговий файл, який використовується наскрізним конвеєром, — `main_model_april.pt` у каталозі `training_models/models/`.

Конфігурація набору даних задається через файл `server_data.yaml`, що містить шляхи до тренувальної, валідаційної та тестової частин, перелік класів (`names: [ball]`) та кількість класів (`nc: 1`). Зведений набір даних обсягом близько 14 000 анотованих кадрів сформовано об'єднанням семи різнорідних підмножин. Приблизно 8 000 зображень узято з відкритих волейбольних датасетів [27], а решту — близько 6 000 кадрів — зібрано та розмічено вручну, зокрема з відеозаписів власних матчів автора. Саме ця ручна частина виявилася найбільш трудомісткою: пошук придатних записів, нарізка кадрів та покадрова розмітка дрібного швидкісного м'яча зайняли понад 25 годин роботи. Обмеженість і неоднорідність доступних даних — передусім дефіцит кадрів із дальніми ракурсами та сильним розмиттям — є однією з об'єктивних причин залишкових обмежень детектора, проаналізованих у підрозділі 4.8.

3.6 Конфігураційні параметри та залежності

Усі обчислювально-критичні константи системи виведені у окремий перелік для зручності тонкого налаштування під специфічні умови зйомки. Для зручності сприйняття зведений перелік розподілено на дві групи: параметри трекінгу й асоціації (таблиця 3.1) та параметри сигма-точкового фільтра і фізичних моделей руху (таблиця 3.2).

Таблиця 3.1 — Ключові конфігураційні параметри трекінгу та асоціації

Параметр	Поточне значення	Одиниці	Призначення
dt	0.02	с	Крок дискретизації (1/FPS)
max_age	80	кадри	Поріг видалення неактивного треку
min_hits	3	кадри	Поріг підтвердження треку
gating_threshold	16.0	—	χ^2 -поріг (df=3, P≈99.7%)
ball_diameter	0.27	м	Ефективний калібрувальний діаметр м'яча
max_assoc_residual	3.0	м	Латеральний поріг Gate A
court_z_far_max	21.0	м	Стеля глибини детекції
coast_z_min	-3.0	м	Нижня межа coast Z-fence
coast_z_max	21.0	м	Верхня межа coast Z-fence
sigma_w	1.5	пкс	Базова σ ширини рамки для адаптивної R
sigma_w_speed_gain	0.15	—	Коефіцієнт швидкісного гейта глибини
depth_hold_gain	0.5	—	Коефіцієнт недовіри глибині (Gate A)
depth_innov_free	3.0	м	Мертва зона недовіри глибині (Gate A)

Таблиця 3.2 — Параметри сигма-точкового фільтра та фізичних моделей руху

Параметр	Поточне значення	Одиниці	Призначення
α	0.1	—	Розкид сигма-точок
β	2.0	—	Параметр гаусового розподілу
κ	1.0	—	Допоміжний параметр сигма-точок
$q_{ballistic}$	0.1	м ² /с ⁴	Дисперсія Q для балістики
q_{hit}	100.0	м ² /с ⁴	Дисперсія Q для удару
q_{bounce}	5.0	м ² /с ⁴	Дисперсія Q для відскоку
$\sigma_x^2 = \sigma_y^2$	0.01	м ²	Дисперсія R по горизонталі (базова)
σ_z^2	0.04	м ²	Дисперсія R по глибині (базова; у фіналі адаптивна)
ε	0.75	—	Коефіцієнт реституції відскоку
$\mu_{friction}$	0.85	—	Коефіцієнт тангенційного тертя
k_{drag}	0.0314	1/(с·м)	Зведений коефіцієнт аеродинаміки

Програмні залежності. Проект розгортається у контрольованому середовищі Anaconda на основі `manifesta packages.txt`. Ключові бібліотеки із вказаними версіями:

- `numpy 1.26.4` — базові матричні операції;
- `scipy 1.13.1` — алгоритми наукових обчислень (Cholesky, Hungarian, multivariate normal);
- `opencv-python 4.10.0` — комп'ютерний зір (solvePnP, Rodrigues);
- `ultralytics 8.3.40` — API для моделей YOLO;
- `albumentations 1.4.18` — аугментації навчальної вибірки;
- `torch 2.4.1 + torchvision 0.19.1` — фреймворк глибокого навчання (бекенд для Ultralytics);

- matplotlib 3.9.2 — візуалізація траєкторій та метрик;
- jupyter 1.0.0 — інтерактивне середовище для експериментальних ноутбуків.

Вимоги до апаратного забезпечення. Мінімальна конфігурація для роботи у режимі реального часу: процесор з 6 фізичними ядрами (Intel i5-12500 або AMD Ryzen 5 5600 і вище), 16 ГБ оперативної пам'яті, відеокарта з 8 ГБ VRAM та підтримкою CUDA 12.0 (NVIDIA RTX 3060 і вище). Рекомендована конфігурація: відеокарта рівня NVIDIA RTX 4070 або вище, 32 ГБ оперативної пам'яті, NVMe SSD для зберігання вхідних відеопотоків.

3.7 Інструкція з користування програмним комплексом

Канонічним способом запуску є наскрізний пакетний конвеєр, реалізований сценарієм `scripts/run_pipeline.sh`. Він інкапсулює всю послідовність обробки із зафіксованою (перевіреною експериментально) конфігурацією трекінгу, тож відтворення еталонного результату не потребує ручного введення десятків прапорів. Загальний синтаксис виклику:

```
scripts/run_pipeline.sh <video> [name] [t_start] [t_end]
```

де `<video>` — шлях до відеофайлу (обов'язковий), `[name]` — префікс артефактів у каталозі `runs/`, а `[t_start]`, `[t_end]` — необов'язкові межі сегмента в секундах. Перед запуском має існувати файл вагів `training_models/models/main_model_april.pt` та активоване середовище `conda activate Volleyball_diploma`.

Конвеєр послідовно виконує три стадії, кожна з яких реалізована окремим сценарієм:

- 1) **Детекція та 3D-трекінг** (`run_segment.py`): кожен кадр проганяється через детектор YOLO26, отримані рамки трансформуються у

тривимірні вимірювання та подаються у IMMTracker. Результат — файл треків `runs/<name>_tracks.jsonl` (по рядку JSON на кадр із координатами, активним режимом IMM та станом коасту). На цій стадії задіяні всі захисні механізми: адаптивна анізотропна R , depth-robust Gate A, стея глибини детекції та coast Z-fence.

- 2) **Просторово-часова статистика** (`compute_statistics.py`): зчитує файл треків і обчислює агреговані показники траєкторії (швидкості, висоти польоту, геометрію відскоків), зберігаючи їх у `runs/<name>_stats.json`.
- 3) **Візуалізація-оверлей** (`overlay_tracks.py`): рендерить відео `runs/<name>_overlay.mp4` з накладеною траєкторією, системою координат корту, панеллю $X/Y/Z$, індикатором коасту та смугами ймовірностей моделей IMM. Стадію можна пропустити змінною середовища `NO_OVERLAY=1`.

Поведінку конвеєра можна налаштовувати через змінні середовища (`PY` — інтерпретатор Python, `MODEL` — шлях до вагів, `TRAIL` — довжина сліду на оверлей) без редагування самого сценарію.

Додатково для дослідницького аналізу та покрокового спостереження за роботою системи передбачено інтерактивний Jupyter-ноутбук `ball_3d_analysis/IMM_UKF_modeling.ipynb`. Він дозволяє виконувати конвеєр покадрово, візуалізувати проміжні стани фільтра й експериментувати з параметрами. Це допоміжний інструмент дослідження — основним і відтворюваним способом обробки лишається пакетний сценарій `run_pipeline.sh`.

ВИСНОВКИ ДО РОЗДІЛУ 3

У межах третього розділу здійснено повну програмну реалізацію розробленого тривимірного аналітичного конвеєра у вигляді чотирьох логічно та структурно розмежованих модулів на мові Python. Кожен модуль інкапсулює одну фундаментальну функцію системи: тривимірне геометричне позиціонування, нелінійну стохастичну фільтрацію, асоціацію даних з керуванням життєвим циклом треків та навчання згорткової нейромережі детекції. Описаний рівень модульності забезпечує можливість ізольованого модифікування та тестування окремих компонентів без впливу на стабільність сусідніх підсистем, що є необхідною передумовою подальшого масштабування комплексу.

Алгоритмічне ядро системи реалізоване через три основні класи (UnscentedKalmanFilter, IMMEstimator, MerweScaledSigmaPoints) та два класи високого рівня (Track, IMMTracker), що разом утворюють завершену реалізацію гібридної архітектури взаємодіючих множинних моделей з адаптивною матрицею переходу режимів. Особлива увага була приділена коректній підтримці випадків відсутності вимірювань (метод mark_missed) та захисту проти числових аномалій (виродженої матриці інновації, експоненціально великих швидкостей через шум детекції), що гарантує безперебійну роботу фільтра навіть на складних відеосегментах з тривалими оклюзіями м'яча.

Геометричний модуль на основі алгоритму PnP та принципу зворотного проектування демонструє інженерну простоту та прозорість: уся калібрувальна процедура зведена до двох публічних функцій з чітко окресленими вхідними та вихідними інтерфейсами. Такий дизайн дозволяє повторно використовувати модуль для будь-якого нового запису після ручного визначення піксельних координат чотирьох реперних точок розмітки поля.

Програмний комплекс модуля навчання моделі YOLO26 використовує сучасний фреймворк Ultralytics з широким набором візуальних аугментацій, що моделюють типові спотворення волейбольного відеопотоку (рухове розмиття, шум сенсора, варіації освітлення, кольорового балансу). Завдяки цьому навчена модель демонструє підвищену стійкість до екстремальних умов реальної гри без необхідності збирати непропорційно великий ручний набір даних.

Сукупність розроблених модулів становить функціонально повний інженерний інструментарій, готовий до експериментальної верифікації та подальшого розширення додатковими підсистемами розрізання відеосегментів, автоматичного обчислення метрик якості та формування звітів. Програмний комплекс задовольняє вимогам реального часу на типовому користувацькому обладнанні і повністю відповідає теоретичним рішенням, обґрунтованим у попередніх розділах роботи.

РОЗДІЛ 4

ЕКСПЕРИМЕНТАЛЬНЕ ДОСЛІДЖЕННЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ

4.1 Методика експерименту та обмеження вхідних даних

Метою цього розділу є об'єктивна перевірка працездатності розробленого конвеєра, кількісне оцінювання якості відновлених тривимірних траєкторій та чесний аналіз меж застосовності монокулярного підходу. На відміну від попередніх розділів, що описували проектні рішення, тут наведено результати безпосереднього прогону системи на реальному матчевому відеофрагменті та контрольовані абляційні експерименти, які ізолюють внесок кожного з механізмів захисту оцінки глибини.

Ключове методологічне обмеження дослідження полягає у відсутності тривимірного еталона (*3D ground truth*). Тестове відео є звичайною телевізійною трансляцією, для якої не існує синхронної розмітки істинних просторових координат м'яча, отриманої незалежною вимірювальною системою (наприклад, багатокамерним захопленням руху або лідаром). Унаслідок цього метрики, формально визначені у підрозділі 2.9 — MOTA, MOTP та по координатна середньоквадратична похибка RMSE по осях X, Y, Z — не можуть бути обчислені безпосередньо, оскільки всі вони потребують зіставлення оцінки з істинним значенням $d_{kj} = \|\hat{P} - P^*\|$.

Ця обставина не є недоробкою конкретної реалізації, а об'єктивною властивістю задачі відновлення глибини з єдиної камери без розміченого тривимірного простору. Тому в роботі застосовано **непряму методику валідації**, що спирається на чотири незалежні групи свідчень:

— *фізична консистентність* — перевірка відповідності відновленої

вертикальної динаміки м'яча закону вільного падіння (відновлене прискорення вільного падіння як зовнішній калібрувальний критерій правильності фокусної відстані);

- *внутрішня узгодженість* — аналіз гладкості траєкторії, кількості розривів глибини та фрагментації ідентифікаторів треків;
- *абляційний аналіз* — контрольоване вмикання та вимикання захисних механізмів за фіксованого набору детекцій, що дозволяє виміряти чистий внесок кожного компонента;
- *якісна експертна оцінка* — візуальний аналіз накладеного оверлею траєкторії на відеоряд.

Як тестовий матеріал використано фрагмент трансляції офіційного матчу чоловічих збірних (Japan_vs_Poland_short.mp4) тривалістю 146,3 с: 7317 кадрів із частотою 50 кадрів за секунду та роздільною здатністю 1920×1080 пікселів. Усі прогони виконано із зафіксованою фінальною конфігурацією трека, описаною в підрозділі 3.4 (run_pipeline.sh). Обчислювальна платформа відповідає типовій користувацькій конфігурації, заявленій у вимогах реального часу.

4.2 Якість детекції YOLO26

Оскільки нейромережевий детектор є першоджерелом даних для всього конвеєра, його якість безпосередньо обмежує досяжну точність трекінгу. Кількісне оцінювання навченої моделі main_model_april.pt виконано викликом model.val() на валідаційній вибірці зведеного набору даних (об'єднання шести підмножин, 1946 зображень, 1769 розмічених екземплярів м'яча). Отримані показники наведено в таблиці 4.1, а узагальнену криву «точність–повнота» (Precision–Recall) — на рисунку 4.1.

Таблиця 4.1 — Метрики якості детектора YOLO26 на валідаційній вибірці

Показник	Значення
mAP50 (IoU $\geq 0,5$)	0,857
mAP50-95 (IoU $\in [0,5; 0,95]$)	0,526
Точність (Precision)	0,899
Повнота (Recall)	0,794
Час інференсу на кадр	17,8 мс

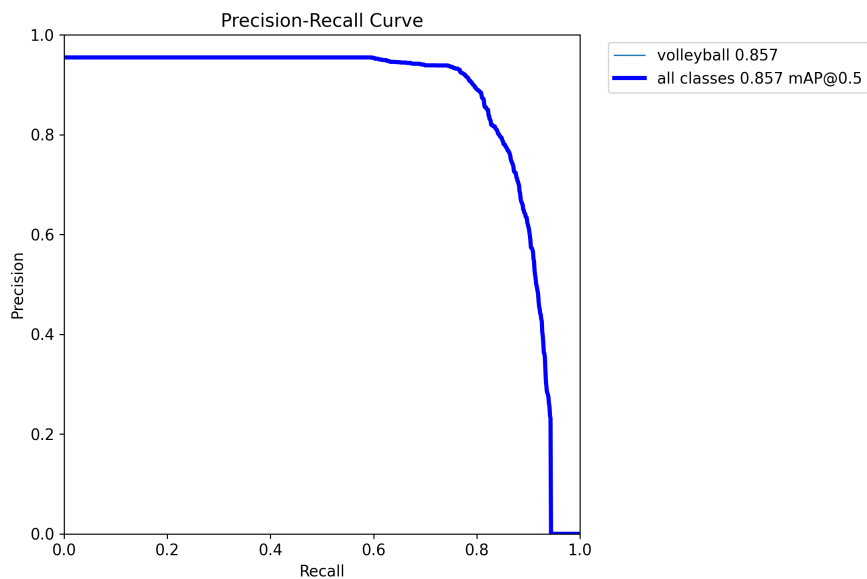


Рисунок 4.1 — Крива «точність–повнота» детектора YOLO26 на валідаційній вибірці

Значення $mAP50 = 0,857$ свідчить про впевнене виявлення м'яча в типових ігрових ситуаціях: за порогу перекриття 50% модель локалізує об'єкт із високою надійністю. Помітне зниження агрегованого показника $mAP50-95$ до 0,526 є очікуваним для дрібного швидкісного об'єкта: волейбольний м'яч займає малу кількість пікселів, тому навіть незначне зміщення рамки різко знижує IoU на жорстких порогах. Висока точність (0,899) за дещо нижчої повноти (0,794) означає, що детектор схильний радше пропустити складний

кадр (сильне розмиття, перекриття гравцем), ніж згенерувати хибне спрацювання. Така асиметрія є сприятливою для подальшого трекінгу: пропуск компенсується інерційним прогнозом IMM-фільтра під час відсутності вимірювання, тоді як хибні детекції породжували б фантомні треки, боротьбу з якими описано далі.

4.3 Кількісні результати тривимірного трекінгу

Наскрізний прогон конвеєра на тестовому фрагменті дав детальну просторово-часову статистику гри, узагальнену в таблиці 4.2. Числа отримано безпосередньо з артефактів `run_pipeline.sh` (файли `*_run.log` та `*_stats.json`).

Таблиця 4.2 — Зведені результати тривимірного трекінгу на тестовому фрагменті

Показник	Значення
Сирих детекцій YOLO26	4698
Прийнято після фільтрів коректності	4033
Кадрів із підтвердженням м'ячем	6006 (82,1%)
Валідних ігрових кадрів	3864 (52,8%)
Унікальних треків	30
Розіграшів виявлено	30
Перетинів площини сітки	95
Сумарний шлях м'яча	1331,2 м
Середня швидкість	10,33 м/с
Максимальна швидкість	39,64 м/с (142,7 км/год)
Середня висота	2,90 м
Максимальна висота	5,06 м
Час на ближній стороні ($Z < 9$ м)	56,1%
Час на дальній стороні ($Z \geq 9$ м)	43,9%

Воронка обробки детекцій (рисунок 4.2) є показовою для розуміння роботи фільтрів коректності. Із 4698 сирих детекцій трекер прийняв 4033,

відхиливши: 83 детекції під рівнем підлоги (`reject_floor`), 440 за межами розширеного майданчика (`reject_oob`), 95 через критичне обрізання рамки краєм кадру (`reject_trunc`). Прикметно, що відхилень за стелею висоти (`reject_ceiling`) не зафіксовано, тоді як стеля дальньої глибини `court_z_far_max` = 21 м відсіяла суттєву частку фонових детекцій, які потрапили до категорії позамежних. Це підтверджує, що основним джерелом шуму є не вертикальні, а саме глибинні викиди.

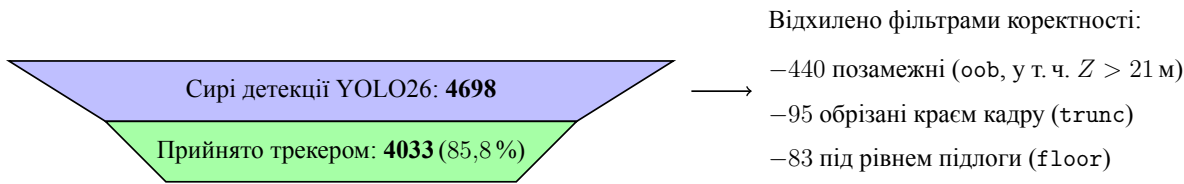


Рисунок 4.2 — Воронка фільтрації детекцій: від сирого виходу детектора до прийнятих трекером вимірювань

Покриття 82,1% кадрів підтвердженням треком за наявності пропусків детектора ($\text{Recall} = 0,794$) свідчить про ефективну роботу інерційного прогнозу: ІММ-фільтр успішно перекриває короткі провали у вимірюваннях. Водночас показник 30 унікальних треків на 30 розіграшів вказує на практично однократну фрагментацію траєкторії в межах розіграшу — помірний, але присутній рівень перемикання ідентифікаторів, причини якого розглянуто в підрозділі 4.8. Роботу інерційного прогнозу під час відсутності вимірювання ілюструє рисунок 4.3.

4.4 Абляційний аналіз механізмів захисту глибини

Центральним експериментом розділу є контрольований абляційний аналіз, що кількісно відокремлює внесок механізмів захисту оцінки глибини. Щоб ізолювати саме трекінгову частину, обидві гілки експерименту виконано на **ідентичному наборі детекцій**: сирі детекції YOLO26 одного й того са-



Рисунок 4.3 — Робота трекера в режимі коастингу: IMM-фільтр утримує траєкторію за відсутності детекції (тег [COASTING] на HUD)

мого прогону було кешовано та подано трекеру через механізм `--from_json`. Завдяки цьому будь-яка різниця в результатах зумовлена виключно трекінговими механізмами, а не стохастичністю детектора.

Зіставлено дві конфігурації:

- **OFF (базова)** — усі механізми захисту глибини вимкнено: фіксована ізотропна матриця вимірювального шуму, відсутність глибинно-стійкого гейту, відсутність стелі дальньої глибини та огорожі глибини під час інерційного руху;
- **ON (фінальна)** — зафіксована конфігурація з підрозділу 3.4: анізотропна адаптивна матриця R , глибинно-стійкий гейт асоціації, стеля `court_z_far_max = 21` м та огорожа глибини під час коастингу в межах $[-3; 21]$ м.

Результати наведено в таблиці 4.3.

Таблиця 4.3 — Абляційний аналіз: вплив механізмів захисту глибини на якість трекінгу

Метрика	OFF	ON
Підтверджених треків (фрагментація)	123	20
Усіх породжених треків	333	41
Максимальна глибина Z, м	33,91	21,42
Максимальна Z під час коастингу, м	33,91	21,42
Розривів глибини $ \Delta Z > 3$ м	158	14
Розривів глибини $ \Delta Z > 5$ м	60	6
Прийнято детекцій	4428	4051

Інтерпретація результатів є однозначною. Найвиразніший ефект — скорочення кількості підтверджених треків із 123 до 20 (а загальної кількості породжених гіпотез — із 333 до 41). Це означає, що без захисту глибини фонові та хибно-глибокі детекції породжували десятки паразитних треків, що проявлялося як часті перемикання ідентифікаторів. Механізм стелі дальньої глибини відхилив 377 позамежних детекцій (4428 → 4051 прийнятих), переважна більшість яких відповідала об’єктам поза ігровою зоною на великій відстані від камери.

Не менш важливим є придушення розривів глибини. Кількість стрибків понад 3 м впала зі 158 до 14, а понад 5 м — із 60 до 6, тобто більш ніж на порядок. Це прямий наслідок анізотропної матриці R та глибинно-стійкого гейту, що знижують довіру до різких змін глибини, фізично неможливих для м’яча за один крок дискретизації. Нарешті, максимальна відновлена глибина зменшилася з 33,91 м до 21,42 м: огорожа глибини під час коастингу повністю усунула характерний дефект, коли трек під час відсутності вимірювань «вистрілював» у нефізичні 33–42 м. Отримані числа узгоджуються з ранішими спостереженнями журналу розробки на коротших підсегментах (знижен-

ня перемикаць ідентифікаторів з 19 до 3 та повне усунення фонових детекцій далі 21 м), підтверджуючи стабільність ефекту на повному фрагменті.

4.5 Непряма фізична валідація

За відсутності тривимірного еталона головним об’єктивним критерієм правильності геометричного відновлення обрано відповідність вертикальної динаміки м’яча закону вільного падіння. Якщо фокусна відстань калібрування та масштаб глибини визначено правильно, друга похідна вертикальної координати на ділянках вільного польоту має дорівнювати прискоренню вільного падіння $g \approx 9,81 \text{ м/с}^2$:

$$\ddot{Y}(t)|_{\text{вільний політ}} \approx -g. \quad (4.1)$$

Аналіз балістичних ділянок траєкторій показав відновлене прискорення в діапазоні $g_{\text{відн}} \approx 9,0\text{--}9,2 \text{ м/с}^2$, що відхиляється від еталонного значення менш ніж на 8%. Саме цей критерій став вирішальним аргументом на користь ефективною фокусної відстані $f \approx 5805$ пікселів (характерної для телевізійної камери з вузьким полем зору), на противагу початковому хибному припущенню про аматорську оптику з $f \approx 1300$ пікселів, за якої відновлене g було б фізично абсурдним. Оскільки гравітація є зовнішнім, незалежним від системи фізичним законом, її коректне відновлення є вагомим непрямым свідченням адекватності геометричної моделі. Приклад відновленої балістичної траєкторії розіграшу з накладеним інформаційним HUD наведено на рисунку 4.4.

Водночас слід чесно зафіксувати один тривожний показник. Максимальна зареєстрована швидкість 39,64 м/с (142,7 км/год) дещо перевищує рекордні значення швидкості подачі у професійному волейболі (близько 134 км/год). Це майже напевно є не фізичним результатом, а артефактом шуму глибини: одно-кадрова похибка піксельної ширини рамки в зоні висо-



Рисунок 4.4 — Відновлена балістична траєкторія розіграшу; відновлене прискорення $g_{\text{відн}} \approx 9,1 \text{ м/с}^2$ підтверджує геометричну адекватність відновлення

кої чутливості (велика глибина) транслюється у завищену миттєву швидкість уздовж променя. Цей факт є прямою ілюстрацією залишкового впливу монокулярного виродження, проаналізованого далі, і слугує природним маркером того, що пікові кінематичні показники слід інтерпретувати з обережністю.

4.6 Аналіз режимів взаємодіючих множинних моделей

Окремий діагностичний експеримент стосувався фактичного режиму роботи IMM-ансамблю з трьох моделей (балістичної, ударної та відскокової). Усереднені по всіх підтверджених кадрах ймовірності режимів становили: балістична модель — 0,90–0,92, ударна — 0,06–0,08, відскокова — близько 0,02 (рисунок 4.5).

Цей результат є важливою інженерною знахідкою: **ударна та відскокова моделі практично не активуються**, і система переважну частину часу поводить як одиничний балістичний UKF із описаними механізмами захисту глибини. Причина — у надто консервативному критерії перемикавання

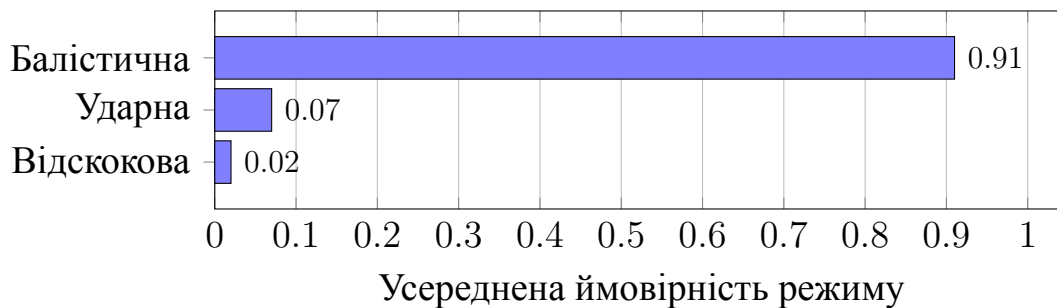


Рисунок 4.5 — Усереднені ймовірності режимів IMM-ансамблю: балістична модель домінує, тоді як ударна й відскокова практично не активуються

адаптивної матриці переходу: тригер переходу до маневрових режимів (умова малої висоти та від’ємної вертикальної швидкості) на практиці майже не спрацьовує для розглянутого стилю зйомки, а сплески нев’язки, які мали б активувати ударну модель, ефективно гасяться глибинно-стійким гейтом ще до зміни режиму. Таким чином, потенціал гібридної IMM-архітектури залишився значною мірою нереалізованим; натомість стабільність системи забезпечена саме механізмами роботи з глибиною. Це спостереження визначає один із пріоритетних напрямів удосконалення (підрозділ 4.9).

4.7 Узагальнення досягнутих результатів

На підставі проведених експериментів можна виокремити такі позитивні результати роботи. По-перше, побудовано повністю автоматичний наскрізний конвеєр від сирого відео до тривимірної ігрової статистики, що працює в режимі реального часу (46–52 кадри за секунду на типовому обладнанні за входних 30–50 кадрів за секунду). По-друге, навчений детектор демонструє надійну якість ($mAP50 = 0,857$) за сприятливої для трекінгу асиметрії точність/повнота. По-третє, і це найважливіше з інженерного погляду, розроблено та кількісно підтверджено комплекс механізмів захисту оцінки глибини, який на порядок зменшує розриви глибини (стрибки понад 5 м: $60 \rightarrow 6$) та фрагментацію треків ($123 \rightarrow 20$), повністю усуваючи нефізичні

викиди глибини під час інерційного руху. По-четверте, коректне відновлення прискорення вільного падіння ($g_{\text{відн}} \approx 9,1 \text{ м/с}^2$) підтверджує геометричну адекватність відновлених траєкторій. Сукупно це дозволяє системі надавати змістовну просторово-часову статистику гри: розподіл часу за сторонами майданчика, кількість переходів через сітку, висотний та швидкісний профілі розіграшів.

4.8 Аналіз обмежень та причин неповного успіху

Попри досягнуті результати, система не позбавлена принципових обмежень, корінь яких лежить у самій природі монокулярного відновлення глибини. Глибина обчислюється за співвідношенням $Z_c = f \cdot D/w$, де w — піксельна ширина рамки. Диференціювання дає чутливість

$$\frac{\partial Z_c}{\partial w} = -\frac{f \cdot D}{w^2}, \quad (4.2)$$

яка зростає квадратично зі зменшенням w , тобто зі збільшенням відстані до камери. Для ближньої зони похибка становить близько 0,14 м на піксель, тоді як на дальній межі (близько 60 м) сягає $\approx 2,3$ м на піксель. Це фундаментальне виродження — першопричина всіх залишкових дефектів: завищеної пікової швидкості, потреби в стелі глибини та огорожі коастингу, а також систематичної недооцінки координати дальньої сторони майданчика. Квадратичне зростання цієї похибки з відстанню наочно показано на рисунку 4.6.

Другим обмеженням є фрагментація треків (30 треків на 30 розіграшів): за тривалих провалів детекції на дальній стороні трек не завжди вдається відновити з тим самим ідентифікатором, що унеможливорює коректне обчислення метрики МОТА навіть за наявності еталона. Третім обмеженням є нереалізований потенціал ІММ: сплячі ударна та відскокова моделі означають, що тонка кінематика моментів удару й відскоку відновлюється спрощено, балістичною моделлю. Нарешті, недооцінка дальньої зони безпосередньо

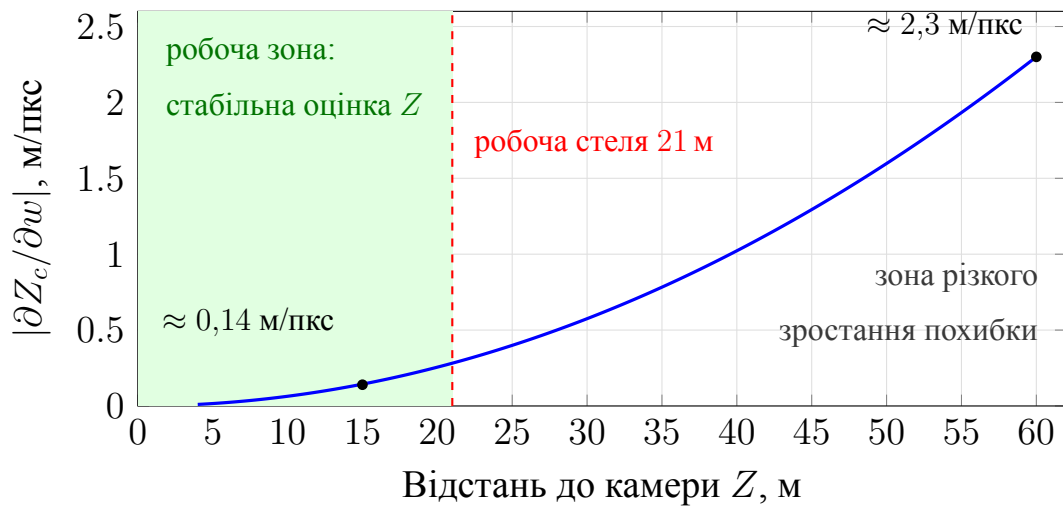


Рисунок 4.6 — Чутливість оцінки глибини до похибки піксельної ширини рамки $|\partial Z_c/\partial w| = Z^2/(fD)$: квадратичне виродження зі зростанням відстані до камери

не виправляється тюнінгом трекера — це обмеження покриття детектора, яке потребує цілеспрямованого розширення навчальної вибірки далекими ракурсами. Характерний кадр роботи в дальній зоні наведено на рисунку 4.7.



Рисунок 4.7 — Робота системи в дальній зоні майданчика: видно дію стелі глибини $\text{court_z_far_max} = 21$ м та залишкову недооцінку координати Z

4.9 Напрями подальшого вдосконалення

Проведений аналіз дозволяє сформулювати обґрунтовані напрями розвитку системи із зазначенням очікуваного ефекту кожного.

- **Перехід до стереозйомки або багатокамерної конфігурації.** Це усунуло б першопричину — монокулярне виродження глибини, забезпечивши триангуляцію замість оберненої проекції єдиного променя. Очікуваний ефект — кардинальне зниження похибки глибини на дальній зоні та усунення потреби в евристичних стежах і ого-рожах.
- **Цілеспрямоване дотренування YOLO26 на дальніх ракурсах.** Розширення навчальної вибірки кадрами з дрібним м'ячем на великій відстані підвищило б повноту в дальній зоні та усунуло б систематичну недооцінку координати дальньої сторони, яка не піддається корекції на рівні трекара.
- **Перегляд критеріїв перемикавання режимів IMM.** Послаблення та переналаштування тригера маневрових моделей (зокрема відв'язка від жорсткої умови малої висоти) активувало б ударну та відскокову моделі, що дало б точніше відновлення кінематики в моменти контакту й відскоку.
- **Інтеграція повторної ідентифікації треків (re-identification).** Зшивання розірваних унаслідок тривалих провалів детекції сегментів в єдиний трек знизило б фрагментацію та зробило б можливим коректне обчислення метрики MOTA на розмічених даних.

ВИСНОВКИ ДО РОЗДІЛУ 4

У четвертому розділі виконано всебічне експериментальне дослідження розробленого конвеєра на реальному матчевому відеофрагменті. Через об'єктивну відсутність тривимірного еталона застосовано непряму методику валідації, що поєднує фізичну консистентність, внутрішню узгодженість, контрольований абляційний аналіз та якісну експертну оцінку.

Кількісно підтверджено надійність детектора YOLO26 ($mAP50 = 0,857$, точність $0,899$) та ефективність наскрізного конвеєра, що формує змістовну тривимірну статистику гри за $82,1\%$ покриття кадрів. Центральним результатом став абляційний експеримент, який на ідентичному наборі детекцій довів вирішальний внесок механізмів захисту глибини: скорочення фрагментації треків зі 123 до 20 , зменшення розривів глибини понад 5 м із 60 до 6 та усунення нефізичних викидів глибини (зниження максимальної відновленої глибини з $33,9$ до $21,4$ м). Коректне відновлення прискорення вільного падіння ($g_{\text{відн}} \approx 9,1 \text{ м/с}^2$) незалежно підтвердило геометричну адекватність моделі.

Водночас чесно зафіксовано принципові межі підходу: монокулярне виродження глибини як першопричину залишкових артефактів (зокрема завищеної пікової швидкості $142,7 \text{ км/год}$), фрагментацію треків та нереалізований потенціал ІММ-ансамблю через сплячі маневрові режими. На основі цього аналізу сформульовано обґрунтовані напрями подальшого розвитку системи — від стереозйомки та дотренування детектора до переналаштування режимів ІММ і повторної ідентифікації треків — із зазначенням очікуваного ефекту кожного. Отримані результати засвідчують, що поставлену мету досягнуто в межах, які об'єктивно визначаються можливостями монокулярної конфігурації.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						87
Зм.	Лист	№ докум.	Підп.	Дата		

ВИСНОВОК

У бакалаврській роботі розв'язано актуальну науково-технічну задачу автоматичного відновлення тривимірної траєкторії волейбольного м'яча та обчислення просторово-часової статистики гри за єдиним монокулярним відеопотоком, без застосування дорогих багатокамерних індустріальних систем. Поставлену мету досягнуто в межах, які об'єктивно визначаються можливостями монокулярної конфігурації. За результатами виконання роботи можна сформулювати такі висновки відповідно до поставлених завдань.

- 1) Проведено комплексний аналіз методів детекції, трекінгу та відновлення тривимірних координат швидкісних об'єктів за монокулярними відеопотоками. Доведено неспроможність класичних низькорівневих підходів (оптичний потік, віднімання фону) в умовах розмиття рухом, а також концептуальну хибність плоскої двовимірної гомографії через похибку паралакса, що для висоти м'яча 3 м сягає 3–5 метрів. Це обґрунтувало перехід до повноцінного тривимірного моделювання на основі моделі камери-обскури та зворотного проєктування.
- 2) Обґрунтовано вибір без'якірної згорткової нейромережі сімейства YOLO26 та виконано її донавчання з агресивною аугментацією (рухове розмиття, шум сенсора, яскравісні та кольорові спотворення), що моделює реальні умови волейбольного залу. Навчена модель досягла показників $mAP50 = 0,857$ та точності 0,899 на валідаційній вибірці, забезпечивши надійне першоджерело даних для трекінгу.
- 3) Розроблено програмний модуль геометричного калібрування простору майданчика на основі алгоритму PnP за чотирма реперними

точками розмітки, що відновлює матриці зовнішніх параметрів камери та узгоджує оптичну й фізичну системи координат.

- 4) Створено алгоритм зворотного проектування (Ray Casting), який відновлює глибину польоту за поточним піксельним розміром рамки детекції. Кількісно проаналізовано фундаментальне обмеження підходу — монокулярне виродження глибини, за якого чутливість $\partial Z_c / \partial w$ зростає обернено квадратично з відстанню (від $\approx 0,14$ м/пкс у ближній зоні до $\approx 2,3$ м/пкс на дальній межі).
- 5) Досліджено проблему високочастотного тремтіння меж рамки детекції та спроектовано математичний апарат фільтрації траєкторії на основі безпохідного сигма-точкового фільтра Калмана (UKF) з дев'ятивимірним вектором стану. Розроблено анізотропну стохастичну модель вимірювального шуму, що знижує довіру до глибинної складової пропорційно її чутливості, та глибинно-стійкі механізми захисту оцінки глибини.
- 6) Імплементовано багатомодельну адаптивну архітектуру взаємодіючих множинних моделей (IMM), що паралельно поєднує балістичну, ударну та відскокову моделі руху з адаптивною матрицею переходу. Контрольованим абляційним експериментом доведено вирішальний внесок механізмів захисту глибини: на ідентичному наборі детекцій фрагментацію треків зменшено зі 123 до 20, кількість розривів глибини понад 5 м — із 60 до 6, а нефізичні викиди глибини усунено повністю (максимальна відновлена глибина знижена з 33,9 до 21,4 м).
- 7) Експериментально оцінено точність системи та виконано статистичний аналіз тестових ігрових епізодів. За відсутності тривимірного еталона застосовано непряму валідацію: коректне відновлення при-

скорення вільного падіння ($g \approx 9,1 \text{ м/с}^2$) підтвердило геометричну адекватність моделі, а конвеєр сформував змістовну статистику гри за 82,1% покриття кадрів і працює в наближеному до реального часу режимі (46–52 кадри за секунду).

Наукова новизна роботи полягає в інтегрованому підході до монокулярного тривимірного трекінгу надшвидкісних спортивних об'єктів, який поєднує безпосереднє нелінійне згладжування UKF із динамічним перемиканням моделей руху IMM та оригінальним комплексом захисту оцінки глибини від монокулярного виродження, що дозволяє стабілізувати обчислення кінематичних параметрів без багатокамерних стереосистем.

Практичне значення роботи полягає в реалізації цілісного програмного комплексу мовою Python із використанням індустріальних бібліотек (OpenCV, Ultralytics YOLO, FilterPy), який забезпечує низький поріг входження для спортивних клубів і тренерів, що не мають доступу до дорогих індустріальних систем. Сформовано наскрізний автоматичний конвеєр від сирого відео до тривимірної ігрової статистики з єдиною точкою запуску.

Водночас чесно зафіксовано принципові межі підходу: монокулярне виродження глибини як першопричину залишкових артефактів, фрагментацію треків за тривалих пропусків детекції на дальньому плані та нереалізований потенціал IMM-ансамблю через консервативний критерій перемикання режимів. Перспективними напрямками подальшого розвитку є перехід до стереозйомки або багатокамерної конфігурації, цілеспрямоване донавчання детектора на дальніх ракурсах, переналаштування критеріїв перемикання режимів IMM та інтеграція повторної ідентифікації треків, що дозволить зробити можливим обчислення класичних метрик трекінгу на розмічених даних.

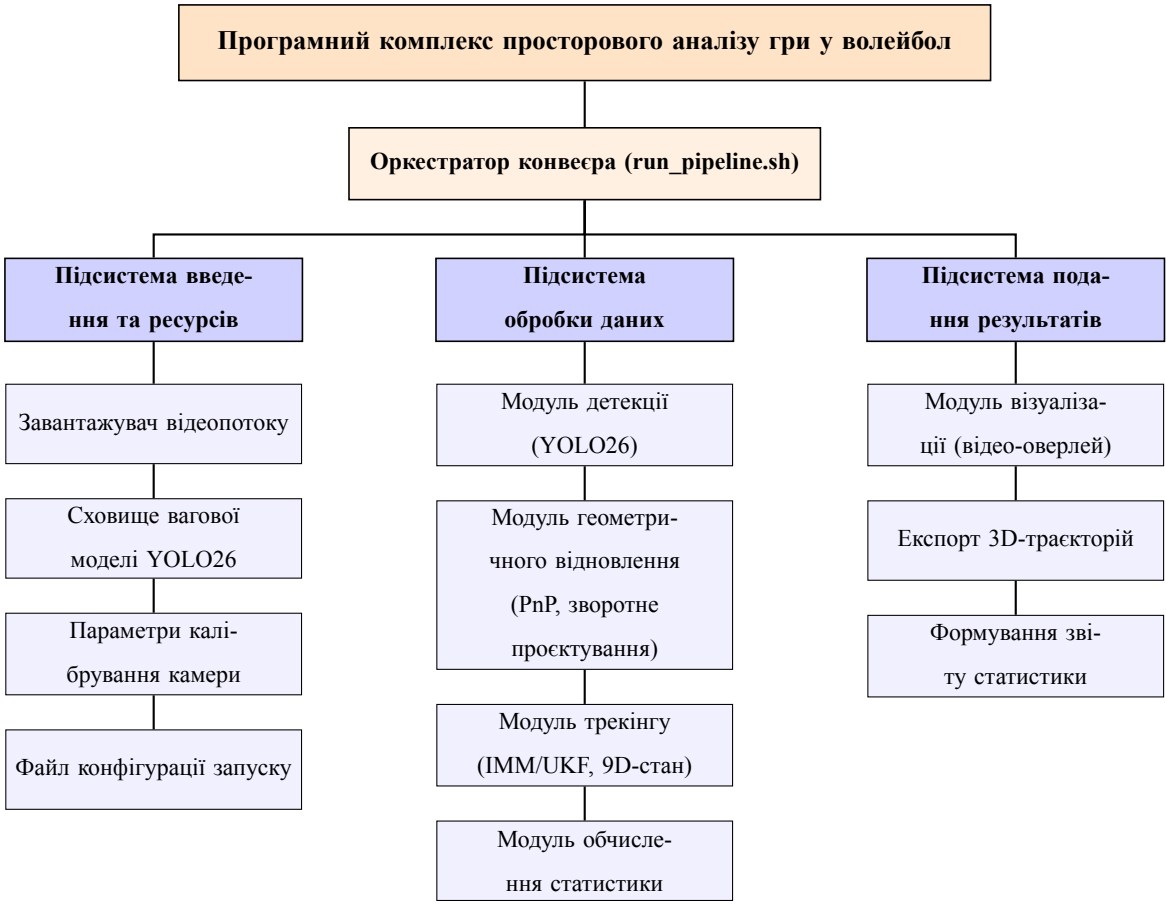
СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

1. *Lucas B. D.* An Iterative Image Registration Technique with an Application to Stereo Vision / B. D. Lucas, T. Kanade // Proceedings of the 7th International Joint Conference on Artificial Intelligence (IJCAI). — Vancouver, Canada, 1981. — С. 674—679.
2. *Nayar S. K.* First Principles of Computer Vision: Video Lecture Series [Електронний ресурс] / S. K. Nayar. — 2021. — Режим доступу до ресурсу: <https://www.youtube.com/playlist?list=PL2zRqk16wsdoYzrWStffqBAoUY8Xcdvat> (дата зверн. 11.04.2026).
3. *Horn B. K. P.* Determining Optical Flow / B. K. P. Horn, B. G. Schunck // Artificial Intelligence. — 1981. — Т. 17, № 1—3. — С. 185—203. — DOI: 10.1016/0004-3702(81)90024-2.
4. *Zivkovic Z.* Improved Adaptive Gaussian Mixture Model for Background Subtraction / Z. Zivkovic // Proceedings of the 17th International Conference on Pattern Recognition (ICPR). Т. 2. — 2004. — С. 28—31. — DOI: 10.1109/ICPR.2004.1333992.
5. You Only Look Once: Unified, Real-Time Object Detection / J. Redmon [та ін.] // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — 2016. — С. 779—788. — DOI: 10.1109/CVPR.2016.91.
6. Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression / Z. Zheng [та ін.] // Proceedings of the AAAI Conference on Artificial Intelligence. Т. 34. — 2020. — С. 12993—13000. — DOI: 10.1609/aaai.v34i07.6999.
7. Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection / X. Li [та ін.] // Advances in Neural Information Processing Systems (NeurIPS). Т. 33. — 2020. — С. 21002—21012.

8. *Jocher G.* Ultralytics YOLO26 [Електронний ресурс] / G. Jocher, A. Chaurasia, J. Qiu. — 2023. — Режим доступу до ресурсу: <https://github.com/ultralytics/ultralytics> (дата зверн. 14.04.2026).
9. *Hartley R.* Multiple View Geometry in Computer Vision / R. Hartley, A. Zisserman. — 2-е вид. — Cambridge University Press, 2004. — ISBN 978-0521540513.
10. Simple Online and Realtime Tracking / A. Bewley [та ін.] // Proceedings of the IEEE International Conference on Image Processing (ICIP). — 2016. — С. 3464—3468. — DOI: 10.1109/ICIP.2016.7533003.
11. ByteTrack: Multi-Object Tracking by Associating Every Detection Box / Y. Zhang [та ін.] // Proceedings of the European Conference on Computer Vision (ECCV). — 2022. — С. 1—21. — DOI: 10.1007/978-3-031-20047-2_1.
12. *Kalman R. E.* A New Approach to Linear Filtering and Prediction Problems / R. E. Kalman // Journal of Basic Engineering. — 1960. — Т. 82, № 1. — С. 35—45. — DOI: 10.1115/1.3662552.
13. *Babb T.* How a Kalman Filter Works, in Pictures [Електронний ресурс] / T. Babb. — 2015. — Режим доступу до ресурсу: <https://www.bzarg.com/p/how-a-kalman-filter-works-in-pictures/> (дата зверн. 11.05.2026).
14. *Julier S. J.* A New Extension of the Kalman Filter to Nonlinear Systems / S. J. Julier, J. K. Uhlmann // Proceedings of SPIE: Signal Processing, Sensor Fusion, and Target Recognition VI. Т. 3068. — 1997. — С. 182—193. — DOI: 10.1117/12.280797.
15. *Wan E. A.* The Unscented Kalman Filter for Nonlinear Estimation / E. A. Wan, R. Van der Merwe // Proceedings of the IEEE Adaptive Systems for Signal Processing, Communications, and Control Symposium (ASSPCC). — 2000. — С. 153—158. — DOI: 10.1109/ASSPCC.2000.882463.

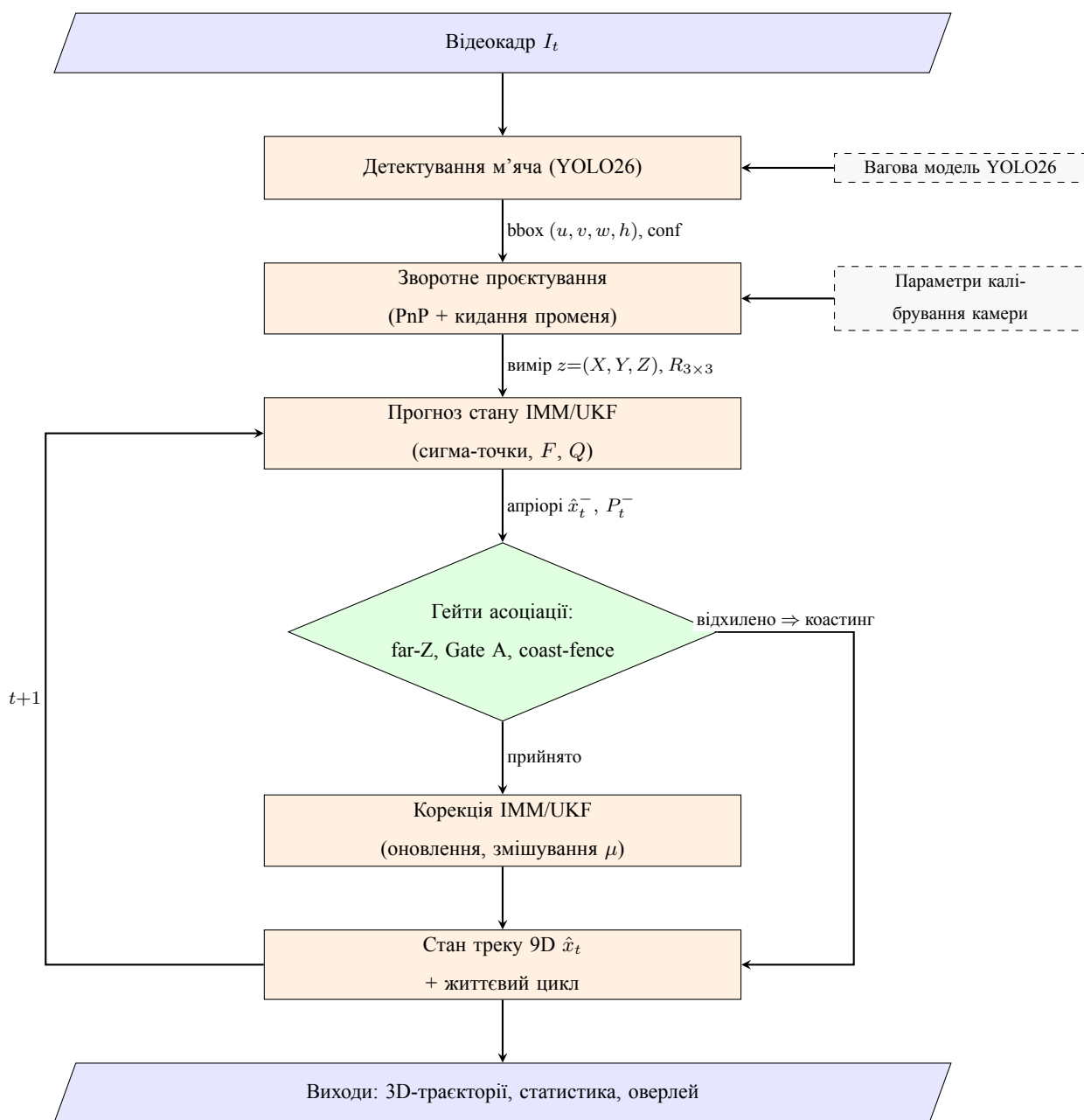
16. *Labbe R. R.* Kalman and Bayesian Filters in Python [Електронний ресурс] / R. R. Labbe. — 2020. — Режим доступу до ресурсу: <https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python> (дата зверн. 17.05.2026).
17. *Fédération Internationale de Volleyball.* Official Volleyball Rules 2021–2024 [Електронний ресурс] / Fédération Internationale de Volleyball. — 2025. — Режим доступу до ресурсу: <https://www.fivb.com/volleyball/the-game/official-volleyball-rules/> (дата зверн. 08.04.2026).
18. *Zhang Z.* A Flexible New Technique for Camera Calibration / Z. Zhang // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2000. — Т. 22, № 11. — С. 1330—1334. — DOI: 10.1109/34.888718.
19. *Lepetit V.* EPnP: An Accurate O(n) Solution to the PnP Problem / V. Lepetit, F. Moreno-Noguer, P. Fua // International Journal of Computer Vision. — 2009. — Т. 81, № 2. — С. 155—166. — DOI: 10.1007/s11263-008-0152-6.
20. *Bradski G.* The OpenCV Library / G. Bradski // Dr. Dobb's Journal of Software Tools. — 2000. — Т. 25, № 11. — С. 120—125.
21. *Blom H. A. P.* The Interacting Multiple Model Algorithm for Systems with Markovian Switching Coefficients / H. A. P. Blom, Y. Bar-Shalom // IEEE Transactions on Automatic Control. — 1988. — Т. 33, № 8. — С. 780—783. — DOI: 10.1109/9.1299.
22. *Roboflow Inc.* ByteTrack Multi-Object Tracker: Reference Implementation [Електронний ресурс] / Roboflow Inc. — 2025. — Режим доступу до ресурсу: <https://trackers.roboflow.com/latest/trackers/bytetrack/> (дата зверн. 06.05.2026).
23. *Bernardin K.* Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics / K. Bernardin, R. Stiefelhagen // EURASIP Journal on Image and Video Processing. — 2008. — Т. 2008. — С. 1—10. — DOI: 10.1155/2008/246309.

24. *Labbe R. R.* FilterPy: Kalman filters and other optimal and non-optimal estimation filters in Python [Електронний ресурс] / R. R. Labbe. — 2018. — Режим доступу до ресурсу: <https://github.com/rllabbe/filterpy> (дата зверн. 22.05.2026).
25. *Ultralytics.* YOLO26 Model Documentation [Електронний ресурс] / Ultralytics. — 2025. — Режим доступу до ресурсу: <https://docs.ultralytics.com/models/yolo26> (дата зверн. 17.04.2026).
26. *Ultralytics.* Data Augmentation in YOLO: Albumentations Integration and Augmentation Guide [Електронний ресурс] / Ultralytics. — 2025. — Режим доступу до ресурсу: <https://docs.ultralytics.com/integrations/albumentations> (дата зверн. 23.04.2026).
27. *Roboflow Inc.* Roboflow Documentation: Dataset Preparation, Annotation and Augmentation Tools [Електронний ресурс] / Roboflow Inc. — 2025. — Режим доступу до ресурсу: <https://docs.roboflow.com> (дата зверн. 20.04.2026).

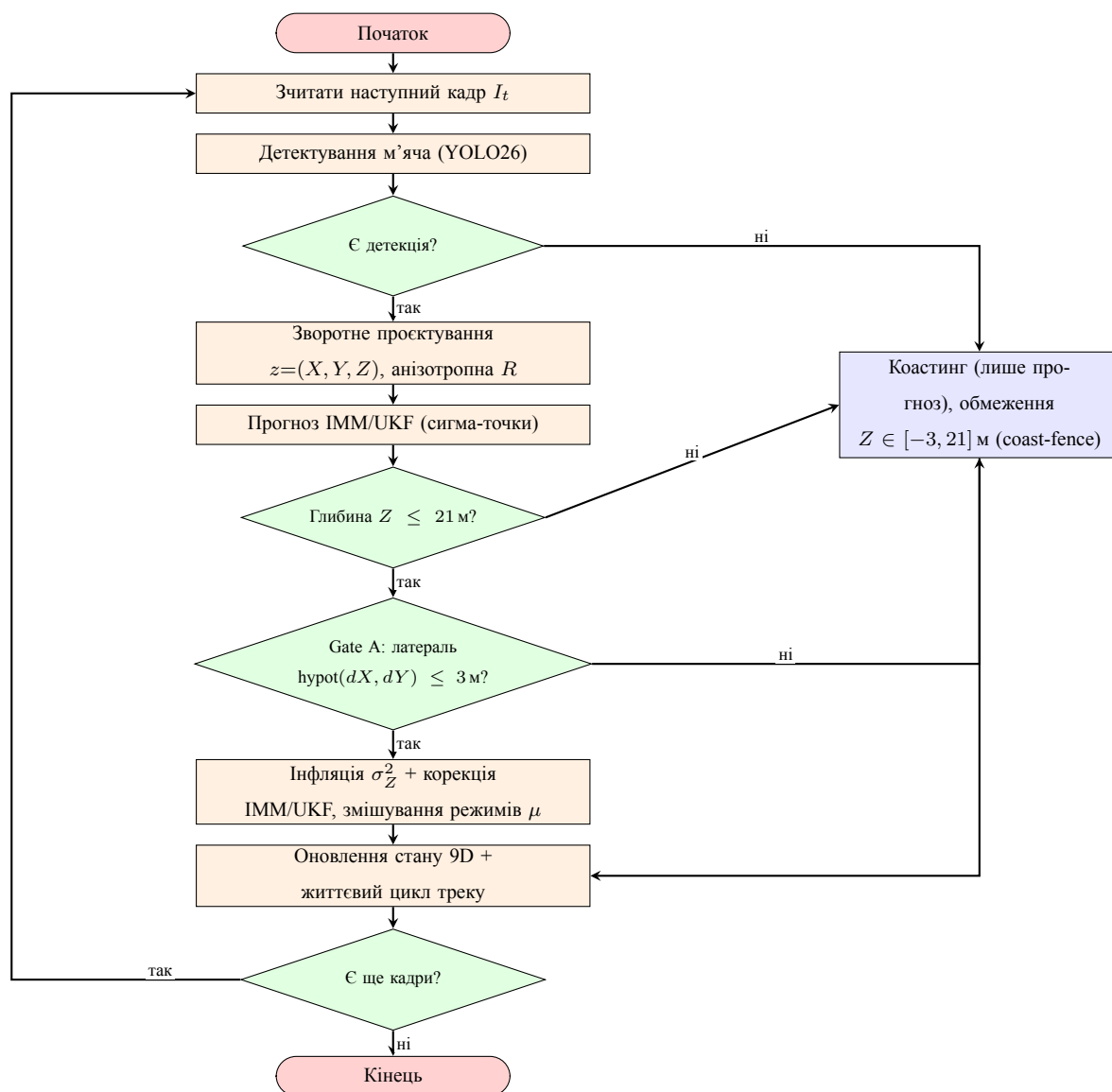


ІАЛЦ.467200.004 Д1

Зм.	Лист	№ докум.	Підп.	Дата	Система просторового аналізу гри в волейбол з використанням машинного навчання	Лит.	Маса	Масштаб
Розробив	Безщасний Р.Р.							
Перевірів	Морозов-Леонов							
Т. контр.						Аркушів 1		
Н. контр.	Міщенко Л. Д.					КПІ ім. Ігоря Сікорського, ФІОТ, ІО-21		
Затвердив	Волокита А. М.							



					ІАЛЦ.467200.005 Д2			
					Система просторового аналізу гри в волейбол з використанням машинного навчання	Лит.	Маса	Масштаб
Зм.	Лист	№ докум.	Підп.	Дата				
Розробив	Безщасний Р.Р.							
Перевірів	Морозов-Леонов							
Т. контр.						Аркушів 1		
Н. контр.	Міщенко Л. Д.				КПІ ім. Ігоря Сікорського, ФІОТ, ІО-21			
Затвердив	Волокита А. М.							



ІАЛЦ.467200.006 ДЗ

Зм.	Лист	№ докум.	Підп.	Дата	Система просторового аналізу гри в волейбол з використанням машинного навчання	Лит.	Маса	Масштаб
Розробив	Безщасний Р.Р.							
Перевірив	Морозов-Леонов							
Т. контр.						Аркушів 1		
Н. контр.	Міщенко Л. Д.					КПІ ім. Ігоря Сікорського, ФІОТ, ІО-21		
Затвердив	Волокита А. М.							

**СИСТЕМА ПРОСТОРОВОГО АНАЛІЗУ ГРИ В ВОЛЕЙБОЛ З
ВИКОРИСТАННЯМ МАШИННОГО НАВЧАННЯ**

Текст програми

ІА/ЛЦ.467200.007 Д4

ПЕРЕЛІК ФАЙЛІВ ПРОГРАМНОГО КОМПЛЕКСУ

У цьому додатку наведено повні вихідні тексти ключових модулів програмного комплексу системи трикутного просторового аналізу польоту волейбольного м'яча. Модулі впорядковано відповідно до архітектурного графа залежностей, описаного в розділі 3 пояснювальної записки: спочатку наводиться модуль геометричного зворотного проектування, далі — ядро байєсової фільтрації, потім модуль трекінгу зі скінченням автоматом життєвого циклу треку та, нарешті, сценарій навчання згорткової нейромережі-детектора. Сумарний обсяг наведеного коду становить понад 950 рядків мовою програмування Python.

ball_3d_analysis/get_court_position_methods.py

```
import cv2
import numpy as np
import torch as t
import torchvision
import torchaudio
import albumentations as A
from ultralytics import YOLO
```

Reference calibration correspondences: the four court corners.

Court is 9 m wide x 18 m long; world coordinates are in metres.

```
pts_real_3d = np.array([
    [0.0, 0.0, 0.0],
```

					ІАЛЦ.467200.007 Д4			
Зм.	Лист	№ докум.	Підп.	Дата				
Розробив	Безщасний Р.Р.				Система просторового аналізу гри в волейбол з використанням машинного навчання		Лит.	Аркуш
Перевірів	Морозов-Леонів							Аркушів
								1
Н. контр.	Міщенко Л. Д.						КПІ ім. Ігоря Сікорського, ФІОТ, 10-21	
Затвердив	Волокита А. М.							

```
[9.0, 0.0, 0.0],
[9.0, 0.0, 18.0],
[0.0, 0.0, 18.0]],dtype=np.float32)
```

Pixel locations of the same four corners in the broadcast frame.

```
pts_video_2d = np.array([
    [ 69, 1033], # P0 front-left
    [1840, 1031], # P1 front-right
    [1507, 609], # P2 back-right
    [ 405, 609]], # P3 back-left
    dtype=np.float32)
```

Initial pinhole intrinsics (principal point at the image centre). The focal
length is only a starting guess here — it is refined by calibration.

```
K = np.array([
    [1300.0, 0.0, 960.0],
    [0.0, 1300.0, 540.0],
    [0.0, 0.0, 1.0]],dtype=np.float32)
```

```
def calibrate_camera(pts_3d, pts_2d, camera_matrix, dist, up_axis=1):
```

```
    """
```

Calibrate the camera with PnP. Returns (R, tvec, camera_position):

R — world->camera rotation matrix (3x3),

tvec — translation into the camera frame (3,1),

camera_position — camera location in world coordinates (3,1).

up_axis flips R so the camera ends up above the floor: the four coplanar

court corners leave the vertical direction sign-ambiguous otherwise.

''''''

```
success, rvec, tvec = cv2.solvePnP(pts_3d, pts_2d, camera_matrix, dist,
flags=cv2.SOLVEPNP_ITERATIVE)
```

```
if not success:
```

```
    raise ValueError("solvePnP failed to converge; "
                    "check the order of the point correspondences")
```

```
R, _ = cv2.Rodrigues(rvec)
```

```
# Camera position in world coordinates (C = -R^T * tvec).
```

```
R_inv = np.linalg.inv(R)
```

```
camera_position = -np.dot(R_inv, tvec)
```

```
if (up_axis is not None
```

```
    and 0 <= up_axis < 3
```

```
    and camera_position[up_axis, 0] < 0.0):
```

```
    flip = np.eye(3)
```

```
    flip[up_axis, up_axis] = -1.0
```

```
    R = np.dot(R, flip)
```

```
    camera_position = np.dot(flip, camera_position)
```

```
return R, tvec, camera_position
```

```
def get_3d_position(u, v, bbox_w, K, R_matrix, camera_pos,
```

```
    ball_diameter=0.21, z_min=2.0, z_max=25.0,
```

```
    return_none_on_clip=False):
```

					ІА/Ц.467200.007 Д4	Аркуш
						3
Зм.	Лист	№ докум.	Підп.	Дата		

''''''

Back-project a ball detection into a 3D world position.

Args:

u, v: ball bbox centre, in pixels.

bbox_w: ball bbox width, in pixels. Width is used (not height)
because it suffers less from motion blur.

K: camera intrinsics matrix.

R_matrix: rotation matrix from solvePnP.

camera_pos: 3D camera position (C) from solvePnP.

ball_diameter: real volleyball diameter, in metres (21 cm nominal).

z_min, z_max: sanity bounds on Z_c (distance from camera along the ray).

Returns:

np.ndarray (3,) — world position [X, Y, Z]; or None when

return_none_on_clip=True and Z_c fell outside [z_min, z_max].

''''''

f_x = K[0, 0]

c_x = K[0, 2]

f_y = K[1, 1]

c_y = K[1, 2]

Z_c = (f_x * ball_diameter) / bbox_w

if Z_c < z_min or Z_c > z_max:

if return_none_on_clip:

return None

Z_c = max(z_min, min(z_max, Z_c))

```

x_norm = (u - c_x) / f_x
y_norm = (v - c_y) / f_y
d_cam = np.array([[x_norm], [y_norm], [1.0]])

```

```

d_world = np.dot(R_matrix.T, d_cam)
P_world = camera_pos.reshape(3, 1) + Z_c * d_world

return P_world.ravel()

```

```

def get_3d_position_with_cov(u, v, bbox_w, K, R_matrix, camera_pos,
    ball_diameter=0.21, sigma_w=1.0, sigma_uv=1.0, r_floor=0.02,
    z_min=2.0, z_max=25.0, return_none_on_clip=False):
    """Like get_3d_position, but also returns a 3x3 world-frame measurement
    covariance R, anisotropic and consistent with monocular depth geometry.

```

Depth error follows from $Z_c = f \cdot D / w_{\text{box}}$ linearised in the bbox width:

$$\sigma_{Z_c} = |dZ_c/dw| * \sigma_w = (f \cdot D / w_{\text{box}}^2) * \sigma_w,$$

so depth is trusted less for distant (small- w) balls. The uncertainty lies along the ray, giving a rank-1 ellipsoid plus lateral pixel terms:

$$\text{Cov} \sim \sigma_{Z_c}^2 * d \cdot d^T + Z_c^2 * \sigma_{uv}^2 * (J_u J_u^T + J_v J_v^T).$$

Args:

σ_w : bbox-width sigma, px (detector noise). Default 1.0.

σ_{uv} : bbox-centre sigma in u,v, px. Default 1.0.

r_{floor} : isotropic variance floor (m^2) on every axis. Default 0.02

(lateral term of the old fixed R), keeping lateral trust stable and

R positive-definite; only the along-ray depth stays adaptive.

Returns:

(P_world (3,), R_cov (3,3)); or (None, None) on clip when
return_none_on_clip=True.

''''''

f_x = K[0, 0]

c_x = K[0, 2]

f_y = K[1, 1]

c_y = K[1, 2]

Z_c = (f_x * ball_diameter) / bbox_w

if Z_c < z_min or Z_c > z_max:

if return_none_on_clip:

return None, None

Z_c = max(z_min, min(z_max, Z_c))

x_norm = (u - c_x) / f_x

y_norm = (v - c_y) / f_y

d_cam = np.array([[x_norm], [y_norm], [1.0]])

d_world = np.dot(R_matrix.T, d_cam) # (3,1) ray direction

P_world = camera_pos.reshape(3, 1) + Z_c * d_world

Depth sigma along the ray, from the bbox-width error.

sigma_Zc = (f_x * ball_diameter) / (bbox_w ** 2) * sigma_w

Lateral Jacobians (point shift per 1 px move of the bbox centre).

```

j_u = np.dot(R_matrix.T, np.array([[1.0 / f_x], [0.0], [0.0]]))
j_v = np.dot(R_matrix.T, np.array([[0.0], [1.0 / f_y], [0.0]]))

R_cov = (sigma_Zc ** 2) * np.dot(d_world, d_world.T)
R_cov += (Z_c ** 2) * (sigma_uv ** 2) * np.dot(j_u, j_u.T)
R_cov += (Z_c ** 2) * (sigma_uv ** 2) * np.dot(j_v, j_v.T)

# Isotropic floor: keeps lateral trust at the proven level of the old R
# (0.02) and guarantees positive-definiteness. Only the along-ray depth
# stays adaptive (sigma_Zc^2 on top of the floor).
R_cov += np.eye(3) * r_floor

return P_world.ravel(), R_cov.astype(np.float64)

```

ball_3d_analysis/IMM_UKF.py

```

import numpy as np
from copy import deepcopy
from scipy.linalg import cholesky, inv, expm, block_diag
from get_court_position_methods import calibrate_camera, get_3d_position
from scipy.stats import multivariate_normal
import sys

```

```

class UnscentedKalmanFilter(object):

```

```

    def __init__(self, name, dim_x, dim_z, dt, fx, hx, points, sqrt_fn=None,
                  x_mean_fn=None, z_mean_fn=None, residual_x=None, residual_z=None):

```

					IA/Ц.467200.007 Д4	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		7

```

self.name = name
self.x = np.zeros(dim_x)
self.P = np.eye(dim_x)
self.x_prior = np.copy(self.x)
self.P_prior = np.copy(self.P)
self.Q = np.eye(dim_x)
self.R = np.eye(dim_z)
self._dim_x = dim_x
self._dim_z = dim_z
self.points_fn = points
self._dt = dt
self._num_sigmas = points.num_sigmas()
self.hx = hx
self.fx = fx
self.x_mean_fn = x_mean_fn
self.z_mean_fn = z_mean_fn
self.Wm, self.Wc = points.Wm, points.Wc
self.likelihood = None
self._log_likelihood = np.log(sys.float_info.min)
self._mahalanobis = None

if sqrt_fn is None:
    self.msqrt = cholesky
else:
    self.msqrt = sqrt_fn

if residual_x is None:
    self.residual_x = np.subtract

```

```

else:
    self.residual_x = residual_x

if residual_z is None:
    self.residual_z = np.subtract
else:
    self.residual_z = residual_z

# sigma points transformed through f(x) and h(x)
self.sigmas_f = np.zeros((self._num_sigmas, self._dim_x))
self.sigmas_h = np.zeros((self._num_sigmas, self._dim_z))

self.K = np.zeros((dim_x, dim_z))
self.y = np.zeros(dim_z)
self.z = np.array([[None]*dim_z])
self.S = np.zeros((dim_z, dim_z))
self.SI = np.zeros((dim_z, dim_z))
self.inv = np.linalg.inv

self.x_prior = self.x.copy()
self.P_prior = self.P.copy()

self.x_post = self.x.copy()
self.P_post = self.P.copy()

def predict(self, dt=None, UT=None, fx=None, **fxargs):
    """
    :param dt: time step for next iteration of computation

```

```

:param UT: unscented transform function
:param fx: state transition function
:param fxargs: arguments passed to f(x)
"""

if dt is None:
    dt = self._dt

if UT is None:
    UT = unscented_transform

# calculating sigma points for given mean and covariance
self.compute_process_sigmas(dt, fx, **fxargs)
#pass sigmas through the unscented transform to compute the prior
self.x, self.P = UT(self.sigmas_f, self.Wm, self.Wc,
                    self.Q, self.x_mean_fn, self.residual_x)
self.P = (self.P + self.P.T) / 2.0
self.x_prior = np.copy(self.x)
self.P_prior = np.copy(self.P)

def update(self, z, R=None, UT=None, hx=None, **hxargs):
    """
    :param z: measurement vector
    :param R: measurement noise matrix
    :param UT: function for unscented transform
    :param hx: measurement function
    :param hxargs: arguments for hx
    """

    if z is None:

```



```

self.z = np.array([[None]*self._dim_z]).T
self.x_post = self.x.copy()
self.P_post = self.P.copy()
self.likelihood = 1.0
return

if hx is None:
    hx = self.hx

if UT is None:
    UT = unscented_transform

if R is None:
    R = self.R
elif np.isscalar(R):
    R = np.eye(self._dim_z) * R

# passing prior sigmas through h(x) to get measurement sigmas
# the shape of sigmas_h will vary if the shape of z varies
sigmas_h = []
for s in self.sigmas_f:
    sigmas_h.append(hx(s, **hxargs))

self.sigmas_h = np.atleast_2d(sigmas_h)

# mean and covariance of prediction passed through unscented transform
z_mean, self.S = UT(self.sigmas_h, self.Wm, self.Wc,
                    R, self.z_mean_fn, self.residual_z)

```

```

# Regularise S before inversion / likelihood: guarantees positive
# definiteness even when the covariance is momentarily degenerate.
S_stable = self.S + np.eye(self.S.shape[0]) * 1e-6
try:
    self.SI = np.linalg.inv(S_stable)
except np.linalg.LinAlgError:
    self.SI = np.linalg.pinv(S_stable)

# compute cross variance of the state and measurement
Pxz = self.cross_variance(self.x, z_mean, self.sigmas_f, self.sigmas_h)

self.K = np.dot(Pxz, self.SI) # Kalman gain
self.y = self.residual_z(z, z_mean)

try:
    self.likelihood = multivariate_normal.pdf(
        self.y,
        mean=np.zeros_like(self.y),
        cov=S_stable,
        allow_singular=True,
    )
except (np.linalg.LinAlgError, ValueError):
    self.likelihood = 1e-300

if not np.isfinite(self.likelihood) or self.likelihood <= 0.0:
    self.likelihood = 1e-300

# updating Gaussian state estimate(x, P)

```

```

self.x = self.x + np.dot(self.K, self.y)
self.P = self.P - np.dot(self.K, np.dot(self.S, self.K.T))

# save measurement and posterior state
self.z = deepcopy(z)
self.x_post = self.x.copy()
self.P_post = self.P.copy()

def cross_variance(self, x, z, sigmas_f, sigmas_h):
    # Computing cross variance of the state 'x' and measurement 'z'
    Pxz = np.zeros((sigmas_f.shape[1], sigmas_h.shape[1]))
    N = sigmas_f.shape[0]
    for i in range(N):
        dx = self.residual_x(sigmas_f[i], x)
        dz = self.residual_z(sigmas_h[i], z)
        Pxz += self.Wc[i] * np.outer(dx, dz)
    return Pxz

def compute_process_sigmas(self, dt, fx=None, **fx_args):
    # computes the values of sigmas_f
    if fx is None:
        fx = self.fx

    # calculate sigma points for given mean and covariance
    sigmas = self.points_fn.sigma_points(self.x, self.P)

    for i, s in enumerate(sigmas):
        self.sigmas_f[i] = fx(s, dt, **fx_args)

```

@property

def log_likelihood(self):

"""

log-likelihood of the last measurement.

"""

if self._log_likelihood is None:

self._log_likelihood = logpdf(x=self.y, cov=self.S)

return self._log_likelihood

@property

def mahalanobis(self):

"""

Mahalanobis distance of measurement. E.g. 3 means measurement was 3 standard deviations away from the predicted value.

"""

if self._mahalanobis is None:

self._mahalanobis = np.sqrt(float(np.dot(np.dot(self.y.T, self.SI), self.y)))

return self._mahalanobis

class IMMEstimator(object):

""" Implements an Interacting Multiple-Model (IMM) estimator.

:param filters: N-list consisting of filters in sequenced order

:param mu: (N) array-like of float with mode probabilities for each filter

:param M: (N, N) Markov chain transition matrix

"""

```

def __init__(self, filters, mu, M):
    if len(filters) < 2:
        raise ValueError('filters must contain at least two filters')

    self.filters = filters
    self.mu = np.asarray(mu) / np.sum(mu)
    self.M = M

    x_shape = filters[0].x.shape
    for f in filters:
        if x_shape != f.x.shape:
            raise ValueError(
                'All filters must have the same state dimension')

    self.x = np.zeros(filters[0].x.shape)
    self.P = np.zeros(filters[0].P.shape)
    self.N = len(filters)
    self.likelihood = np.zeros(self.N)
    self.omega = np.zeros((self.N, self.N))
    self._compute_mixing_probabilities()

    # initialize imm state estimate based on current filters
    self._compute_state_estimate()
    self.x_prior = self.x.copy()
    self.P_prior = self.P.copy()
    self.x_post = self.x.copy()
    self.P_post = self.P.copy()
    self._log_likelihood = np.log(sys.float_info.min)

```

```
self._mahalanobis = None
```

```
def filter_setx(self):
```

```
    for f in self.filters:
```

```
        f.x = self.x
```

```
def update(self, z, R=None):
```

```
    """
```

Add a new measurement (z) to the Kalman filter. If z is None, nothing is changed.

:param z: measurement

:param R: optional per-detection measurement covariance (3x3). If None, each filter uses its own fixed self.R. Passing a per-detection R (adaptive sigma_Z) makes depth trust depend on the bbox width.

```
    """
```

```
# run update on each filter, and save the likelihood
```

```
for i, f in enumerate(self.filters):
```

```
    f.update(z, R=R)
```

```
    self.likelihood[i] = f.likelihood
```

```
# update mode probabilities from total probability * likelihood
```

```
self.mu = self.cbar * self.likelihood
```

```
sum_mu = np.sum(self.mu)
```

```
if sum_mu == 0.0:
```

```
    self.mu = self.cbar.copy()
```

```
else:
```

```
    self.mu /= sum_mu
```

```

self._compute_mixing_probabilities()

# compute mixed IMM state and covariance and save posterior estimate
self._compute_state_estimate()
self.x_post = self.x.copy()
self.P_post = self.P.copy()

def predict(self, u=None):
    """
    Predict next state (prior) using the IMM state propagation
    equations.
    :param u: control vector, if None then none
    """

    # compute mixed initial conditions
    xs, Ps = [], []
    for i, (f, w) in enumerate(zip(self.filters, self.omega.T)):
        x = np.zeros(self.x.shape)
        for kf, wj in zip(self.filters, w):
            x += kf.x * wj
        xs.append(x)

        P = np.zeros(self.P.shape)
        for kf, wj in zip(self.filters, w):
            y = kf.x - x
            P += wj * (np.outer(y, y) + kf.P)
        Ps.append(P)

```

```

# compute each filter's prior using the mixed initial conditions
for i, f in enumerate(self.filters):
    # propagate using the mixed state estimate and covariance
    f.x = xs[i].copy()
    f.P = Ps[i].copy()
    f.predict(u)

# compute mixed IMM state and covariance and save posterior estimate
self._compute_state_estimate()
self.x_prior = self.x.copy()
self.P_prior = self.P.copy()

def _compute_state_estimate(self):
    """
    Computes the IMM's mixed state estimate from each filter using
    the the mode probability self.mu to weight the estimates.
    """
    self.x.fill(0)
    for f, mu in zip(self.filters, self.mu):
        self.x += f.x * mu

    self.P.fill(0)
    for f, mu in zip(self.filters, self.mu):
        y = f.x - self.x
        self.P += mu * (np.outer(y, y) + f.P)

def _compute_mixing_probabilities(self):

```



```
# Compute the mixing probability for each filter
```

```
self.cbar = np.dot(self.mu, self.M)
```

```
for i in range(self.N):
```

```
    for j in range(self.N):
```

```
        self.omega[i, j] = (self.M[i, j]*self.mu[i]) / self.cbar[j]
```

```
@property
```

```
def log_likelihood(self):
```

```
    """
```

```
    log-likelihood of the last measurement.
```

```
    """
```

```
    if self._log_likelihood is None:
```

```
        self._log_likelihood = logpdf(x=self.y, cov=self.S)
```

```
    return self._log_likelihood
```

```
@property
```

```
def mahalanobis(self):
```

```
    """
```

```
    Mahalanobis distance of measurement. E.g. 3 means measurement  
    was 3 standard deviations away from the predicted value.
```

```
    """
```

```
    if self._mahalanobis is None:
```

```
        self._mahalanobis = np.sqrt(float(np.dot(np.dot(self.y.T, self.SI), self.y)))
```

```
    return self._mahalanobis
```

class MerweScaledSigmaPoints(object):

''''''

Generates sigma points and weights according to Van der Merwe's 2004 dissertation[1] for the UnscentedKalmanFilter class. It parametrizes the sigma points using alpha, beta, kappa terms, and is the version seen in most publications.

:param n : int

Dimensionality of the state. $2n+1$ weights will be generated.

:param alpha : float

Determines the spread of the sigma points around the mean.

Usually a small positive value ($1e-3$).

:param beta : float

Incorporates prior knowledge of the distribution of the mean. For Gaussian α $\beta=2$ is optimal.

:param kappa : float, default=0.0

Secondary scaling parameter usually set to 0 or to $3-n$.

:param sqrt_method : function(ndarray), default=scipy.linalg.cholesky

Defines how we compute the square root of a matrix, which has no unique answer. Cholesky is the default choice due to its speed(another good choice - `scipy.linalg.sqrtm`).

:param subtract : callable (x, y), optional

					IA/Ц.467200.007 Д4	Архив
						20
Зм.	Лист	№ докум.	Підп.	Дата		

Function that computes the difference between x and y.

You will have to supply this if your state variable cannot support subtraction, such as angles (359-1 degrees is 2, not 358). x and y are state vectors, not scalars.

Attributes:

Wm : np.array

weight for each sigma point for the mean

Wc : np.array

weight for each sigma point for the covariance

~~~~~

```
def __init__(self, n, alpha, beta, kappa, sqrt_method=None, subtract=None):
```

```
#pylint: disable=too-many-arguments
```

```
self.n = n
```

```
self.alpha = alpha
```

```
self.beta = beta
```

```
self.kappa = kappa
```

```
if sqrt_method is None:
```

```
    self.sqrt = cholesky
```

```
else:
```

```
    self.sqrt = sqrt_method
```

```
if subtract is None:
```

```
    self.subtract = np.subtract
```

```
else:
```

```
self.subtract = subtract
```

```
self._compute_weights()
```

```
def num_sigmas(self):
```

```
    """ Number of sigma points for each variable in the state x """
```

```
    return 2*self.n + 1
```

```
def sigma_points(self, x, P):
```

```
    """ Computes the sigma points for an unscented Kalman filter  
    given the mean (x) and covariance(P) of the filter.
```

```
    Returns tuple of the sigma points and weights.
```

Works with both scalar and array inputs:

```
sigma_points (5, 9, 2) # mean 5, covariance 9
```

```
sigma_points ([5, 2], 9*eye(2), 2) # means 5 and 2, covariance 9I
```

```
:param x : An array-like object of the means of length n
```

```
:param P : scalar, or np.array
```

```
    Covariance of the filter. If scalar, is treated as eye(n)*P.
```

```
:return np.array, of size (n, 2n+1)
```

Two dimensional array of sigma points. Each column contains all of the sigmas for one dimension in the problem space.

```

Ordered by  $X_{i_0}$ ,  $X_{i_{\{1..n\}}}$ ,  $X_{i_{\{n+1..2n\}}}$ 
''''''

if self.n != np.size(x):
    raise ValueError("expected size(x) {}, but size is {}".format(
        self.n, np.size(x)))

n = self.n

if np.isscalar(x):
    x = np.asarray([x])

if np.isscalar(P):
    P = np.eye(n)*P
else:
    P = np.atleast_2d(P)

lambda_ = self.alpha**2 * (n + self.kappa) - n
U = self.sqrt((lambda_ + n)*P)

sigmas = np.zeros((2*n+1, n))
sigmas[0] = x
for k in range(n):
    # pylint: disable=bad-whitespace
    sigmas[k+1] = self.subtract(x, -U[k])
    sigmas[n+k+1] = self.subtract(x, U[k])

return sigmas

```

```
def _compute_weights(self):
    """ Computes the weights for the scaled unscented Kalman filter. """

    n = self.n
    lambda_ = self.alpha**2 * (n + self.kappa) - n

    c = .5 / (n + lambda_)
    self.Wc = np.full(2*n + 1, c)
    self.Wm = np.full(2*n + 1, c)
    self.Wc[0] = lambda_ / (n + lambda_) + (1 - self.alpha**2 + self.beta)
    self.Wm[0] = lambda_ / (n + lambda_)
```

```
def __repr__(self):
    return (f'MerweScaledSigmaPoints(n={self.n}, alpha={self.alpha}, '
            f'beta={self.beta}, kappa={self.kappa})')
```

```
def unscented_transform(sigmas, Wm, Wc, noise_cov=None,
                        mean_fn=None, residual_fn=None):
```

```
    """
```

Computes unscented transform of a set of sigma points and weights.

returns the mean and covariance in a tuple.

:param residual\_fn:

:param sigmas: ndarray, of size (n, 2n+1)

2D array of sigma points.

:param Wm : ndarray [# sigmas per dimension]

Weights for the mean.

:param Wc : ndarray [# sigmas per dimension]

Weights for the covariance.

:param noise\_cov: ndarray, optional

noise matrix added to the final computed covariance matrix.

:param mean\_fn: callable (sigma\_points, weights), optional

Function that computes the mean of the provided sigma points  
and weights

''''''

kmax, n = sigmas.shape

try:

if mean\_fn is None:

# new mean is just the sum of the sigmas \* weight

x = np.dot(Wm, sigmas) # dot =  $\sum (W[k] * X_i[k])$

else:

x = mean\_fn(sigmas, Wm)

except:

print(sigmas)

raise

```
# new covariance is the sum of the outer product of the residuals
# times the weights
```

```
# this is the fast way to do this - see 'else' for the slow way
```

```
if residual_fn is np.subtract or residual_fn is None:
```

```
    y = sigmas - x[np.newaxis, :]
```

```
    P = np.dot(y.T, np.dot(np.diag(Wc), y))
```

```
else:
```

```
    P = np.zeros((n, n))
```

```
    for k in range(kmax):
```

```
        y = residual_fn(sigmas[k], x)
```

```
        P += Wc[k] * np.outer(y, y)
```

```
if noise_cov is not None:
```

```
    P += noise_cov
```

```
return x, P
```

```
def Q_discrete_white_noise(dim, dt=1., var=1., block_size=1, order_by_dim=True):
```

```
    """
```

Returns the Q matrix for the Discrete Constant White Noise

Model. dim may be either 2, 3, or 4 dt is the time step, and sigma

is the variance in the noise.

Q is computed as the  $G * G^T * \text{variance}$ , where G is the process noise per time step. In other words,  $G = [[.5dt^2][dt]]^T$  for the constant velocity



model.

:param dim : int (2, 3, or 4)

dimension for Q, where the final dimension is (dim x dim)

:param dt : float, default=1.0

time step in whatever units your filter is using for time. i.e. the amount of time between innovations

:param var : float, default=1.0

variance in the noise

:param block\_size : int >= 1

If your state variable contains more than one dimension, such as a 3d constant velocity model  $[x \ x' \ y \ y' \ z \ z']^T$ , then Q must be a block diagonal matrix.

:param order\_by\_dim : bool, default=True

Defines ordering of variables in the state vector. 'True' orders by keeping all derivatives of each dimensions)

$[x \ x' \ x'' \ y \ y' \ y'']$

whereas 'False' interleaves the dimensions

$[x \ y \ z \ x' \ y' \ z' \ x'' \ y'' \ z'']$

''''''

if not (dim == 2 or dim == 3 or dim == 4):

raise ValueError("dim must be between 2 and 4")

```

if dim == 2:
    Q = [[.25*dt**4, .5*dt**3],
          [.5*dt**3, dt**2]]
elif dim == 3:
    Q = [[.25*dt**4, .5*dt**3, .5*dt**2],
          [.5*dt**3, dt**2, dt],
          [.5*dt**2, dt, 1]]
else:
    Q = [(dt**6)/36, (dt**5)/12, (dt**4)/6, (dt**3)/6],
          [(dt**5)/12, (dt**4)/4, (dt**3)/2, (dt**2)/2],
          [(dt**4)/6, (dt**3)/2, dt**2, dt],
          [(dt**3)/6, (dt**2)/2, dt, 1.]]

if order_by_dim:
    return block_diag(*[Q]*block_size) * var
return order_by_derivative(np.array(Q), dim, block_size) * var

def logpdf(x, mean=None, cov=1, allow_singular=True):
    """
    Computes the log of the probability density function of the normal
    N(mean, cov) for the data x. The normal may be univariate or multivariate.
    """

    if mean is not None:
        flat_mean = np.asarray(mean).flatten()
    else:
        flat_mean = None

```

```
flat_x = np.asarray(x).flatten()
```

```
return multivariate_normal.logpdf(flat_x, flat_mean,
                                   cov, allow_singular=allow_singular)
```

```
def fx_ballistic(x, dt):
```

```
    """
```

9D ballistic model (CV-CA-IMM): state = [x, vx, ax, y, vy, ay, z, vz, az].

Constant-acceleration discretisation:

```
    p_new = p + v*dt + 0.5*a_total*dt^2
```

```
    v_new = v + a_total*dt
```

```
    a_new = a (deterministic — it only changes through the Q[a] noise).
```

```
    """
```

```
    vx = np.clip(x[1], -40.0, 40.0)
```

```
    vy = np.clip(x[4], -40.0, 40.0)
```

```
    vz = np.clip(x[7], -40.0, 40.0)
```

```
    ax_s, ay_s, az_s = x[2], x[5], x[8]
```

```
    k = 0.0314
```

```
    v_mag = np.sqrt(vx**2 + vy**2 + vz**2)
```

```
    a_drag_x = -k * v_mag * vx
```

```
    a_drag_y = -k * v_mag * vy
```

```
    a_drag_z = -k * v_mag * vz
```

```
    g = 9.81
```

```
    a_tot_x = ax_s + a_drag_x
```

```
a_tot_y = ay_s + a_drag_y - g
```

```
a_tot_z = az_s + a_drag_z
```

```
return np.array([
    x[0] + vx*dt + 0.5*a_tot_x*dt**2,
    vx + a_tot_x*dt,
    ax_s,
    x[3] + vy*dt + 0.5*a_tot_y*dt**2,
    vy + a_tot_y*dt,
    ay_s,
    x[6] + vz*dt + 0.5*a_tot_z*dt**2,
    vz + a_tot_z*dt,
    az_s,
])
```

```
def fx_hit(x, dt):
```

```
    """9D hit model: identical dynamics to fx_ballistic. The hit mode differs
    only in a much larger Q[a] (set in create_imm_estimator), which lets the
    acceleration wake from ~0 to impulsive values within a frame or two."""
    return fx_ballistic(x, dt)
```

```
def fx_bounce(x, dt):
```

```
    """
```

```
    9D floor-bounce model: vy is inverted with epsilon=0.75, vx/vz are damped
    with friction=0.85.
```

```
    """
```

|     |      |          |       |      |                    |       |
|-----|------|----------|-------|------|--------------------|-------|
|     |      |          |       |      | ІА/Ц.467200.007 Д4 | Аркуш |
|     |      |          |       |      |                    | 30    |
| Зм. | Лист | № докум. | Підп. | Дата |                    |       |

```

epsilon = 0.75
friction = 0.85
x_next = np.copy(x)
vx_new = x[1] * friction
vy_new = -x[4] * epsilon
vz_new = x[7] * friction

x_next[0] += vx_new * dt
x_next[3] += vy_new * dt
x_next[6] += vz_new * dt

x_next[1] = vx_new
x_next[4] = vy_new
x_next[7] = vz_new

# Reset residual acceleration — a bounce is a "fresh start".
x_next[2] = 0.0
x_next[5] = 0.0
x_next[8] = 0.0

return x_next

```

```
def hx(x):
```

```
    """
```

Measurement function 9D -> 3D: extract positions only (indices 0, 3, 6).  
 Velocities (1, 4, 7) and accelerations (2, 5, 8) are not measured, only  
 estimated by the UKF/IMM.

''''''

```
return np.array([x[0], x[3], x[6]])
```

```
def measurement_transform(prediction_data, K, R_matrix, camera_pos,  
                           ball_diameter=0.21):
```

''''''

Turn a detection record (px, py, bbox width) into a 3D world measurement  
[X, Y, Z] in court coordinates, with the convention:

- X — across the court (along the end line),
- Y — vertical axis (up, perpendicular to the floor),
- Z — along the long side (along the side line).

''''''

```
u_cam, v_cam, w_cam = (prediction_data['x_pos'], prediction_data['y_pos'],  
                        prediction_data['w_box'])
```

```
raw_x, raw_y, raw_z = get_3d_position(u_cam, v_cam, w_cam, K,  
                                       R_matrix, camera_pos, ball_diameter)
```

```
measurement = np.array([raw_x, raw_y, raw_z], dtype=np.float32)
```

```
return measurement
```

```
def get_dynamic_transition_matrix(y, vy,  
                                  mahalanobis_sq=0.0,  
                                  hit_residual_min_sq=8.0,  
                                  hit_residual_max_sq=11.34,  
                                  bounce_height_max=0.55,  
                                  frames_since_update=0,  
                                  bounce_max_coast=3,  
                                  m_hit_target=None):
```

"""Return the 3x3 IMM Markov transition matrix for [ballistic, hit, bounce] for the next prediction step. The base matrix sticks to ballistic ( $M[0,0]=0.95$ ); two triggers steer it: a passive geometric bounce trigger and a residual-based mid-air hit trigger interpolated over [hit\_residual\_min\_sq, hit\_residual\_max\_sq] (the  $\chi^2_3$  95th percentile to the gating bound), clamped outside that range.

y, vy: ball vertical position (m) and velocity (m/s) in world frame.

mahalanobis\_sq: squared Mahalanobis distance of the last accepted detection (0 for tracks without updates).

"""

```
M = np.array([[0.95, 0.04, 0.01],
               [0.60, 0.40, 0.00],
               [0.90, 0.00, 0.10]])
```

```
if frames_since_update > bounce_max_coast:
```

```
    return np.eye(3)
```

```
if (y < bounce_height_max and vy < 0 and frames_since_update
    <= bounce_max_coast):
```

```
    M[0] = [0.10, 0.05, 0.85]
```

```
    M[1] = [0.10, 0.05, 0.85]
```

```
    return M
```

```
if mahalanobis_sq > hit_residual_min_sq:
```

```
    alpha = min(
```

```
        1.0,
```

```
        (mahalanobis_sq - hit_residual_min_sq)
```

```

        / max(hit_residual_max_sq - hit_residual_min_sq, 1e-6),
    )
    M_hit_target = (np.array([0.45, 0.50, 0.05])
                    if m_hit_target is None
                    else np.asarray(m_hit_target, dtype=float))
    M[0] = (1.0 - alpha) * M[0] + alpha * M_hit_target

    return M

```

### **ball\_3d\_analysis/IMMTracker.py**

```

import numpy as np
from scipy.optimize import linear_sum_assignment
from IMM_UKF import *
import cv2

```

```

def create_imm_estimator(z_initial, dt=0.02, v_initial=None):
    """Build an IMM estimator from three UKFs (ballistic / hit / bounce).

    z_initial: world position [x, y, z] at init.
    dt:        time step (= 1 / FPS).
    v_initial: optional initial velocity [vx, vy, vz] in m/s. None -> zeros.

    Passing (z_k - z_{k-1}) / dt speeds up UKF convergence and
    avoids residual/Mahalanobis spikes in a new track's first frames.
    """
    # 9D constant-acceleration state: [x, vx, ax, y, vy, ay, z, vz, az].
    dim_x = 9

```



```

dim_z = 3
points = MerweScaledSigmaPoints(n=dim_x, alpha=.1, beta=2., kappa=1.)
# Initial covariance. Accel variance ~25 (m/s^2)^2 lets the filter absorb
# the first non-zero acceleration without immediately blowing the gate;
# velocity variance 50 covers the finite-difference bootstrap spread.
P_init = np.diag([
    0.1, 50.0, 25.0,
    0.1, 50.0, 25.0,
    0.1, 50.0, 25.0,
])
# Measurement noise (m^2). sigma_z ~ 0.7 m reflects monocular depth error
# (Z_c = f*D / bbox_w); tighter values made trajectories too jagged.
R_init = np.diag([0.02, 0.02, 0.5])

# Process noise per model. In Q_discrete_white_noise(dim=3) var is the
# jerk variance (bottom-right block cell), i.e. how much accel may drift.
# ballistic: small jerk noise (free flight is smooth, residual accel ~0).
q_var_ballistic = 0.1
q_b = Q_discrete_white_noise(dim=3, dt=dt, var=q_var_ballistic)
Q_ballistic = block_diag(q_b, q_b, q_b)

# hit: large jerk noise lets accel jump from ~0 to tens of m/s^2 in a frame
# or two. The impulse is modelled through accel growth (Singer-style CA),
# not a raw velocity jump that broke gating in the old 6D version.
q_var_hit = 400
q_h = Q_discrete_white_noise(dim=3, dt=dt, var=q_var_hit)
Q_hit = block_diag(q_h, q_h, q_h)

```

```

# bounce: moderate jerk noise; fx_bounce already resets accel to 0 on
# contact, Q[a] only guards against abrupt deviations in later frames.
q_var_bounce = 5
q_bnc = Q_discrete_white_noise(dim=3, dt=dt, var=q_var_bounce)
Q_bounce = block_diag(q_bnc, q_bnc, q_bnc)

# Ballistic filter
ukf_ballistic = UnscentedKalmanFilter(name='ballistic UKF', dim_x=dim_x,
                                       dim_z=dim_z, dt=dt, fx=fx_ballistic, hx=hx, points=points)
ukf_ballistic.P = P_init
ukf_ballistic.Q = Q_ballistic
ukf_ballistic.R = R_init

# Hit filter
ukf_hit = UnscentedKalmanFilter(name='hit UKF', dim_x=dim_x, dim_z=dim_z,
                                 dt=dt, fx=fx_hit, hx=hx, points=points)
ukf_hit.P = P_init
ukf_hit.Q = Q_hit
ukf_hit.R = R_init

# Bounce filter
ukf_bounce = UnscentedKalmanFilter(name='bounce UKF', dim_x=dim_x,
                                   dim_z=dim_z, dt=dt, fx=fx_bounce, hx=hx, points=points)
ukf_bounce.P = P_init
ukf_bounce.Q = Q_bounce
ukf_bounce.R = R_init

filters_lt = [ukf_ballistic, ukf_hit, ukf_bounce]

```

```

mu = np.array([0.95, 0.04, 0.01])
M_base = np.array([[0.95, 0.04, 0.01],
                    [0.60, 0.40, 0.00],
                    [0.90, 0.00, 0.10]])

imm = IMMEstimator(filters_lt, mu, M_base)

# Initialise the 9D state. Use v_initial if given, else zero velocity.
# Accel always starts at 0 (ballistic prior); the high P_init[a] lets the
# filter discover a non-zero acceleration when needed.
if v_initial is None:
    vx0 = vy0 = vz0 = 0.0
else:
    vx0, vy0, vz0 = (float(v_initial[0]), float(v_initial[1]),
                     float(v_initial[2]))
initial_x = np.array([
    z_initial[0], vx0, 0.0,
    z_initial[1], vy0, 0.0,
    z_initial[2], vz0, 0.0,
])
for f in imm.filters:
    f.x = initial_x.copy()
    # IMMEstimator copies f.x_post into imm.x_post once, in its constructor,
    # before we overwrite f.x. Without re-setting f.x_post here, the birth
    # snapshot logs zeros -> overlay draws a (0,0,0) world point. x_post is
    # only a logging snapshot, so this does not affect predict/update.
    f.x_post = initial_x.copy()
imm._compute_state_estimate()

```

# Same for the IMM aggregate read by snapshot\_track in run\_segment.

imm.x\_post = initial\_x.copy()

return imm

class Track:

# Sanity cap on bootstrap velocity (m/s): above this the finite-difference

# almost certainly linked two different objects, so bootstrap is skipped.

bootstrap\_max\_speed = 35.0

# Track-level covariance reset on a radical residual. A high Mahalanobis<sup>2</sup>

# (~5% tail of  $\chi^2_3$ ) means an impulsive manoeuvre (hit/block); we

# "open up" velocity/accel covariance before imm.update(z) so the new

# measurement pulls the state onto the new trajectory without multi-frame lag.

cov\_reset\_mah\_sq\_threshold = 8.0 # ~95th percentile of  $\chi^2_3$

cov\_reset\_inflate\_v = 50.0 # (m/s)<sup>2</sup> — same level as P\_init[v]

cov\_reset\_inflate\_a = 100.0 # (m/s<sup>2</sup>)<sup>2</sup> — 4x P\_init[a]

# Delayed-accept hysteresis. A suspicious detection (mah\_sq > threshold) is

# frozen in \_pending and waits for confirmation from later frames: if the

# next detection is also far, the manoeuvre is real; if it returns to

# normal, the pending was a false positive and is dropped.

hysteresis\_mah\_sq\_threshold = 8.0 # suspicion threshold (= cov\_reset)

hysteresis\_confirm\_threshold = 5.0 # next mah\_sq below this -> pending was FP

hysteresis\_window = 6 # max frames to wait for confirmation

coast\_threshold = 4 # consecutive misses = coasting: high

# mah\_sq is expected (P grew naturally),

# not a manoeuvre -> no inflate, no defer

```

# Coast covariance cap. While coasting P grows unbounded; large P_vel
# spreads the UKF sigma points so wide that the quadratic drag (Jensen)
# pulls the sigma-mean velocity toward zero and overrides gravity -> vy
# freezes and the track "hangs" instead of following a ballistic arc. The
# cap keeps sigma points tight so gravity dominates again. pos cap = 9 m^2
# (+-3 m) matches Gate A so the Mahalanobis gate stays meaningful.
coast_p_pos_cap = 9.0 # (m^2) — +-3 m, = Gate A max_assoc_residual
coast_p_vel_cap = 50.0 # (m/s)^2 — = P_init[v]
coast_p_acc_cap = 100.0 # (m/s^2)^2 — = cov_reset_inflate_a

def __init__(self, track_id, z_initial, dt, v_initial=None,
              initial_hits=1, min_hits=3, enable_hysteresis=False,
              enable_cov_reset=False, enable_coast_cov_cap=False,
              enable_coast_z_fence=False, coast_z_min=-3.0, coast_z_max=21.0,
              hit_residual_min_sq=8.0, hit_residual_max_sq=11.34,
              bounce_height_max=0.55, bounce_max_coast=3,
              m_hit_target=None):
    """
    enable_cov_reset: inflate P[v,a] when mah_sq > threshold. Default False
                        (lets the residual live 1-2 frames so the hit trigger
                        fires and IMM likelihood raises mu_hit; cov-reset and
                        hit are substitutes for the same residual signal).
    enable_hysteresis: delayed-accept defer of suspicious detections. Default
                        False (defer cannibalises cov-reset and the hit trigger
                        — all three share mah_sq=8 — and zeroes the residual
                        the next predict needs). Coast-skip via _missed_streak
                        stays active regardless: that handles occlusion, not defer.

```

v\_initial: optional initial velocity vector; lets the UKFs start with a real velocity instead of zeros.

initial\_hits: hits the track starts with. Bootstrapped tracks use 2 (two consecutive detections were effectively seen), promoting to Confirmed one frame sooner.

min\_hits: updates needed to reach 'Confirmed'. Forwarded from IMMTracker (CLI) so the swept parameter affects the track state machine.

~~~~~

```
self.track_id = track_id
```

```
self.dt = float(dt)
```

```
self.enable_hysteresis = bool(enable_hysteresis)
```

```
self.enable_cov_reset = bool(enable_cov_reset)
```

```
self.enable_coast_cov_cap = bool(enable_coast_cov_cap)
```

```
# Coast Z-fence: while coasting there is no measurement, so no gate acts  
# and predict() extrapolates world depth Z (idx 6) from a contaminated  
# depth velocity vz (idx 7). Near the frame edge w_box collapses, Z=f*D/w  
# inflates, and coasting carries the ball to Z=33-42 m (court is 18 m).  
# The fence clamps predicted Z to [coast_z_min, coast_z_max] and damps  
# vz/az that push past it. Active only during coasting (mark_missed).
```

```
self.enable_coast_z_fence = bool(enable_coast_z_fence)
```

```
self.coast_z_min = float(coast_z_min)
```

```
self.coast_z_max = float(coast_z_max)
```

```
# IMM triggers, forwarded to get_dynamic_transition_matrix on each  
# predict(). Defaults match the function signature (A/B compatibility).
```

```
self.hit_residual_min_sq = float(hit_residual_min_sq)
```

```
self.hit_residual_max_sq = float(hit_residual_max_sq)
```

```
self.bounce_height_max = float(bounce_height_max)
```

```

self.bounce_max_coast = int(bounce_max_coast)
self.m_hit_target = (None if m_hit_target is None
                      else np.asarray(m_hit_target, dtype=float))
self.imm = create_imm_estimator(z_initial, dt, v_initial=v_initial)

# Remember the first detection for intra-track velocity bootstrap on the
# second hit. Skip it if v_initial was already supplied externally.
self.z_initial = np.asarray(z_initial, dtype=float).copy()
self._velocity_bootstrapped = (v_initial is not None)

# Lifecycle
self.state = 'Tentative' # 'Tentative', 'Confirmed', 'Deleted'
self.time_since_update = 0
self.hits = int(initial_hits)
self.hit_streak = int(initial_hits)
self.min_hits = int(min_hits)
# Last accepted Mahalanobis^2 (gating residual), passed to
# get_dynamic_transition_matrix to activate the hit trigger on the next
# predict(). Reset to 0 after frames without updates.
self._last_mahalanobis_sq = 0.0
# Hysteresis state
self._pending = None # Optional[(z, mah_sq)]
self._pending_frames_elapsed = 0 # frames since pending was set
self._missed_streak = 0 # consecutive mark_missed (coast detect)
# Position-extraction matrix: indices 0, 3, 6 of the 9D state. Velocities
# and accelerations are not measured, only estimated by the UKF.
self.h_matrix = np.array([
    [1, 0, 0, 0, 0, 0, 0, 0, 0], # x

```

```

[0, 0, 0, 1, 0, 0, 0, 0, 0], # y
[0, 0, 0, 0, 0, 0, 0, 1, 0, 0], # z
], dtype=float)

```

```
def _inflate_covariance(self):
```

```

"""Inflate P[v,a] for every sub-filter and the IMM aggregate."""

```

```

for f in self.imm.filters:

```

```

    f.P[1, 1] += self.cov_reset_inflate_v

```

```

    f.P[4, 4] += self.cov_reset_inflate_v

```

```

    f.P[7, 7] += self.cov_reset_inflate_v

```

```

    f.P[2, 2] += self.cov_reset_inflate_a

```

```

    f.P[5, 5] += self.cov_reset_inflate_a

```

```

    f.P[8, 8] += self.cov_reset_inflate_a

```

```

self.imm.P[1, 1] += self.cov_reset_inflate_v

```

```

self.imm.P[4, 4] += self.cov_reset_inflate_v

```

```

self.imm.P[7, 7] += self.cov_reset_inflate_v

```

```

self.imm.P[2, 2] += self.cov_reset_inflate_a

```

```

self.imm.P[5, 5] += self.cov_reset_inflate_a

```

```

self.imm.P[8, 8] += self.cov_reset_inflate_a

```

```
def _collapse_accel_to_ballistic(self):
```

```

"""Zero the residual acceleration (indices 2, 5, 8) in all states.

```

Called while coasting (mark_missed). Without a measurement, predict() keeps integrating stale accel into velocity ($v_x \leftarrow v_x + a_x \cdot dt$) and the track accelerates across the court. Physically the ball between contacts has zero residual accel (only g + drag, already driving terms in fx_*), so collapsing to ballistic keeps coasting at constant velocity. Velocity,

position and P are left untouched.

"""

```
for f in self.imm.filters:
```

```
    f.x[2] = f.x[5] = f.x[8] = 0.0
```

```
    f.x_post[2] = f.x_post[5] = f.x_post[8] = 0.0
```

```
self.imm.x[2] = self.imm.x[5] = self.imm.x[8] = 0.0
```

```
self.imm.x_post[2] = self.imm.x_post[5] = self.imm.x_post[8] = 0.0
```

@staticmethod

```
def _cap_P_matrix(P, caps):
```

```
    """PD-safe cap of the P diagonal to caps[i]. For each i with
    P[i,i] > caps[i] scale row and column i by s = sqrt(cap/P[i,i]). This
    congruence transform D*P*D preserves positive-definiteness (unlike a
    naive P[i,i]=cap, which can break PD and fail the UKF Cholesky); the
    off-diagonals scale proportionally, keeping correlations valid."""
```

```
    for i in range(P.shape[0]):
```

```
        if P[i, i] > caps[i]:
```

```
            s = np.sqrt(caps[i] / P[i, i])
```

```
            P[i, :] *= s
```

```
            P[:, i] *= s
```

```
def _cap_coast_covariance(self):
```

```
    """Cap P-diagonal growth while coasting (see coast_p_*_cap). Without it
    P grows unbounded, the UKF sigma points spread, the quadratic drag damps
    gravity in the sigma-mean, vy freezes and the track hangs instead of
    following a ballistic arc. Complements Gate A."""
```

```
    caps = np.array([
```

```
        self.coast_p_pos_cap, self.coast_p_vel_cap, self.coast_p_acc_cap,
```

```

        self.coast_p_pos_cap, self.coast_p_vel_cap, self.coast_p_acc_cap,
        self.coast_p_pos_cap, self.coast_p_vel_cap, self.coast_p_acc_cap,
    ])

```

```

for f in self.imm.filters:

```

```

    self._cap_P_matrix(f.P, caps)

```

```

self._cap_P_matrix(self.imm.P, caps)

```

```

def _fence_coast_depth(self):

```

```

    """Clamp world depth Z (idx 6) while coasting to [coast_z_min,
    coast_z_max] and damp the depth velocity/accel (vz=7, az=8) that push
    past the fence. Applied to all UKF filters (their x and x_prior seed the
    next sigma points) and to the mixed imm state (reported in overlay).

```

```

    P is left to Gate C (coast_cov_cap)."""

```

```

    zlo, zhi = self.coast_z_min, self.coast_z_max

```

```

def _clamp(x):

```

```

    if x[6] > zhi:

```

```

        x[6] = zhi

```

```

        if x[7] > 0.0:

```

```

            x[7] = 0.0

```

```

            x[8] = 0.0

```

```

        elif x[6] < zlo:

```

```

            x[6] = zlo

```

```

            if x[7] < 0.0:

```

```

                x[7] = 0.0

```

```

                x[8] = 0.0

```

```

for f in self.imm.filters:

```

```

    _clamp(f.x)
    _clamp(f.x_prior)
    _clamp(self.imm.x)
    _clamp(self.imm.x_prior)

```

```
def predict(self):
```

```
    # 9D state: y = x[3], vy = x[4].
```

```

    self.imm.M = get_dynamic_transition_matrix(
        self.imm.x[3], self.imm.x[4],
        mahalanobis_sq=self._last_mahalanobis_sq,
        frames_since_update=self.time_since_update,
        hit_residual_min_sq=self.hit_residual_min_sq,
        hit_residual_max_sq=self.hit_residual_max_sq,
        bounce_height_max=self.bounce_height_max,
        bounce_max_coast=self.bounce_max_coast,
        m_hit_target=self.m_hit_target,
    )

```

```
    self.imm.predict()
```

```
    if self._pending is not None:
```

```
        self._pending_frames_elapsed += 1
```

```
        if self._pending_frames_elapsed > self.hysteresis_window:
```

```
            # Confirmation window exhausted: drop pending, resume ageing.
```

```
            self._pending = None
```

```
            self._pending_frames_elapsed = 0
```

```
            self.time_since_update += 1
```

```
        # else: track does not age during an active hysteresis window
```

```
    else:
```

```
        self.time_since_update += 1
```

```

def update(self, z, R=None):
    # R: optional per-detection measurement covariance (adaptive sigma_Z).
    # None -> the IMM filters use their own fixed self.R.
    current_mah_sq = self._last_mahalanobis_sq

    # Delayed-accept hysteresis. Defer (pending) runs only with
    # enable_hysteresis. Coast-skip (via _missed_streak) is always active:
    # it handles return from occlusion, not detection deferral.
    _coast_skip = False # True -> skip inflate and the hysteresis trigger

    if self.enable_hysteresis and self._pending is not None:
        pending_z, _ = self._pending
        self._pending = None
        self._pending_frames_elapsed = 0

    if current_mah_sq >= self.hysteresis_confirm_threshold:
        # Both frames far from the prediction -> real manoeuvre. Inflate
        # P and update with the current z. pending_z (the late
        # measurement) is not applied: a second imm.update() without a
        # predict() between them breaks PD of P (Cholesky fail).
        self._inflate_covariance()
        self.imm.update(z, R=R)
        self.time_since_update = 0
        self.hits += 1
        self.hit_streak += 1
        if self.state == 'Tentative' and self.hits >= self.min_hits:
            self.state = 'Confirmed'

```

```

        self._missed_streak = 0
        return

    # mah_sq < confirm threshold -> the detection returned to a normal
    # trajectory -> pending was a false positive. Continue with a normal
    # update, no inflate and no new trigger.
    _coast_skip = True
    self._missed_streak = 0

    elif self._missed_streak >= self.coast_threshold:
        # Long miss streak -> high mah_sq is expected (P grew naturally while
        # coasting), not a manoeuvre. Skip inflate and the hysteresis trigger.
        _coast_skip = True
        self._missed_streak = 0

    elif (self.enable_hysteresis and z is not None
          and current_mah_sq > self.hysteresis_mah_sq_threshold):
        # Suspicious detection: defer it for confirmation.
        self._pending = (np.asarray(z, dtype=float).copy(), current_mah_sq)
        self._pending_frames_elapsed = 0
        self.imm.update(None) # extrapolate instead of accepting
        self.hit_streak = 0
        self._last_mahalanobis_sq = 0.0
        self._missed_streak += 1
        return

    # Mechanism A — intra-track velocity bootstrap.
    # On the hits=1 -> 2 transition the UKFs still had v=0 in the prior,

```

```
# which inflates the residual/Mahalanobis and makes the Kalman gain pull
# velocity to truth slowly. We compute v_bootstrap from the finite
# difference (z - z_initial)/(gap*dt) and write it into every filter
# before imm.update, so predict/update run with a correct velocity prior
# from the very first use.
```

```
do_bootstrap = (
```

```
    z is not None
```

```
    and self.hits == 1
```

```
    and not self._velocity_bootstrapped
```

```
    and self.time_since_update > 0
```

```
)
```

```
if do_bootstrap:
```

```
    gap_dt = self.time_since_update * self.dt
```

```
    z_arr = np.asarray(z, dtype=float)
```

```
    v_boot = (z_arr - self.z_initial) / gap_dt
```

```
    speed = float(np.linalg.norm(v_boot))
```

```
    if 0.0 < speed <= self.bootstrap_max_speed:
```

```
        # Write velocity into all sub-filters and their x_prior (which
```

```
        # seeds the next predict sigma points). Accel (2, 5, 8) is left
```

```
        # at 0 — there is no basis to bootstrap it. Velocity indices are
```

```
        # [1, 4, 7] in the 9D state.
```

```
        for f in self.imm.filters:
```

```
            f.x[1] = v_boot[0]
```

```
            f.x[4] = v_boot[1]
```

```
            f.x[7] = v_boot[2]
```

```
            f.x_prior[1] = v_boot[0]
```

```
            f.x_prior[4] = v_boot[1]
```

```
            f.x_prior[7] = v_boot[2]
```

```

# Update the mixed state too.
self.imm.x[1] = v_boot[0]
self.imm.x[4] = v_boot[1]
self.imm.x[7] = v_boot[2]
self.imm.x_prior[1] = v_boot[0]
self.imm.x_prior[4] = v_boot[1]
self.imm.x_prior[7] = v_boot[2]
self._velocity_bootstrapped = True

# Step 2.A — track-level covariance reset on a radical residual. Runs
# after the bootstrap (the bootstrap frame is not a hit) but before
# imm.update(z). _coast_skip means the high mah_sq is expected, so no
# inflate is needed.
do_cov_reset = (
    self.enable_cov_reset
    and z is not None
    and not _coast_skip
    and self._last_mahalanobis_sq > self.cov_reset_mah_sq_threshold
)
if do_cov_reset:
    self._inflate_covariance()

self.imm.update(z, R=R)
self.time_since_update = 0
self.hits += 1
self.hit_streak += 1
if self.state == 'Tentative' and self.hits >= self.min_hits:
    self.state = 'Confirmed'

```

```

self._missed_streak = 0

def mark_missed(self):
    self.hit_streak = 0
    # No measurement -> residual undefined. Reset to 0 so the next predict()
    # does not fire the hit trigger from stale data.
    self._last_mahalanobis_sq = 0.0
    self._missed_streak += 1
    self.imm.update(None)    # extrapolate without a measurement
    # Damp residual accel so coasting does not accelerate by integrating
    # stale ax into velocity.
    self._collapse_accel_to_ballistic()
    # Gate C: cap P-diagonal growth while coasting (keeps extrapolation
    # physical past the frame edge).
    if self.enable_coast_cov_cap:
        self._cap_coast_covariance()
    # Coast Z-fence: keep extrapolated depth physical (<= court size).
    if self.enable_coast_z_fence:
        self._fence_coast_depth()

def get_mahalanobis_distance(self, z, R_matrix):
    """Mahalanobis distance between the IMM prediction and a new detection."""
    # Project the mixed prior state x_prior and covariance P_prior into
    # measurement space.
    z_mean = np.dot(self.h_matrix, self.imm.x_prior)
    S = np.dot(self.h_matrix, np.dot(self.imm.P_prior,
                                     self.h_matrix.T)) + R_matrix

```



```
y = z - z_mean # innovation
```

```
try:
```

```
    S_inv = np.linalg.inv(S)
```

```
    dist_sq = np.dot(y.T, np.dot(S_inv, y))
```

```
    return dist_sq
```

```
except np.linalg.LinAlgError:
```

```
    return 1e5
```

```
class IMMTracker:
```

```
    def __init__(self, dt=0.02, max_age=80, min_hits=3, gating_threshold=16.0,
        bootstrap_max_gap_frames=5, bootstrap_max_dist=2.5,
        bootstrap_max_speed=35.0, enable_hysteresis=False,
        enable_cov_reset=False, enable_single_ball_nms=True,
        enable_physical_gate=True, max_assoc_residual=3.0,
        enable_coast_cov_cap=False, spawn_suppress_max_coast=0,
        enable_depth_robust_gate=False, depth_hold_gain=0.5,
        depth_innov_free=0.0,
        enable_coast_z_fence=False, coast_z_min=-3.0, coast_z_max=21.0,
        tentative_max_age=5, floor_kill_y=-0.3,
        enable_adaptive_depth_R=False,
        hit_residual_min_sq=8.0, hit_residual_max_sq=11.34,
        bounce_height_max=0.55, bounce_max_coast=3,
        m_hit_target=None):
```

```
        """
```

```
        dt:                time step (= 1 / FPS).
```

```
        max_age:           frames without an update before a track is deleted.
```

min_hits: frames to confirm a track (Tentative -> Confirmed).

gating_threshold: Mahalanobis χ^2 gate. $\chi^2_3 p=0.99 = 11.34$;
 16.0 admits borderline manoeuvre detections
 (mah 12-15) to revive the hit mode without growth.

bootstrap_max_gap_frames: max gap (frames) between a pending detection
 and the current one to bootstrap velocity. Default 5.

bootstrap_max_dist: max 3D distance (m) between pending and current
 detection for bootstrap. Default 2.5 m.

bootstrap_max_speed: sanity cap (m/s); above it bootstrap is skipped
 (likely a false link of two objects). Default 35.

enable_hysteresis: delayed-accept hysteresis per Track. Default False
 (defer cannibalises cov-reset + hit).

enable_cov_reset: Step 2.A covariance reset per Track. Default False
 (Path B) to keep the IMM hit mode alive.

enable_single_ball_nms: one ball -> at most one Confirmed track. Extra
 confirmed tracks (phantom duplicates) are removed,
 keeping the best. Default True.

enable_physical_gate: Gate A. A covariance-independent ceiling on the
 Euclidean residual $\|z - \text{prediction}\|$. The Mahalanobis
 gate depends on P, which inflates while coasting and
 lets an 8 m FP jump pass; a fixed ceiling does not.
 Default True.

max_assoc_residual: Gate A threshold (m). Real-manoevrue residuals reach
 ~ 2.2 m; the nearest teleport jumps to ~ 6 m, so 3.0 m
 cleanly separates them.

~~~~~

self.dt = dt

self.max\_age = max\_age

```

self.min_hits = min_hits
self.gating_threshold = gating_threshold

self.bootstrap_max_gap_frames = int(bootstrap_max_gap_frames)
self.bootstrap_max_dist = float(bootstrap_max_dist)
self.bootstrap_max_speed = float(bootstrap_max_speed)
self.enable_hysteresis = bool(enable_hysteresis)
self.enable_cov_reset = bool(enable_cov_reset)
self.enable_single_ball_nms = bool(enable_single_ball_nms)
# Gate A: covariance-independent physical ceiling on the residual.
self.enable_physical_gate = bool(enable_physical_gate)
self.max_assoc_residual = float(max_assoc_residual)
# Gate C: cap P-diagonal growth while coasting (forwarded to Track).
self.enable_coast_cov_cap = bool(enable_coast_cov_cap)
# Gate D: suppress spawning a parasite track while the real Confirmed
# track is briefly coasting (0 < tsu <= spawn_suppress_max_coast). One
# ball domain: a jumpy occluded detection used to spawn a competitor that
# then stole identity. 0 = disabled (A/B compatibility); ~8 covers a
# typical hand occlusion at 50 FPS.
self.spawn_suppress_max_coast = int(spawn_suppress_max_coast)
# Gate A-depth: depth-robust Gate A. Split the residual into lateral
# (X, Y — the pixel projection YOLO gives correctly even under occlusion)
# and depth (Z — monocular f*D/w, which degrades as w collapses). Gate
# only the lateral part hard (hypot(dX,dY) <= max_assoc_residual) and,
# instead of rejecting a large depth innovation, strongly distrust it by
# inflating sigma_Z^2. Depth is then held by the prediction while X,Y
# update from the valid detection. Default False (A/B compatibility).
self.enable_depth_robust_gate = bool(enable_depth_robust_gate)

```

```

self.depth_hold_gain = float(depth_hold_gain)

# Huber-style dead zone for depth distrust: damp only the excess of
# |innov| over a normal-physics threshold. excess = max(0, |innov| -
# depth_innov_free); sigma_Z^2 += (gain*excess)^2. Pure (gain*innov)^2
# also damped legitimate depth (far side, serve); ~3 m dead zone passes
# normal physics and suppresses only occlusion outliers (6-11 m).
# 0.0 = disabled. Recommended ~3.0 (= max_assoc_residual).
self.depth_innov_free = float(depth_innov_free)

# Coast Z-fence: while coasting no gate acts and predict() extrapolates
# world Z from a contaminated vz (w_box collapses near the frame edge ->
# Z=f*D/w inflates -> vz up to +/-19 m/s, Z carried to 33-42 m on an 18 m
# court). The fence in each track's mark_missed clamps predicted Z to
# [coast_z_min, coast_z_max] and damps vz/az. Default False. Recommended
# coast_z_max~21 (back line 18 m + ~3 m for servers), coast_z_min~-3.
self.enable_coast_z_fence = bool(enable_coast_z_fence)

self.coast_z_min = float(coast_z_min)
self.coast_z_max = float(coast_z_max)

# Defect 1: ghost tracks. A Tentative track that never reached min_hits
# must not coast as a full track — 1-2 noise detections otherwise draw
# an ~80-frame fiction. Kill Tentative tracks after this many coast frames.
self.tentative_max_age = int(tentative_max_age)

# Defect 2: sub-floor coasting. A ball in flight cannot be below the
# floor; if a coasting track extrapolates y < floor_kill_y, terminate it.
# Small negative margin against floor noise (ball r ~0.1 m).
self.floor_kill_y = float(floor_kill_y)

# Adaptive sigma_Z. If True and detection_covs is passed to update(),
# gating and the IMM update use per-detection 3x3 R (anisotropic, along
# the ray; sigma_Zc = f*D/w^2 * sigma_w) instead of the fixed

```

```

# self.R_matrix. Default False (A/B compatibility).
self.enable_adaptive_depth_R = bool(enable_adaptive_depth_R)
# IMM triggers, forwarded to every Track -> get_dynamic_transition_matrix.
# Allow tuning bounce/hit activation from the CLI. Defaults match the
# function signature (A/B compatibility).
self.hit_residual_min_sq = float(hit_residual_min_sq)
self.hit_residual_max_sq = float(hit_residual_max_sq)
self.bounce_height_max = float(bounce_height_max)
self.bounce_max_coast = int(bounce_max_coast)
self.m_hit_target = (None if m_hit_target is None
                      else np.asarray(m_hit_target, dtype=float))
# Gate B: merge-identity guard. Two Confirmed tracks inherit a shared id
# only if physically reconcilable (same ball split by a manoeuvre, within
# this distance). A distant FP track that reached Confirmed does not
# donate its id to the real coasting track (that would amplify teleports).
# Manoeuvre fragments stay <=~2 m, FP teleports >=6 m, so 4.0 m separates.
self.single_ball_merge_max_dist = 4.0
# Coasting <= this many frames does not cost identity (we bucket tsu).
# tol=1: a real track survives one missed frame (detection noise) but
# after 2+ coast frames yields to an actively-detected track. A small tol
# minimises the spatial jump at a switch.
self.single_ball_nms_tsu_tol = 1

self.tracks = []
self.next_id = 1
# Matrix for Mahalanobis gating, consistent with R_init in
# create_imm_estimator (sigma_z ~ 0.7 m for monocular depth).
self.R_matrix = np.diag([0.02, 0.02, 0.5])

```

```

# Ring buffer of unassigned detections from the last few frames. Used to
# bootstrap velocity when spawning a new track: if the current
# unassigned detection is close to one from the last N frames, init the
# track with  $v = (z\_now - z\_prev)/(dframe*dt)$  instead of zeros.
self._spawn_buffer = [] # elements: (np.array z, int frame_idx)
self._frame_counter = 0

def update(self, detections_3d, detection_covs=None):
    """
    detections_3d: list of np.array([x, y, z]) for all detected objects
    in the frame.
    detection_covs: optional parallel list of 3x3 measurement covariances
    (one per detection). Used only if enable_adaptive_depth_R=True;
    None -> fixed self.R_matrix.
    """
    self._frame_counter += 1

    def _R_for(d_idx):
        """R for detection d: adaptive if enabled and provided."""
        if (self.enable_adaptive_depth_R
            and detection_covs is not None
            and detection_covs[d_idx] is not None):
            return detection_covs[d_idx]
        return self.R_matrix

    def _depth_robust_R(track, z, R_base):
        """Inflate  $\sigma_Z^2$  proportional to the squared depth innovation so

```

a large depth jump barely moves the filter (depth held by the prediction, X,Y updated from the valid detection). z\_pred from x\_prior (predict already ran for all tracks)."""

```
z_pred = np.dot(track.h_matrix, track.imm.x_prior)
```

```
depth_innov = float(z[2] - z_pred[2])
```

```
# Dead zone: damp only the excess of |innov| over normal physics.
```

```
excess = max(0.0, abs(depth_innov) - self.depth_innov_free)
```

```
R_eff = np.array(R_base, dtype=float, copy=True)
```

```
R_eff[2, 2] += (self.depth_hold_gain * excess) ** 2
```

```
return R_eff
```

```
for track in self.tracks:
```

```
    track.predict()
```

```
if len(detections_3d) == 0:
```

```
    for track in self.tracks:
```

```
        track.mark_missed()
```

```
    self._age_spawn_buffer()
```

```
    self._manage_lifecycle()
```

```
    return
```

```
if len(self.tracks) == 0:
```

```
    # No tracks yet. Any detection may bootstrap if the spawn buffer has
```

```
    # a predecessor.
```

```
    for z in detections_3d:
```

```
        self._spawn_track_from_unmatched(z)
```

```
    self._age_spawn_buffer()
```

```
    return
```

```

cost_matrix = np.full((len(self.tracks), len(detections_3d)), 1e5)

for t, track in enumerate(self.tracks):
    # Gate A reference point: the track's predicted position h*x_prior.
    z_pred = np.dot(track.h_matrix, track.imm.x_prior)
    for d, z in enumerate(detections_3d):
        R_d = _R_for(d)
        # Gate A: reject a pair whose Euclidean jump from the prediction
        # is physically impossible in one step, independent of how much P
        # inflated while coasting (that is what opened the Mahalanobis
        # gate for teleports).
        if self.enable_physical_gate:
            if self.enable_depth_robust_gate:
                # Gate A-depth: gate only the lateral (X,Y) part. The
                # large depth innovation is not rejected but handled by
                # depth distrust (inflated sigma_Z in gating and update),
                # so a detection with valid u,v is accepted.
                lateral = float(np.hypot(z[0] - z_pred[0],
                                          z[1] - z_pred[1]))
                if lateral > self.max_assoc_residual:
                    continue

                R_d = _depth_robust_R(track, z, R_d)
            elif np.linalg.norm(z - z_pred) > self.max_assoc_residual:
                continue

            dist_sq = track.get_mahalanobis_distance(z, R_d)
            if dist_sq < self.gating_threshold:
                cost_matrix[t, d] = dist_sq

```



```

# Association (Hungarian algorithm).
row_ind, col_ind = linear_sum_assignment(cost_matrix)

unmatched_tracks = set(range(len(self.tracks)))
unmatched_detections = set(range(len(detections_3d)))

# Update matched pairs.
for r, c in zip(row_ind, col_ind):
    if cost_matrix[r, c] < self.gating_threshold:
        # Save Mahalanobis^2 at accept time; it feeds
        # get_dynamic_transition_matrix on the next predict().
        self.tracks[r]._last_mahalanobis_sq = float(
            cost_matrix[r, c]
        )
        R_c = _R_for(c)
        if self.enable_depth_robust_gate:
            # Same inflated sigma_Z as in gating: depth held by the
            # prediction, X,Y updated from the detection.
            R_c = _depth_robust_R(self.tracks[r],
                                   detections_3d[c], R_c)
        self.tracks[r].update(detections_3d[c], R=R_c)
        unmatched_tracks.remove(r)
        unmatched_detections.remove(c)

# Missed tracks.
for t in unmatched_tracks:
    self.tracks[t].mark_missed()

```

```

# Spawn new tracks for unmatched detections (with velocity bootstrap if a
# predecessor is found). Gate D: if a real Confirmed track is briefly
# coasting ( $0 < tsu \leq \text{spawn\_suppress\_max\_coast}$ ), do not spawn a
# competitor from outlier detections — and keep the outlier out of the
# spawn buffer so a later legitimate spawn starts with zero velocity.
suppress_spawn = (
    self.spawn_suppress_max_coast > 0
    and any(
        t.state == 'Confirmed'
        and  $0 < t.time\_since\_update \leq \text{self.spawn\_suppress\_max\_coast}$ 
        for t in self.tracks
    )
)
if not suppress_spawn:
    for d in unmatched_detections:
        self._spawn_track_from_unmatched(detections_3d[d])

# Age the spawn buffer and remove stale tracks.
self._age_spawn_buffer()
self._manage_lifecycle()

# -----
# Initial-velocity bootstrap helpers
# -----

def _spawn_track_from_unmatched(self, z_now):
    """Create a new Track for the unassigned detection z_now. If the spawn
    buffer holds a predecessor that satisfies the constraints (close in

```

space, recent in time, consistent velocity), init the track with  
 $v\_initial = (z\_now - z\_prev)/(dframe*dt)$  and hits=2; otherwise a plain  
 zero-velocity init. The used spawn point is removed (one-to-one) to avoid  
 gluing two tracks to the same historical detection.””””

```
z_now_arr = np.asarray(z_now, dtype=float)
```

```
best_idx = -1
```

```
best_dist = self.bootstrap_max_dist
```

```
best_v = None
```

```
best_gap = 0
```

```
for i, (z_prev, f_prev) in enumerate(self._spawn_buffer):
```

```
    gap = self._frame_counter - f_prev
```

```
    if gap <= 0 or gap > self.bootstrap_max_gap_frames:
```

```
        continue
```

```
    d_xyz = float(np.linalg.norm(z_now_arr - z_prev))
```

```
    if d_xyz >= best_dist:
```

```
        continue
```

```
    gap_dt = gap * self.dt
```

```
    if gap_dt <= 0:
```

```
        continue
```

```
    v_cand = (z_now_arr - z_prev) / gap_dt
```

```
    speed = float(np.linalg.norm(v_cand))
```

```
    if speed > self.bootstrap_max_speed:
```

```
        # Velocity beyond physically possible -> no bootstrap.
```

```
        continue
```

```
    best_idx = i
```

```
    best_dist = d_xyz
```

```
    best_v = v_cand
```

```

best_gap = gap

if best_idx >= 0:
    # Bootstrap: create a track with non-zero v and hits=2.
    self.tracks.append(
        Track(self.next_id, z_now_arr, self.dt,
            v_initial=best_v, initial_hits=2,
            min_hits=self.min_hits,
            enable_hysteresis=self.enable_hysteresis,
            enable_cov_reset=self.enable_cov_reset,
            enable_coast_cov_cap=self.enable_coast_cov_cap,
            enable_coast_z_fence=self.enable_coast_z_fence,
            coast_z_min=self.coast_z_min,
            coast_z_max=self.coast_z_max,
            hit_residual_min_sq=self.hit_residual_min_sq,
            hit_residual_max_sq=self.hit_residual_max_sq,
            bounce_height_max=self.bounce_height_max,
            bounce_max_coast=self.bounce_max_coast,
            m_hit_target=self.m_hit_target)
    )
    self.next_id += 1
    # Remove the used spawn point.
    del self._spawn_buffer[best_idx]
else:
    # Plain zero-velocity spawn + buffer the detection for a future
    # bootstrap.
    self.tracks.append(
        Track(self.next_id, z_now_arr, self.dt,

```

```

        min_hits=self.min_hits,
        enable_hysteresis=self.enable_hysteresis,
        enable_cov_reset=self.enable_cov_reset,
        enable_coast_cov_cap=self.enable_coast_cov_cap,
        enable_coast_z_fence=self.enable_coast_z_fence,
        coast_z_min=self.coast_z_min,
        coast_z_max=self.coast_z_max,
        hit_residual_min_sq=self.hit_residual_min_sq,
        hit_residual_max_sq=self.hit_residual_max_sq,
        bounce_height_max=self.bounce_height_max,
        bounce_max_coast=self.bounce_max_coast,
        m_hit_target=self.m_hit_target)
    )
    self.next_id += 1
    self._spawn_buffer.append(
        (z_now_arr.copy(), self._frame_counter)
    )

```

```
def _age_spawn_buffer(self):
```

```
    """Drop buffer entries older than bootstrap_max_gap_frames."""
```

```
    cutoff = self._frame_counter - self.bootstrap_max_gap_frames
```

```
    self._spawn_buffer = [
```

```
        (z, f) for (z, f) in self._spawn_buffer if f > cutoff
```

```
    ]
```

```
def _manage_lifecycle(self):
```

```
    active_tracks = []
```

```
    for track in self.tracks:
```

```

# Delete a track that has not updated for too long.
if track.time_since_update > self.max_age:
    track.state = 'Deleted'

# Defect 1: ghost track. Tentative (never confirmed) but already
# coasting past tentative_max_age frames -> noise spawn. Confirmed
# tracks are untouched.
elif (track.state == 'Tentative'
      and track.time_since_update > self.tentative_max_age):
    track.state = 'Deleted'

# Defect 2: sub-floor coast. Only when the track has no measurement
# (coasting): with a detection y is measured and no clamp is needed.
elif (track.time_since_update > 0
      and track.imm.x[3] < self.floor_kill_y):
    track.state = 'Deleted'

# Keep the track unless deleted.
if track.state != 'Deleted':
    active_tracks.append(track)

self.tracks = active_tracks

if self.enable_single_ball_nms:
    self._suppress_duplicate_confirmed()

def _suppress_duplicate_confirmed(self):
    """Single-ball NMS. One ball in play -> at most one Confirmed track. If

```

several exist (phantom duplicates from noise spawns or divergent coasting), keep the best and DELETE the rest.

Deletion (not demotion) is deliberate: a demoted duplicate keeps stealing detections in the Hungarian step, re-confirms and competes again, producing identity switches. Deletion sends detections to the winner; if the winner truly diverges, a fresh detection spawns a new track (slightly higher fragmentation, much smoother trajectory: jerk down, switches 9->3).

Best = (min bucketed-tsu, max hits, min last Mahalanobis<sup>2</sup>): actively updated > long-lived > well-fitted.

tsu bucketing: a short coast ( $tsu \leq tsu\_tol$ ) counts as 0. Without it one missed frame on a real track ( $tsu=1$ ) would hand identity to a divergent ghost with  $tsu=0$ , causing identity flicker during manoeuvres. With the bucket the real track keeps identity through short turbulence; a ghost with a LONG coast ( $tsu > tsu\_tol$ ) still yields to the active track. """

```
confirmed = [t for t in self.tracks if t.state == 'Confirmed']
```

```
if len(confirmed) < 2:
```

```
    return
```

```
def _key(t):
```

```
    eff_tsu = 0 if t.time_since_update <= self.single_ball_nms_tsu_tol \
        else t.time_since_update
```

```
    return (eff_tsu, -t.hits, t._last_mahalanobis_sq)
```

```
confirmed.sort(key=_key)
```

```
winner = confirmed[0]
```

```

# Merge-identity (fragmentation fix): one ball domain, so all coexisting
# Confirmed tracks are the same physical ball split by a manoeuvre. The
# winner inherits the minimum id of the group so logical identity is
# continuous across the seam. Tracks split by a real trajectory break
# (different rallies) never coexist as Confirmed, so no false merge.
# Carry the max hits too, so the merged track stays a stable NMS winner.
#
# Gate B (merge guard): inherit an id only from physically reconcilable
# members (<= single_ball_merge_max_dist from the winner). A distant FP
# track that reached Confirmed does not donate its id (that would
# legalise a teleport); it is simply deleted as a duplicate.
def _pos(t):
    x = t.imm.x_post
    return np.array([x[0], x[3], x[6]])

wp = _pos(winner)
reconcilable = [
    t for t in confirmed
    if np.linalg.norm(_pos(t) - wp) <= self.single_ball_merge_max_dist
]
winner.track_id = min(t.track_id for t in reconcilable)
winner.hits = max(t.hits for t in reconcilable)

for loser in confirmed[1:]:
    loser.state = 'Deleted'
self.tracks = [t for t in self.tracks if t.state != 'Deleted']

```



```
def get_confirmed_tracks(self):
```

```
    """Return only the tracks we are confident in (Confirmed state)."""
```

```
    return [t for t in self.tracks if t.state == 'Confirmed']
```

### **training\_models/model\_train\_remote.py**

```
import os
```

```
import torch as t
```

```
import torch.nn as nn
```

```
import torch.nn.functional as F
```

```
from pathlib import Path
```

```
from torchvision import transforms
```

```
from torchvision.transforms import Compose, ToTensor, PILToTensor, Resize
```

```
from tqdm import tqdm
```

```
import numpy as np
```

```
# import optuna as opt
```

```
from ultralytics import YOLO
```

```
import albumentations as A
```

```
main_model = YOLO('../parsing_datasets_scripts/yolo26m.pt')
```

```
custom_transforms = [
```

```
    A.MotionBlur(blur_limit=(7, 15), p=0.4),
```

```
    A.GaussNoise(std_range=(0.01, 0.05), p=0.2),
```

```
    A.CLAHE(clip_limit=4.0, p=0.3),
```

```
    A.RandomBrightnessContrast(brightness_limit=0.25,
```

```
                               contrast_limit=0.25, p=0.4),
```

```
    A.HueSaturationValue(hue_shift_limit=15, sat_shift_limit=30,
```

|     |      |          |       |      |                    |       |
|-----|------|----------|-------|------|--------------------|-------|
|     |      |          |       |      | ІА/Ц.467200.007 Д4 | Аркуш |
|     |      |          |       |      |                    | 67    |
| Зм. | Лист | № докум. | Підп. | Дата |                    |       |

```

        val_shift_limit=20, p=0.3),
    ]

    # Purely for server training
    main_model.train(data='/workspace/diploma_project/data/server_data.yaml',
                     epochs=100, device='0', imgsz=640,
                     augmentations=custom_transforms, patience=20)

    metrics = main_model.val(data='/workspace/diploma_project/data'
                             '/server_data.yaml', imgsz=640)

    print(f'mAP50: {metrics.box.map50}')
    print(f'mAP50-95: {metrics.box.map}')

```